

© 2026

Matan Y. Shachnai

ALL RIGHTS RESERVED

PRACTICAL FORMAL METHODS FOR THE LINUX EBPf VERIFIER: SCALABLE  
VERIFICATION AND WITNESS GENERATION

By

MATAN Y. SHACHNAI

A dissertation submitted to the

School of Graduate Studies

Rutgers, The State University of New Jersey

In partial fulfillment of the requirements

For the degree of

Doctor of Philosophy

Graduate Program in Computer Science

Written under the direction of

Santosh Nagarakatte

And approved by

---

---

---

---

---

New Brunswick, New Jersey

October 2026

## ABSTRACT OF THE DISSERTATION

Practical Formal Methods for the Linux eBPF Verifier: Scalable Verification and Witness  
Generation

by MATAN Y. SHACHNAI

Dissertation Director:  
Santosh Nagarakatte

The eBPF framework allows developers to extend the functionality of the Linux kernel without directly modifying kernel source code. It enables user-supplied programs to execute within the kernel for tasks such as networking, tracing, observability, access control, load balancing, and security monitoring. To preserve kernel safety, eBPF combines restricted execution with static verification. Before an eBPF program is loaded into the kernel, it is statically analyzed by the Linux eBPF verifier, which checks safety requirements such as termination, safe memory access, and well-defined arithmetic behavior. Once accepted, an eBPF program may be just-in-time compiled and executed as part of the kernel. The verifier must therefore soundly approximate the behavior of accepted programs while remaining precise enough to accept useful ones. Bugs in the verifier can cause it to accept programs whose runtime behavior is not safely captured by its static analysis, allowing unsafe programs to execute in the kernel context. This risk has materialized in practice through security vulnerabilities in value tracking, pointer reasoning, and state pruning. At the same time, imprecision in the verifier’s analysis can cause it to reject safe programs. The verifier must therefore be sound, precise, and efficient for use in a production kernel.

This dissertation focuses on value tracking, a central part of the verifier’s static analysis. Value tracking approximates the possible scalar values held in registers and program variables, and the

verifier uses this information to prove properties such as well-defined arithmetic, valid bounds checks, and safe memory access. Establishing the correctness and precision of this logic is challenging because it is implemented as handwritten, optimized kernel code that evolves over time. The verifier uses abstract interpretation to track possible values across several domains, including signed and unsigned interval domains for 64-bit and 32-bit values and the bitwise tnum domain. However, the verifier does not keep these domains cleanly separated. Its abstract operators combine information from multiple domains and rely on a shared reduction operator to improve precision across domains. These interactions improve precision, but they make formal reasoning harder because they are embedded in the production implementation rather than expressed as the standard modular abstractions typically used in abstract interpretation. We address this challenge by developing formal techniques for reasoning about the correctness and precision of the eBPF verifier’s value-tracking logic as implemented in the Linux kernel.

We first address the scalability of implementation-level verification. To check whether a verifier abstract operator soundly overapproximates the corresponding concrete eBPF instruction, existing techniques translate the verifier’s kernel C implementation and a formal soundness specification into SMT formulas, which are then checked by an SMT solver. Prior work encoded each full abstract operator together with the shared reduction operator as a single monolithic proof obligation. This approach repeatedly encoded the same reduction reasoning for different instructions, making verification increasingly expensive across kernel versions. We instead decompose each full operator into an instruction-specific operator and a shared reduction operator. Each instruction-specific operator is verified independently, the shared reduction operator is verified once, and the results are composed to establish soundness of the full operator. This decomposition reduces solver cost across kernel versions and reveals latent unsoundness in several bitwise operators. We identify and patch the affected operators so that the instruction-specific operators satisfy the modular soundness specification independently.

We then introduce differential synthesis, an SMT-based approach for generating executable witnesses of unsound verifier behavior. A failed soundness check can show that an abstract op-

erator is unsound for some abstract inputs, but those inputs may not arise during the analysis of any real eBPF program because the verifier starts from a restricted set of initial values. Differential synthesis bridges this gap by searching for bounded eBPF instruction sequences that drive the verifier to the relevant abstract inputs and then execute an instruction that exposes the mismatch between concrete execution and the verifier’s abstract execution. The resulting programs turn abstract verification failures into reproducible evidence that developers can inspect, debug, and use for regression testing.

Finally, we extend the framework from soundness to precision. Soundness is required for verifier safety, but a sound operator can still be too imprecise and cause the verifier to reject safe programs. This matters because verifier operators are handwritten and change over time as developers improve the verifier’s reasoning. When an operator changes, developers need to check that the new implementation remains sound and does not lose precision relative to the old one. We formalize precision comparison for verifier operators and adapt differential synthesis to generate executable precision witnesses. These witnesses are eBPF programs that reach verifier states where one sound operator is strictly more precise than another. We use this framework to evaluate precision-improving kernel patches, measure the precision gained from the shared reduction operator, and develop a more precise multiplication operator that has been upstreamed to the Linux kernel.

## ACKNOWLEDGMENTS

To my advisor, Santosh Nagarakatte, I am eternally grateful for all your support, for teaching me how to critically evaluate scientific work and produce academically rigorous research, and for showing me the importance of telling a good story. To Srinivas Narayana, my official-unofficial advisor, I am grateful for your astute insights and ideas in our many discussions, for your unyielding positivity, and for lending a helping hand and a kind word when my resolve wavered.

To my dissertation committee, He Zhu, Rich Martin, and Dongyoon Lee, I am thankful for your thoughtful feedback and guidance, as well as your belief in the value of this work.

To Hari Vishwanathan, my brother in the Ph.D. trenches—the inordinate amount of time we spent building, debugging, bouncing ideas, and laughing together, has made this journey infinitely better. I’d also like to thank my past and present collaborators and lab mates at the RAPL group, Jay Lim, Mohammadreza Soltaniyeh, Sangeeta Chowdhary, Sehyeok Park, Alborz Jelvani, and Tung-Che Chang, for making trips to the lab enjoyable and thought provoking. I’d like to give special thanks to Jay for his mentorship, support, and kindness throughout the years.

To my family and friends, I am fortunate and grateful for all those who have shared with me their love, wisdom, and a laugh - especially the Tivon family. To my parents, Hadas and Boaz, I suspect no words will prevail here to express my love for you. To Reut, my abstract half, and Maayan, my concrete half—without you it’s a waste of time.

To my mother, Dr. Hadas Shachnai

## TABLE OF CONTENTS

|                                                                                    |      |
|------------------------------------------------------------------------------------|------|
| <b>Abstract</b> . . . . .                                                          | ii   |
| <b>Acknowledgments</b> . . . . .                                                   | v    |
| <b>List of Tables</b> . . . . .                                                    | xii  |
| <b>List of Figures</b> . . . . .                                                   | xiii |
| <b>Chapter 1: Introduction</b> . . . . .                                           | 1    |
| 1.1 The Linux eBPF Verifier . . . . .                                              | 1    |
| 1.2 Automated Verification of the eBPF Verifier . . . . .                          | 3    |
| 1.3 Dissertation Statement . . . . .                                               | 7    |
| 1.4 Contributions of the Dissertation . . . . .                                    | 8    |
| 1.5 Scalable Verification through Divide and Conquer . . . . .                     | 9    |
| 1.6 Differential Synthesis for Generating Unsound Witnesses . . . . .              | 10   |
| 1.7 Differential Synthesis for Comparing Precision of Abstract Operators . . . . . | 13   |
| 1.8 Contributions to This Dissertation . . . . .                                   | 15   |
| 1.9 Organization of This Dissertation . . . . .                                    | 16   |
| <b>Chapter 2: Background</b> . . . . .                                             | 17   |
| 2.1 The eBPF Framework . . . . .                                                   | 18   |

|       |                                                                              |           |
|-------|------------------------------------------------------------------------------|-----------|
| 2.2   | Static Analysis . . . . .                                                    | 20        |
| 2.3   | The eBPF Verifier . . . . .                                                  | 22        |
| 2.4   | Abstract Interpretation . . . . .                                            | 25        |
| 2.4.1 | Abstract Domains, Concretization, and Abstraction . . . . .                  | 25        |
| 2.4.2 | Abstract Operators and Soundness . . . . .                                   | 27        |
| 2.4.3 | Best Abstract Operators . . . . .                                            | 28        |
| 2.4.4 | Product Domains and Reductions . . . . .                                     | 29        |
| 2.5   | Satisfiability Modulo Theories . . . . .                                     | 30        |
| 2.6   | Value Tracking in the eBPF Verifier . . . . .                                | 32        |
| 2.6.1 | Abstract Domains for Value Tracking . . . . .                                | 32        |
| 2.6.2 | Abstract Operators for Value Tracking . . . . .                              | 34        |
| 2.7   | Soundness Specifications for eBPF Abstract Operators . . . . .               | 36        |
| 2.7.1 | Concrete and Abstract Values . . . . .                                       | 36        |
| 2.7.2 | Arithmetic and Logic Operators . . . . .                                     | 36        |
| 2.7.3 | Branch Operators . . . . .                                                   | 37        |
| 2.7.4 | Product-Domain Specifications . . . . .                                      | 38        |
| 2.8   | Implementation-Level Verification with Agni . . . . .                        | 39        |
| 2.8.1 | From Verifier Code to SMT Formulas . . . . .                                 | 40        |
| 2.8.2 | Implementation-Level Soundness Checking . . . . .                            | 41        |
| 2.8.3 | Refined Soundness Specification . . . . .                                    | 42        |
|       | <b>Chapter 3: Scalable Verification through Divide and Conquer . . . . .</b> | <b>44</b> |
| 3.1   | Implementation of Interval Domains in the eBPF Verifier . . . . .            | 45        |

|       |                                                                                     |           |
|-------|-------------------------------------------------------------------------------------|-----------|
| 3.2   | A Divide-and-Conquer Verification Strategy . . . . .                                | 46        |
| 3.2.1 | Soundness Specification for $op_g$ . . . . .                                        | 48        |
| 3.2.2 | Soundness Specification for $sro$ . . . . .                                         | 49        |
| 3.2.3 | Composing the Two Proof Obligations . . . . .                                       | 50        |
| 3.3   | Latent Unsoundness in Bitwise Operators . . . . .                                   | 51        |
| 3.3.1 | Root Cause of the Unsoundness . . . . .                                             | 51        |
| 3.3.2 | Patching Latent Unsoundness . . . . .                                               | 54        |
| 3.4   | Evaluating Modular Verification of Value-Tracking Abstract Operators . . . . .      | 55        |
| 3.4.1 | Experimental Setup . . . . .                                                        | 55        |
| 3.4.2 | Modular Verification of Patched Operators . . . . .                                 | 56        |
| 3.4.3 | Performance . . . . .                                                               | 56        |
| 3.5   | Summary . . . . .                                                                   | 58        |
|       | <b>Chapter 4: Differential Synthesis for Generating Unsound Witnesses . . . . .</b> | <b>59</b> |
| 4.1   | Differential Synthesis . . . . .                                                    | 60        |
| 4.1.1 | Notation . . . . .                                                                  | 61        |
| 4.1.2 | Single-Instruction Witnesses . . . . .                                              | 63        |
| 4.1.3 | Adding a Prefix . . . . .                                                           | 64        |
| 4.1.4 | General Synthesis Query . . . . .                                                   | 66        |
| 4.1.5 | Loop-Free Programs with Branches . . . . .                                          | 68        |
| 4.1.6 | Validation of Generated Witnesses . . . . .                                         | 70        |
| 4.2   | Guarantees and Limitations . . . . .                                                | 71        |
| 4.3   | Differential Synthesis in Practice . . . . .                                        | 71        |

|                                                                                              |                                                                                 |            |
|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|------------|
| 4.3.1                                                                                        | Generating Witness Programs for Unsound Kernels . . . . .                       | 72         |
| 4.3.2                                                                                        | Witness-Generation Performance . . . . .                                        | 73         |
| 4.3.3                                                                                        | Comparison with Existing Proofs of Concept . . . . .                            | 74         |
| 4.4                                                                                          | Summary . . . . .                                                               | 74         |
| <b>Chapter 5: Differential Synthesis for Comparing Precision of Abstract Operators . . .</b> |                                                                                 | <b>76</b>  |
| 5.1                                                                                          | Why Precision Matters in the eBPF Verifier . . . . .                            | 77         |
| 5.2                                                                                          | Reduction and Precision Comparison in the Verifier . . . . .                    | 79         |
| 5.2.1                                                                                        | Precision in the Verifier’s Product Domain . . . . .                            | 79         |
| 5.2.2                                                                                        | The Reduction Operator in the eBPF Verifier . . . . .                           | 81         |
| 5.2.3                                                                                        | Comparing the Precision of Abstract Operators in the Verifier . . . . .         | 82         |
| 5.3                                                                                          | Improving the Precision of Abstract Operators in the eBPF Verifier . . . . .    | 84         |
| 5.4                                                                                          | Precision Specifications and Witness Synthesis . . . . .                        | 89         |
| 5.4.1                                                                                        | Logical Specification for Precision Comparison . . . . .                        | 90         |
| 5.4.2                                                                                        | Reachability in the Verifier Abstract State Space . . . . .                     | 91         |
| 5.4.3                                                                                        | Synthesizing Precision Witnesses . . . . .                                      | 92         |
| 5.5                                                                                          | Evaluating Precision Comparison of Abstract Operators in the Verifier . . . . . | 97         |
| 5.5.1                                                                                        | Experimental Setup . . . . .                                                    | 97         |
| 5.5.2                                                                                        | Evaluating Precision-Related Kernel Patches . . . . .                           | 98         |
| 5.5.3                                                                                        | Generated Precision Witnesses . . . . .                                         | 99         |
| 5.5.4                                                                                        | Precision Impact of the Reduction Operator . . . . .                            | 100        |
| 5.6                                                                                          | Summary . . . . .                                                               | 101        |
| <b>Chapter 6: Related Work . . . . .</b>                                                     |                                                                                 | <b>103</b> |

|                   |                                                         |            |
|-------------------|---------------------------------------------------------|------------|
| 6.1               | eBPF Safety and Verification . . . . .                  | 103        |
| 6.1.1             | Formal Methods for eBPF Safety . . . . .                | 104        |
| 6.1.2             | Testing and Fuzzing the eBPF Verifier . . . . .         | 105        |
| 6.1.3             | eBPF Safety Beyond the Verifier . . . . .               | 106        |
| 6.2               | Abstract Interpretation and Domain Refinement . . . . . | 106        |
| 6.3               | Program Synthesis and Witness Generation . . . . .      | 107        |
| <b>Chapter 7:</b> | <b>Conclusion . . . . .</b>                             | <b>110</b> |
| 7.1               | Dissertation Summary . . . . .                          | 110        |
| 7.2               | Future Directions . . . . .                             | 112        |

## LIST OF TABLES

|     |                                                                            |     |
|-----|----------------------------------------------------------------------------|-----|
| 2.1 | eBPF instructions handled by Agni . . . . .                                | 40  |
| 3.1 | Verification results before and after the bitwise-operator patch . . . . . | 56  |
| 3.2 | Verification runtime for old and new strategies . . . . .                  | 57  |
| 4.1 | Witness generation results across kernel versions . . . . .                | 72  |
| 5.1 | Precision gains from the improved multiplication operator . . . . .        | 88  |
| 5.2 | Reachable and unreachable verifier abstract states . . . . .               | 92  |
| 5.3 | Precision comparison results for verifier operator changes . . . . .       | 98  |
| 5.4 | Precision gains from the verifier reduction operator . . . . .             | 101 |

## LIST OF FIGURES

|     |                                                                       |     |
|-----|-----------------------------------------------------------------------|-----|
| 2.1 | Basic eBPF workflow . . . . .                                         | 19  |
| 2.2 | Branch refinement in the eBPF verifier . . . . .                      | 23  |
| 3.1 | Monolithic and divide-and-conquer verification strategies . . . . .   | 48  |
| 3.2 | bpf_and update structure and source of unsoundness . . . . .          | 52  |
| 3.3 | Patch for the bitwise abstract operators . . . . .                    | 54  |
| 4.1 | Concrete and abstract semantics in a synthesized witness . . . . .    | 62  |
| 4.2 | Comparison of manually crafted and synthesized witnesses . . . . .    | 74  |
| 5.1 | Safe array access after masking an index register. . . . .            | 78  |
| 5.2 | Overview of precision comparison with a bounded prefix . . . . .      | 89  |
| 5.3 | Witness program demonstrating improved precision in bpf_mul . . . . . | 99  |
| 5.4 | Witness program demonstrating a precision gap in bpf_and . . . . .    | 100 |

## CHAPTER 1

### INTRODUCTION

The Linux eBPF framework [53] allows user-supplied programs to execute within the kernel for tasks such as networking, tracing, and security monitoring. To ensure safety, every eBPF program is statically analyzed by the kernel’s verifier before it is allowed to run. If the verifier accepts a program, the kernel assumes that it will behave safely at runtime. This design places the verifier in a critical role. It must accurately approximate program behavior in order to reject unsafe programs while accepting useful ones. As a result, the verifier is part of the kernel’s trusted computing base; errors in its logic can allow unsafe programs to execute with kernel privileges.

Ensuring the correctness of the eBPF verifier is therefore essential, but also challenging. The verifier is a production system that evolves over time and incorporates complex analysis logic. Understanding how the verifier enforces safety in practice, and how its correctness can be established at the level of its implementation, are central problems addressed in this work.

#### 1.1 The Linux eBPF Verifier

Modern operating systems increasingly support extending kernel functionality without directly modifying kernel source code. In the Linux kernel, eBPF has become a central mechanism for this purpose. eBPF programs are used in a wide range of systems contexts, including networking, observability, tracing, telemetry, access control, load balancing, security monitoring, and performance analysis [11, 19, 22, 23, 26, 55, 60, 74, 99, 103, 113]. These applications rely on the ability to execute user-supplied programs efficiently within the kernel, while preserving the safety and integrity of the operating system.

The flexibility of eBPF is made possible by a combination of restricted execution and static verification. eBPF programs are written for a register-based virtual machine with a 64-bit instruction set. Before an eBPF program is loaded into the kernel, it is checked by the Linux eBPF

verifier. The verifier statically analyzes the program to ensure that it satisfies safety requirements, including termination, safe memory access, well-defined arithmetic behavior, and restricted access to kernel memory and helper functions. Once accepted, an eBPF program may be just-in-time compiled to native machine code and executed as part of the kernel. Hence, the verifier serves as a bulwark to prevent incorrect eBPF programs from crashing or compromising the kernel once they are incorporated into its address space.

Because the verifier runs in a production kernel, its correctness is security-critical. A verifier bug can invalidate the safety argument for eBPF programs and may allow unsafe programs to be accepted. Such bugs are especially consequential because they occur at the boundary between untrusted program inputs and privileged kernel execution. This risk has materialized in practice; the eBPF verifier has repeatedly been the source of security vulnerabilities, including bugs in range tracking, arithmetic reasoning, pointer reasoning, and state pruning [3, 10, 12–18, 31–34, 50, 63–65]. Several of these bugs have enabled privilege-escalation attacks by causing the verifier to accept programs whose runtime behavior was not safely captured by its static analysis [76, 77, 87, 95]. Thus, the verifier is not merely an implementation of a static analysis algorithm, it is part of the trusted computing base of the eBPF ecosystem.

The eBPF verifier establishes program safety using abstract interpretation [48]. In particular, it performs value tracking for program variables and registers, and uses the resulting information to determine whether operations such as pointer arithmetic and memory accesses are safe. The verifier maintains several abstract domains for this purpose. Four of these domains are interval domains, tracking signed and unsigned bounds for both 64-bit and 32-bit values [47]. The verifier also uses a bit-level domain, known as the tristate number or tnum domain [108], to represent which bits of a value are known and which bits remain unknown. Together, these domains allow the verifier to overapproximate the possible values that may arise during execution.

However, the production verifier does not implement value tracking as a clean composition of independent abstract domains. In standard abstract interpretation literature [81], multiple domains are often combined through an explicit product construction, with reduction operators used

to propagate information between domains in a controlled way. Under such a design, one may reason about the soundness of each abstract domain and then reason separately about the soundness of their composition. The Linux eBPF verifier, by contrast, combines abstract-domain updates and cross-domain reductions in implementation-specific ways. Its abstract operators may update one domain using information from another, and shared reduction logic propagates information across domains after individual operations. Consequently, the correctness of the verifier cannot be established by considering each domain in isolation. We must also account for the interactions among domains, the concrete implementation of abstract operators, and the reduction procedures that connect them.

This structure creates a difficult verification problem. The verifier must be sound to reject unsafe programs, efficient enough to run in a live production kernel, and precise enough to accept useful programs. Its implementation reflects these competing constraints, it is highly optimized, evolves with the kernel, and incorporates domain interactions that are essential for precision. These features make the eBPF verifier an important and challenging target for formal methods. Verifying the eBPF static analyzer therefore requires techniques that account for the verifier’s production implementation, not only for simplified models of its abstract interpretation.

## 1.2 Automated Verification of the eBPF Verifier

The eBPF verifier relies on the soundness of its static analysis to decide whether a program may safely run in the kernel. The verifier maintains abstract states that summarize the possible values of program variables, and it updates these states using abstract operators that approximate the behavior of concrete eBPF instructions. These operators must overapproximate concrete execution; the abstract state computed by the verifier must include every runtime behavior that the program can exhibit. If this property fails, the verifier may miss a possible runtime behavior and accept a program whose execution is unsafe.

Establishing this property is difficult because the object of verification is the production verifier implemented in the Linux kernel, not a simplified mathematical model of the analysis. A

simplified model can clarify the intended design of the verifier, but it can miss implementation details where real bugs arise. Conversely, automated verification of kernel C code must faithfully model a codebase that is constantly evolving. The central challenge is therefore to connect the implementation-level behavior of the verifier to formal soundness specifications while remaining faithful to the deployed kernel code.

Prior work has addressed eBPF safety from several complementary directions. One line of work develops alternative analyzers or execution mechanisms that enforce safety outside the eBPF verifier. Another line tests the production verifier by generating eBPF programs that exercise its implementation and expose incorrect behavior. A third line seeks to verify parts of the verifier’s own analysis logic. These approaches serve different purposes. Alternative analyzers and execution mechanisms show that eBPF programs can be checked or executed safely under designs that differ from the production eBPF verifier. Testing reveals concrete bugs in the deployed verifier. Verification connects the verifier’s implementation to formal soundness properties.

PREVAIL [57] illustrates the first direction. It provides a formally grounded analyzer for eBPF programs outside the Linux kernel, using a relational abstract domain and a verification architecture distinct from the in-kernel verifier. Related work also applies formal methods to other parts of the eBPF pipeline, including trustworthy execution mechanisms and verified JIT compilation after verifier acceptance [84, 85, 112]. These approaches show how formal reasoning can support eBPF safety. Other work focuses on testing and validating the Linux verifier by generating eBPF programs that exercise its implementation. These include fuzzing-based and program-generation-based techniques that search for programs on which the verifier behaves incorrectly or inconsistently [66, 73, 88]. One such approach generates structured eBPF programs that are likely to pass the verifier, then uses runtime instrumentation or sanitization to detect whether the accepted programs can still trigger unsafe behavior [105]. Another approach validates the eBPF verifier by checking consistency between its abstract states and actual concrete execution states. The idea is to run a program to collect concrete states, then embed checks into a modified program that force the verifier to prove those states are included in its abstract approximations, where any mismatch

reveals a logic bug [104]. These techniques are valuable because they operate on the production Linux verifier and can expose real, executable bugs. However, they are primarily bug-finding techniques; they provide evidence of incorrect verifier behavior when they discover a witness program, but they do not by themselves establish formal soundness obligations for the verifier’s abstract operators.

These approaches do not answer whether the Linux verifier implementation itself can be formally verified. Harishankar et al. [108] verified the tnum abstract domain, which is used by the verifier to track bit-level uncertainty in register values. This work formalizes the tnum domain and manually proves soundness and optimality properties for several tnum arithmetic operators, including addition and subtraction, and develops a sound and more efficient multiplication operator that was merged into the Linux kernel. However, this verification is carried out by manually specifying and reasoning about the tnum operators, rather than by automatically verifying the C implementation as it appears in the kernel. It also focuses on a single abstract domain, and therefore does not address the verifier’s broader value-tracking analysis, which combines tnums with signed and unsigned interval domains through shared cross-domain reduction logic. Bhat et al. [29] build on the idea behind VeRA [36], an implementation-level verification framework for range analysis in the Firefox JIT compiler. VeRA takes the range-analysis code used by the JIT compiler, translates that implementation into logical constraints, and discharges the resulting verification conditions with an SMT solver. Its central insight is that range-analysis code can be checked automatically at the level of the implementation, rather than only by proving properties of a separate mathematical model. Bhat et al. apply this style of implementation-level verification to the Linux eBPF verifier. Their framework translates verifier C code into a Python-based intermediate representation, generates SMT formulas from that representation, and checks the resulting formulas against soundness specifications for value tracking.

Agni [110] supersedes these prior efforts by providing a unified implementation-level verification framework for the eBPF verifier’s value-tracking analysis. Compared with the tnum verification work, Agni does not require abstract operators to be manually encoded and proved in isolation.

Instead, it automatically generates verification conditions from the Linux kernel’s C implementation, so the object of verification is the verifier code that is actually maintained and deployed. This allows Agni to reason not only about the tnum domain, but also about the verifier’s signed and unsigned interval domains and the way these domains interact during value tracking. Agni also goes beyond the implementation-level range-analysis verification of Bhat et al. by formalizing the non-modular structure of the eBPF verifier’s abstract operators. In the Linux implementation, instruction-specific updates and cross-domain reductions are intertwined; arithmetic and branch operations may update several domains, use information from one domain to refine another, and rely on shared reduction routines to maintain consistency across the verifier’s abstract state. Agni accounts for this structure directly in its soundness specifications, rather than treating reduction only as a precondition on operator inputs. It further extends verification to both arithmetic and branch operators, which is essential because branches refine verifier states along different control-flow paths.

Finally, Agni turns implementation-level verification into an automated workflow that can be applied as the kernel evolves. It automatically extracts verification conditions from kernel source code, checks them against formal soundness specifications, and has been integrated into the kernel continuous integration workflow for automated checking across kernel versions. Thus, Agni moves beyond manually verified abstract-domain algorithms and beyond earlier source-level checks for selected range-analysis code.

At the same time, implementation-level verification exposes several challenges that motivate the techniques developed in this dissertation. The first challenge is scalability. A direct verification strategy encodes each abstract operator together with the verifier’s shared reduction logic as one large verification condition. This monolithic formulation is faithful to the implementation, but it duplicates the same reduction reasoning across many operators and becomes increasingly difficult for automated solvers as the verifier evolves. As a result, a strategy that works for one range of kernel versions may become prohibitively expensive for later versions.

The second challenge is diagnosability. A failed soundness check may produce a solver model

showing that an abstract state violates the specification, but that state may not be reachable during the analysis of any real eBPF program. Such a counterexample is useful for understanding the verifier’s abstract logic, but it does not by itself demonstrate executable verifier behavior. To make implementation-level verification actionable, failed checks should be connected to concrete eBPF programs that reproduce the mismatch between concrete execution and the verifier’s abstract analysis.

The third challenge is precision. Soundness does not fully characterize the quality of the verifier’s static analysis. A verifier can be sound and still be too conservative, if its abstract operators lose too much information, the verifier may reject programs whose concrete executions are safe. This matters for eBPF because the verifier must both enforce safety and retain enough information to prove that practical programs satisfy its safety checks. When an abstract operator changes, the relevant question is therefore not only whether the new implementation remains sound, but also whether it preserves at least as much information as the old implementation and whether any precision improvement is observable in real verifier executions.

Together, these challenges show that formal methods for production static analyzers must go beyond a yes-or-no soundness check. For the Linux eBPF verifier, they must scale to evolving implementation code, turn failed soundness checks into executable evidence, and support systematic reasoning about the precision of competing abstract operators.

### **1.3 Dissertation Statement**

The goal of this dissertation is to make formal methods practical for real-world development of the Linux eBPF verifier. In this setting, a useful method must scale to the verifier’s implementation and turn failed soundness checks into reproducible and debuggable evidence. This dissertation first focuses on soundness, through scalable implementation-level verification and executable witness generation. It then studies precision as a complementary property of the verifier’s static analysis, developing techniques to compare abstract operators and synthesize programs that make precision differences observable in real verifier executions. Together, these contributions provide formal

tools for analyzing both the correctness and the precision of the verifier’s value-tracking logic.

## 1.4 Contributions of the Dissertation

This dissertation develops implementation-level techniques for analyzing the Linux eBPF verifier’s abstract operators. The contributions address three questions; how to verify these operators efficiently, how to produce executable evidence when soundness checks fail, and how to compare the precision of competing operator implementations. The contributions are summarized as follows:

1. We present a divide-and-conquer approach for verifying abstract operators in the eBPF verifier. Rather than verifying each operator together with the verifier’s shared reduction logic as a single monolithic proof obligation, this approach separates instruction-specific reasoning from shared reduction reasoning. The resulting decomposition preserves the soundness guarantees of implementation-level verification while improving scalability across kernel versions.
2. We develop a differential synthesis technique for generating executable witnesses of unsound verifier behavior. When a soundness check fails, the resulting solver counterexample may describe an abstract state that is not reachable from any real eBPF program. Our approach searches for concrete eBPF programs that drive the verifier to states exposing a mismatch between concrete execution and abstract interpretation, thereby turning failed soundness checks into reproducible evidence.
3. We extend differential synthesis to compare the precision of abstract operators. Soundness alone does not determine how much information an operator preserves, and two sound operators may differ in precision. We formalize precision comparison for eBPF abstract operators and synthesize executable programs that make precision differences observable in the verifier.

The following sections provide a brief overview of each contribution and highlight the key ideas. These techniques are discussed in greater detail in the subsequent chapters.

## 1.5 Scalable Verification through Divide and Conquer

Implementation-level verification of the eBPF verifier checks whether the verifier’s abstract semantics soundly overapproximates concrete eBPF execution. Agni [110] performs this check by generating verification conditions from the verifier’s C implementation and discharging them with an SMT solver. However, this approach becomes difficult to scale as the verifier evolves. The verifier’s abstract operators are implemented as two sequential steps. We write the full abstract operator as

$$abs_g = sro \circ op_g.$$

The instruction-specific update  $op_g$  runs first and computes the abstract effect of the eBPF operation, such as addition, multiplication, bitwise operations, or shifts. The shared reduction operator  $sro$  runs afterward and propagates information across the verifier’s abstract domains.

A direct verification strategy treats these two steps as one proof obligation. For each abstract operator, it encodes both the instruction-specific update and the shared reduction logic into a single verification condition. This monolithic formulation is faithful to the implementation, but it repeatedly duplicates the same reduction reasoning across many operators. As the shared reduction logic becomes more complex, the resulting formulas become large and difficult for automated solvers to discharge.

The divide-and-conquer approach developed in this dissertation exploits this structure. Instead of verifying the instruction-specific update followed by shared reduction as one large formula, the approach verifies the two components separately. The instruction-specific update  $op_g$  is checked against the soundness requirement for the corresponding concrete eBPF operation. The shared reduction operator  $sro$  does not correspond to an eBPF operation that changes the concrete value. Instead, it may improve the precision of the abstract state, but it must not remove any concrete value represented before reduction. Soundness of the full abstract operator then follows by composing these two results.

This decomposition also reveals a different class of verifier bugs. Some abstract operators vi-

olate the general soundness specification when considered as standalone  $op_g$  components, even though the violating abstract states are not reachable during the analysis of any current eBPF program. Thus, these failures do not correspond to executable verifier exploits in the current verifier pipeline. However, they still expose incorrect logic in the verifier’s abstract-operator implementations. Future changes to reduction logic, operator composition, or state reachability could make such failures reachable and turn them into real executable bugs. We refer to these unreachable but implementation-level soundness failures as *latent unsoundness*. We identify latent unsoundness in six bitwise operators, namely the 64-bit and 32-bit variants of AND, OR, and XOR, and patch them so that their instruction-specific updates satisfy the general soundness specification independently.

The resulting strategy is significantly more scalable for the production eBPF verifier. We evaluate it on Linux kernel versions v4.14 through v6.8. The prior monolithic strategy required hours on earlier versions and timed out starting with v6.4, whereas divide-and-conquer verification completes in under 5 minutes for v4.14–v5.13, under 15 minutes for v5.19–v6.7, and under 30 minutes for v6.8. These results show that separating  $op_g$  from  $sro$  makes repeated verification across kernel versions practical.

After applying the patches, all six affected bitwise operators verify under the modular soundness specification for  $op_g$ , without relying on shared reduction to repair the output. Thus, this work both improves the verifier implementation and provides a compositional method for scaling implementation-level verification as the kernel evolves.

## 1.6 Differential Synthesis for Generating Unsound Witnesses

Implementation-level soundness checking can identify flaws in the eBPF verifier’s abstract operators, but a failed soundness check is not always actionable by itself. The solver may produce an abstract counterexample showing that the verifier’s abstract state can exclude a concrete behavior. However, that counterexample may not correspond to any state reachable during the analysis of a real eBPF program because the verifier begins execution from a restricted set of initial values. As a result, developers may still need to determine manually whether the reported failure can occur in

practice and how to reproduce it.

This dissertation addresses this problem using *differential synthesis*, the automatic generation of eBPF programs whose concrete execution diverges from the verifier’s abstract analysis. Given an abstract operator that violates the soundness specification, the goal is to synthesize an instruction sequence that shows the violating abstract inputs are reachable from valid verifier initial states. This turns an abstract counterexample into an executable program that can be run against the production verifier.

The synthesis procedure searches over bounded, loop-free eBPF programs. For a program length  $k$ , it enumerates program skeletons in which the first  $k - 1$  instructions are arithmetic, logical, or branch instructions, and the  $k$ -th instruction is the concrete operation corresponding to the abstract operator that failed the soundness check. The skeleton fixes the opcodes of the candidate program, but leaves operands, data flow, and control flow symbolic. For each skeleton, the synthesizer asks an SMT solver to fill in these symbolic parts so that the resulting program satisfies the structural constraints of eBPF programs and exposes the mismatch between concrete execution and abstract analysis. These constraints ensure, for example, that instructions read values produced by earlier instructions or by valid initial registers, and that branch targets are consistent with the chosen control flow.

The search starts from the verifier’s initial abstract values. At the beginning of verifier analysis, each live register is represented either by an exact abstract value, when the verifier knows the value precisely, or by an unconstrained abstract value, when the value is unknown. An abstract value is reachable if some sequence of eBPF instructions can produce it from these initial verifier states.

For each program skeleton, the synthesizer builds a single SMT query that jointly chooses the symbolic operands, data dependencies, control-flow choices, abstract states, and concrete states. The query is constrained so that the candidate program satisfies eBPF structural constraints, its abstract execution reaches the state needed to expose the bug, and its final instruction demonstrates

the mismatch between concrete and abstract execution. At a high level, the query enforces

$$\text{Reach}(P, \hat{s}) \wedge s \in \gamma(\hat{s}) \wedge \text{concreteExec}(i, s) \notin \gamma(\text{abstractExec}(i, \hat{s})).$$

Here,  $\text{Reach}(P, \hat{s})$  means that analyzing the synthesized prefix  $P$ , starting from the verifier’s initial abstract values, reaches the abstract verifier state  $\hat{s}$ . The function  $\gamma$  maps this abstract state to the concrete states it represents. Because the prefix is assumed sound,  $\hat{s}$  contains all concrete states that can arise after the prefix. The solver picks one such state  $s$ , written  $s \in \gamma(\hat{s})$ , and checks whether the final instruction produces a concrete result outside the verifier’s abstract output. A satisfying model gives an executable eBPF witness program with  $i$  as its last instruction.

The algorithm begins with  $k = 1$ . If no program of length  $k$  manifests the violation, it increments  $k$  and repeats the search until it either finds a satisfying model or reaches a timeout. This bounded search is incomplete, but it is effective in practice because many verifier bugs can be exposed by short instruction sequences. Branch instructions are included in the search because they can refine abstract states and make additional violating inputs reachable.

These witnesses make verification failures reproducible and useful for developers. Instead of inspecting only a solver model over abstract states, a developer can run the synthesized program against the production verifier and observe the instruction sequence that triggers the mismatch. For known vulnerabilities, the generated programs are similar to existing proof-of-concept exploits. For previously unknown bugs, the verifier state observed during kernel execution can be compared against the SMT model and the runtime result, validating that the synthesized program manifests the predicted divergence between concrete and abstract semantics.

The differential synthesis framework was evaluated as part of Agni [110] across Linux kernel versions 4.14 through 5.19. For the soundness violations reported by the soundness specification, it generated proof-of-concept programs for approximately 97% of the violations across kernel versions. Many of these witnesses required multiple eBPF instructions, showing that single-instruction tests are often insufficient to expose verifier unsoundness. These results show that

differential synthesis can turn implementation-level verification failures into concrete, executable evidence of unsound verifier behavior.

## 1.7 Differential Synthesis for Comparing Precision of Abstract Operators

Soundness ensures that the verifier does not miss unsafe behavior, but its utility hinges on its precision. An abstract operator may be sound and still be unnecessarily imprecise; it may compute bounds that are too wide, lose useful tnum information, or fail to propagate facts across domains. This loss of information can cause the verifier to reject programs whose concrete executions are safe, because the computed abstract state is too weak to prove the required safety checks.

This dissertation addresses this problem by extending differential synthesis from soundness witnesses to precision witnesses. The goal is to compare two implementations of the same abstract operator, such as an existing kernel operator and a proposed replacement. The comparison asks two questions. First, is the new operator at least as precise as the old one for all valid abstract inputs? Second, if the new operator is strictly more precise for some input, can that input actually arise during the analysis of a real eBPF program?

At a high level,  $op_{new}$  is at least as precise as  $op_{old}$  when, for every valid abstract input  $\hat{x}$ , the new output represents no more concrete values than the old output:

$$\forall \hat{x}. \quad \gamma(op_{new}(\hat{x})) \subseteq \gamma(op_{old}(\hat{x})).$$

Here,  $op_{old}$  and  $op_{new}$  are the two abstract operator implementations being compared, and  $\gamma$  maps an abstract value to the set of concrete values it represents. The new operator is strictly more precise when it is at least as precise as the old operator and the containment is strict for at least one input:

$$(\forall \hat{x}. \quad \gamma(op_{new}(\hat{x})) \subseteq \gamma(op_{old}(\hat{x}))) \wedge (\exists \hat{x}. \quad \gamma(op_{new}(\hat{x})) \subset \gamma(op_{old}(\hat{x}))).$$

Thus, the new operator is never less precise than the old one, and in some cases it rules out concrete values that the old operator could not rule out. However, not all pairs of operators are ordered by

precision. For some inputs,  $op_{new}$  may produce a more precise result than  $op_{old}$ , while for other inputs  $op_{old}$  may produce a more precise result than  $op_{new}$ . In this case, neither implementation is at least as precise as the other, and the two operators are incomparable with respect to precision.

Even when one operator is more precise on some abstract input, that difference is useful only if the input can arise during verifier analysis. The verifier does not analyze programs from arbitrary abstract states. It starts from a restricted set of initial states and reaches later states only through the abstract semantics of eBPF instructions and the verifier’s reduction logic. Therefore, an input on which  $op_{new}$  improves over  $op_{old}$  may be a logical precision difference that never occurs in practice.

Differential synthesis addresses this reachability question. Given an abstract input  $\hat{x}$  where the two operators differ in precision, the synthesizer searches for a bounded eBPF instruction sequence  $P$  whose verifier analysis reaches  $\hat{x}$ :

$$\text{Reach}(P, \hat{x}) \wedge \gamma(op_{new}(\hat{x})) \subset \gamma(op_{old}(\hat{x})).$$

Here,  $\text{Reach}(P, \hat{x})$  means that analyzing the synthesized program  $P$ , starting from the verifier’s initial abstract values, reaches the abstract input  $\hat{x}$ . If the query is satisfiable, the model gives an executable eBPF program that makes the precision improvement observable in the production verifier.

The synthesis procedure jointly reasons about the two abstract executions. It symbolically encodes bounded eBPF instruction sequences, the verifier states produced by those sequences, and the old and new implementations of the operator being compared. The solver chooses operands, data dependencies, and control-flow choices so that the generated program reaches an input where the new operator produces a strictly tighter abstract result than the old operator. The resulting program is a concrete witness that the improvement is reachable, reproducible, and relevant to real verifier behavior.

We use this framework to evaluate precision-improving changes to the eBPF verifier. In partic-

ular, we show that a new `bpf_mul` implementation is at least as precise as the previous implementation and strictly more precise on reachable verifier states. The synthesized witness demonstrates this improvement to kernel developers, and the corresponding patch was upstreamed to the Linux kernel. We also apply the framework to prior precision-related kernel patches and generate witness programs for each of them. These results show that differential synthesis can be used not only to expose unsoundness, but also to validate and demonstrate precision improvements in the verifier’s abstract operators.

## 1.8 Contributions to This Dissertation

The ideas and approaches presented in this dissertation are drawn from previously published papers written with my advisor, Santosh Nagarakatte, and collaborators Srinivas Narayana and Harishankar Vishwanathan. The following papers are listed in the order in which their ideas appear in the dissertation, rather than in publication order:

1. “Fixing Latent Unsound Abstract Operators in the eBPF Verifier of the Linux Kernel” [97], which develops a divide-and-conquer approach for scaling implementation-level verification of the eBPF verifier. This work separates instruction-specific reasoning from shared reduction reasoning, identifies latent unsoundness that prevents modular verification, and extends automated verification to newer versions of the Linux kernel.
2. “Verifying the Verifier: eBPF Range Analysis Verification” [110], which develops differential synthesis for generating executable eBPF witnesses of unsound verifier behavior. A failed soundness check may produce only an abstract counterexample, and that counterexample may not correspond to any state reachable during the analysis of a real eBPF program. This work addresses that gap by synthesizing concrete eBPF programs whose execution exposes a divergence between the verifier’s abstract semantics and the concrete semantics of eBPF execution.
3. “Comparing the Precision of Abstract Operators in the eBPF Verifier using Differential Syn-

thesis” [98], which extends differential synthesis from witnessing unsoundness to comparing the precision of abstract operators. This work formalizes precision comparison for eBPF abstract operators and synthesizes executable eBPF programs that demonstrate observable precision differences between competing operator implementations.

Together, these papers form the basis of the dissertation. The first motivates and develops modular verification techniques for scaling soundness checking of production verifier code. The second provides the implementation-level verification framework and develops witness generation for unsoundness. The third generalizes witness generation from soundness failures to precision comparison between abstract operators.

## **1.9 Organization of This Dissertation**

The remainder of this dissertation is organized as follows. Chapter 2 provides background on eBPF, the Linux eBPF verifier, and the abstract domains used for value tracking. Chapter 3 presents our divide-and-conquer approach for scaling implementation-level verification of the verifier’s abstract operators and fixing latent unsoundness. Chapter 4 presents differential synthesis for generating executable eBPF witnesses of unsound verifier behavior. Chapter 5 presents our approach for comparing the precision of competing abstract operators and synthesizing eBPF programs that make precision differences observable in real verifier executions. Chapter 6 discusses prior work on eBPF verification and testing, abstract interpretation, and program synthesis. Chapter 7 concludes this dissertation and presents future directions.

## CHAPTER 2

### BACKGROUND

The needs of production systems often outpace the rate at which new functionality can be incorporated into the kernel. Adding functionality to the kernel requires review, integration, release, distribution, and deployment, a process that can take years before a feature becomes available on production machines. At the same time, changing workloads may require new ways to observe system behavior, enforce security policies, process packets, or measure performance. Traditionally, supporting these needs required either changing the kernel or writing a kernel module. Kernel modules can be deployed locally without waiting for a new kernel release, but writing correct kernel modules is difficult. Modules execute with kernel privilege and are not statically verified before they run, so a bug in a module can corrupt kernel memory, crash the machine, or compromise system security. Moreover, if the functionality provided by a local module is meant to become broadly available and maintained as part of Linux, it must still go through the kernel community's review and acceptance process.

eBPF provides a different model for kernel extensibility. It allows small, event-driven programs to be loaded into the kernel on demand and attached to well-defined hooks, without recompiling the kernel, rebooting the machine, or waiting for a new kernel feature to be distributed. Users can therefore add specialized kernel functionality for tasks such as networking, tracing, observability, and security.

The eBPF framework aims to provide this flexibility without giving loaded programs unrestricted kernel privilege. eBPF programs execute in a restricted environment, interact with the kernel through controlled interfaces, and must be statically verified before they run. The Linux eBPF verifier is therefore essential to the framework. Because eBPF programs are supplied from outside the kernel but execute in kernel context, the kernel must establish their safety before execution. This chapter provides the background needed to understand that verification process. It

first introduces the eBPF framework and the verifier, then reviews the static-analysis and abstract-interpretation concepts used throughout the dissertation, describes the verifier’s value-tracking domains and soundness specifications, and finally introduces Agni, the implementation-level verification infrastructure on which the later chapters build.

## 2.1 The eBPF Framework

Extended Berkeley Packet Filter, or eBPF, is a mechanism for extending the Linux kernel with user-supplied programs. Although eBPF originated as a packet-filtering facility, it has evolved into a general-purpose in-kernel execution environment used for networking, tracing, observability, performance monitoring, security enforcement, load balancing, and access control [11, 19, 22, 23, 26, 55, 60, 74, 99, 103, 113]. The key idea is to allow small programs to be loaded into the kernel at runtime and attached to specific events, without requiring developers to modify kernel source code, rebuild the kernel, or deploy traditional kernel modules.

An eBPF program is compiled into bytecode for a restricted virtual machine implemented by the kernel. Modern eBPF uses a 64-bit register-based instruction set with arithmetic and logic instructions, memory loads and stores, conditional branches, helper-function calls, and an explicit exit instruction. Programs execute only when the events to which they are attached occur. These attachment points include system calls, tracepoints, kernel function entry and exit points, network events, cgroup hooks, uprobes, kprobes, and Linux Security Module security hooks. When such an event occurs, the kernel invokes the attached eBPF program with a context object describing the event.

The eBPF execution environment is intentionally restricted. Programs cannot access arbitrary kernel memory, call arbitrary kernel functions, allocate memory dynamically, use recursion, or use floating-point arithmetic. They also operate under strict resource limits. Each program has a small bounded stack, typically 512 bytes, and the verifier enforces a limit on program size and complexity. Historically, eBPF programs were limited to a few thousand instructions, while modern kernels allow programs with up to one million instructions. Modern eBPF also supports bounded loops,

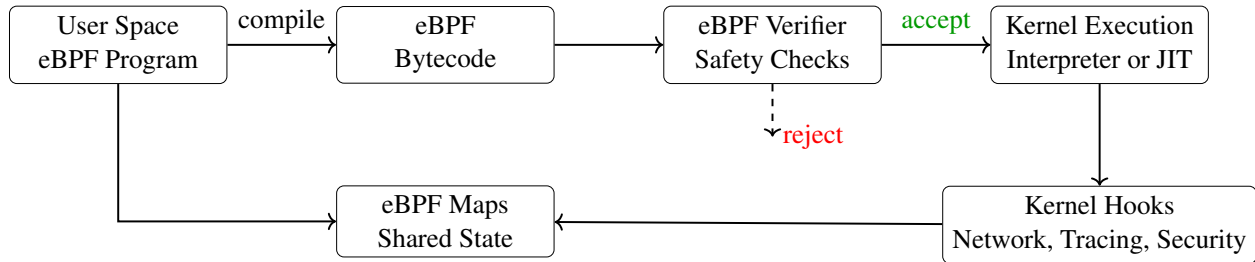


Figure 2.1: Basic eBPF workflow. User space submits eBPF bytecode to the kernel. The verifier rejects unsafe programs and permits safe programs to execute at kernel hooks. Maps provide shared state between user space and eBPF programs.

but the verifier must be able to prove that they terminate. Programs may use tail calls to transfer control to another eBPF program, but the number of tail calls in one invocation is bounded, with at most 32 tail-call jumps allowed. In addition, the helpers and context fields available to a program depend on the program type and attachment point. These restrictions allow eBPF to provide runtime kernel extensibility while keeping program loading a finite static-validation problem.

The eBPF execution model separates user-space control logic from kernel-space program execution. A user-space application typically compiles an eBPF program, loads the resulting bytecode into the kernel, configures any required state, and attaches the program to a hook. Once loaded, the program runs in kernel context whenever the hook is triggered. Communication between user space and eBPF programs is commonly mediated through *maps*, which are kernel-managed key-value data structures. Maps provide persistent state across program executions and allow user-space applications to read or update data maintained by eBPF programs. For example, a networking program may use a map to count packets by source address, while a user-space process periodically reads the map to report statistics.

Figure 2.1 summarizes this workflow. A user-space program is compiled to eBPF bytecode and submitted to the kernel. The verifier checks the bytecode before execution. Rejected programs do not run, while accepted programs execute in the kernel, either through the eBPF interpreter or the JIT compiler, when their attached hooks are triggered. eBPF maps provide shared state between user space and kernel-resident eBPF programs.

eBPF programs also interact with the kernel through helper functions. Helpers are kernel-

provided functions that expose controlled operations to eBPF programs, such as looking up map entries, reading time, accessing packet data, or emitting tracing events. Since eBPF programs cannot freely call arbitrary kernel functions or access arbitrary kernel memory, maps and helpers form the main interfaces through which eBPF programs communicate with the rest of the system.

Before an eBPF program is allowed to run, it must pass the Linux eBPF verifier. The verifier checks the bytecode before execution and rejects programs whose safety cannot be established. This validation step is what allows eBPF programs to execute in privileged kernel contexts while remaining constrained by kernel-enforced safety rules.

## 2.2 Static Analysis

Static analysis derives properties of a program without executing it on particular concrete inputs. For a program  $P$ , let  $\mathcal{B}(P)$  denote the set of runtime behaviors that  $P$  can exhibit. A static analysis computes an approximation  $A(P)$  of these behaviors. For safety checking, this approximation must be conservative. It may include behaviors that cannot occur at runtime, but it must include every behavior that can occur. Formally, a sound analysis satisfies

$$\mathcal{B}(P) \subseteq A(P).$$

This inclusion justifies using static analysis to prove safety. If no behavior represented by  $A(P)$  violates a safety property, then no concrete runtime behavior of  $P$  violates that property.

Approximation is necessary because exact static reasoning about nontrivial semantic properties of programs is undecidable in general, as captured by Rice’s theorem [94]. Programs may branch on input-dependent values, execute loops, manipulate pointers, and interact with large state spaces. A static analysis therefore replaces exact execution states with compact abstract descriptions, such as intervals, symbolic constraints, or sets of possible pointer targets, that summarize many concrete states at once. The choice of abstraction determines the tradeoff between precision and scalability. A more precise analysis may report fewer false alarms or reject fewer safe programs, while a

coarser analysis may scale better but include more spurious behaviors.

Static analyses differ both in the properties they compute and in the information they retain. One central distinction is between *flow-sensitive* and *flow-insensitive* analyses. Flow-sensitive analyses distinguish facts at different program points and account for execution order, whereas flow-insensitive analyses collapse some or all program-order information and compute a more global approximation. A second distinction is between *path-sensitive* and *path-insensitive* analyses. Path-sensitive analyses keep information from different control-flow paths separate and can preserve correlations introduced by branches, while path-insensitive analyses merge facts from multiple paths, usually at join points. Keeping more information can improve precision, but it also increases running time, memory use, or solver effort.

Different analysis techniques compute different kinds of facts. Type checking derives or checks types for program phrases and rejects operations applied to incompatible types [89]. Dataflow analysis computes facts such as liveness, reaching definitions, available expressions, or constant values by solving equations over a control-flow graph [70]. Control-flow analysis approximates possible transfers of control, such as the possible successors of a program point or the possible targets of a call. Pointer and alias analyses approximate which memory objects a pointer may refer to, or whether two references may denote the same object. Constraint-based analyses encode the facts to be inferred as variables in a system of constraints and compute analysis results by solving that system [86]. Such constraints can express properties such as types, points-to relationships, dataflow facts, or numeric invariants. Model checking verifies properties by exploring the states of a formal model of a system, often against temporal-logic specifications, and can produce counterexample executions when a property fails [41].

Symbolic execution is a path-oriented analysis technique. It follows program paths using symbolic inputs, accumulates path constraints, and uses a solver to check whether those paths are feasible or to generate concrete inputs [71]. This can produce precise counterexamples, but it may not scale when the number of feasible paths grows quickly. Abstract interpretation provides a general framework for building sound static analyses [48]. It replaces concrete program states with

abstract states drawn from an abstract domain and computes a fixed point using abstract transfer functions. The abstract domain determines which facts the analysis can represent, while the transfer functions describe how those facts change across program operations.

The eBPF verifier is a static analyzer for eBPF bytecode. It analyzes each program before execution and accepts it only when the analysis establishes the required safety properties. The verifier therefore makes the same fundamental tradeoff as other static analyses. It must be conservative enough to reject unsafe programs, precise enough to accept useful safe programs, and efficient enough to run whenever an eBPF program is loaded.

### **2.3 The eBPF Verifier**

The Linux eBPF verifier [20] is the kernel component responsible for deciding whether an eBPF program is safe to execute. The verifier is necessary because accepted eBPF programs execute in kernel mode, either through the interpreter or after just-in-time compilation to native machine code. Without verification, an eBPF program could corrupt kernel memory, leak sensitive information, crash the kernel, or cause the kernel to hang. The verifier therefore acts as the gatekeeper between user-supplied bytecode and privileged kernel execution.

The verifier enforces a collection of safety rules imposed by the eBPF execution environment. These rules include termination, initialized register and stack use, bounded stack and memory accesses, valid pointer arithmetic, valid helper-function calls, and program-type-specific restrictions on context fields, helpers, and memory regions. For example, a networking program may access packet data only within packet bounds, while a program that receives a pointer from a map lookup must check that the pointer is non-null before dereferencing it. Programs that violate these requirements, or whose safety cannot be proved, are rejected before execution.

To enforce these rules, the verifier applies the static analysis techniques introduced earlier in a coordinated way. It first checks the structure of the program, constructs a control-flow graph from branch instructions, and verifies that control transfers are well formed. Control flow analysis allows the verifier to identify the possible execution paths through the program, ensuring that jumps are

valid, exits are reachable, and loops or repeated paths remain bounded.

The verifier then analyzes the program using symbolic execution over abstract states. It initializes the starting state according to the program type. For example, register `r1` often contains a pointer to the program context, while other registers initially have restricted or unknown contents. As the verifier walks over instructions, it updates an abstract description of registers, stack locations, pointer metadata, and scalar values. Type and pointer analysis track the kind of value held in each register or stack location, distinguishing ordinary scalar values from pointers to stack memory, packet data, map values, context objects, or other restricted memory regions. This information is essential for proving that memory accesses are valid and that pointers are not misused.

Symbolic execution propagates these abstract states along each feasible path, allowing the verifier to reason about all possible inputs rather than a single concrete execution. At branches, the verifier may fork the current state and refine each successor state using the branch condition. Range analysis, also called value tracking, refines this reasoning by tracking numeric bounds and bit-level information for values such as offsets, lengths, and loop counters. These approximations are especially important for proving that pointer arithmetic, array accesses, packet accesses, and helper-function arguments remain within safe limits.

Figure 2.2 illustrates this path-sensitive reasoning on a simple branch. The verifier uses the branch condition to refine the possible values of `r1` differently on the taken and fall-through paths. Such refinements are important because later safety checks may depend on the refined bounds.

---

```

1  /* verifier knows: r1 in [0, 100] */
2
3  if (r1 < 64) {
4      /* r1 in [0, 63] */
5  } else {
6      /* r1 in [64, 100] */
7  }

```

---

Figure 2.2: A simple value-tracking example showing how the verifier refines scalar bounds along different branch paths.

Although the verifier reasons separately about different execution paths, it must ensure that ver-

ification itself always terminates and remains practical during program loading. This requirement is closely tied to the restricted design of the eBPF execution environment. eBPF programs cannot use recursion, allocate memory dynamically, call arbitrary kernel functions, or access arbitrary kernel memory. They also execute under bounded resources, including a small fixed-size stack, limits on program size and verifier complexity, bounded loops whose termination must be proved, and a bounded number of tail-call jumps. Together, these restrictions ensure that eBPF programs can be checked statically and that the verifier’s exploration remains finite.

Branches may cause the verifier to fork abstract states and queue unexplored states for later analysis, while loops and repeated control flow may cause similar states to be encountered many times. To keep this process tractable, the verifier bounds the amount of exploration it performs, including the number of instructions inspected, the number of states retained, and the amount of repeated exploration allowed along complex control-flow paths. It also uses state pruning to avoid reanalyzing states that are equivalent to, or safely covered by, states that have already been explored. Liveness analysis supports this pruning by identifying registers and stack locations whose current contents cannot affect future execution, allowing the verifier to ignore irrelevant differences between states. These mechanisms prevent path-sensitive analysis from growing without bound.

Together, these analyses allow the verifier to establish key safety properties. Unsafe code paths are unreachable, memory accesses remain within valid bounds, accepted programs cannot execute without bound, and redundant verifier states can be safely eliminated. The verifier therefore balances the same precision and scalability tradeoff that arises in static analysis more generally. If the analysis is too imprecise, it rejects useful programs even when they are safe. If it is too permissive or unsound, it may accept programs that can violate kernel safety. If it requires too much time or memory, program loading can become slow or consume excessive kernel resources. The verifier must therefore combine several static analyses in a way that is precise enough to accept practical eBPF programs, conservative enough to protect the kernel, and efficient enough to run as part of the loading process.

This dissertation focuses on the verifier’s value-tracking logic. Value tracking approximates

the possible numeric and bit-level values held in eBPF registers. These approximations are used to justify safety checks involving bounds, offsets, branches, pointer arithmetic, and helper arguments. If value tracking is too imprecise, the verifier may reject safe programs. If it is unsound, the verifier may accept programs whose runtime behavior violates the assumptions used by the verifier's safety checks. The following sections introduce abstract interpretation, which provides the formal framework for reasoning about this value-tracking logic.

## 2.4 Abstract Interpretation

Abstract interpretation is a framework for constructing sound static analyses. Instead of tracking the exact values that program variables may take across all executions, an abstract interpretation tracks *abstract values* that compactly represent sets of concrete values. For example, in an interval domain, the abstract value  $[x, y]$ , where  $x, y \in \mathbb{Z}$  and  $x \leq y$ , represents the set

$$\{z \in \mathbb{Z} \mid x \leq z \leq y\}.$$

Such abstractions are compact, but they may introduce imprecision. The concrete set  $\{1, 4, 6\}$ , for instance, can be represented by the interval  $[1, 6]$ , which also includes 2, 3, and 5. Abstract interpretation therefore trades precision for tractability; the analysis may overapproximate the set of possible values, but it must not miss any value that can occur at runtime.

### 2.4.1 Abstract Domains, Concretization, and Abstraction

Let  $\mathcal{C}$  be a domain of individual concrete values, and let  $\mathbb{C} = \mathcal{P}(\mathcal{C})$  be the collecting concrete domain. Elements of  $\mathbb{C}$  are sets of possible concrete values. Let  $\mathbb{A}$  be an abstract domain that compactly represents such sets.

The meaning of an abstract value is given by a concretization function

$$\gamma : \mathbb{A} \rightarrow \mathbb{C}.$$

For example, if  $\mathcal{C} = \mathbb{Z}$ , then  $\mathbb{C} = \mathcal{P}(\mathbb{Z})$ , and an interval  $a = [\underline{a}, \bar{a}]$  represents all integers between its lower and upper bounds:

$$\gamma(a) = \{z \in \mathbb{Z} \mid \underline{a} \leq z \leq \bar{a}\}.$$

The abstract domain is equipped with a partial order  $\sqsubseteq_{\mathbb{A}}$ , where  $a_1 \sqsubseteq_{\mathbb{A}} a_2$  means that  $a_1$  is at least as precise as  $a_2$ . For intervals, this order is determined by the sets they represent:  $a_1 \sqsubseteq_{\mathbb{A}} a_2$  when  $\gamma(a_1) \subseteq \gamma(a_2)$ . Thus,  $[0, 7] \sqsubseteq_{\mathbb{A}} [0, 15]$ , since  $\gamma([0, 7]) \subset \gamma([0, 15])$ .

Conversely, an abstraction function

$$\alpha : \mathbb{C} \rightarrow \mathbb{A}$$

maps concrete sets to abstract values. For the interval domain, abstraction maps a concrete set  $c$  to the smallest interval containing that set:

$$\alpha(c) = \begin{cases} [\min(c), \max(c)] & \text{if } c \neq \emptyset, \\ \perp & \text{if } c = \emptyset. \end{cases}$$

Here,  $\perp$  denotes the bottom element of the abstract domain, representing the empty set of concrete values. For example,  $\alpha(\{1, 4, 6\}) = [1, 6]$ . The abstraction preserves the original values but may introduce additional ones. Thus, an abstract value  $a$  is a sound abstraction of a concrete set  $c \in \mathbb{C}$  when

$$c \sqsubseteq_{\mathbb{C}} \gamma(a).$$

Since  $\mathbb{C}$  is a powerset domain in this section, this condition is simply

$$c \subseteq \gamma(a).$$

### 2.4.2 Abstract Operators and Soundness

Program operations are modeled by transfer functions. We write  $\bar{f}$  for the concrete operation on individual values and  $f$  for its collecting semantics over sets of concrete values. For example, if  $\bar{f} : \mathcal{C} \rightarrow \mathcal{C}$  is unary, then its collecting semantics is the function  $f : \mathbb{C} \rightarrow \mathbb{C}$  defined by

$$f(c) = \{\bar{f}(x) \mid x \in c\}.$$

An abstract operator  $g : \mathbb{A} \rightarrow \mathbb{A}$  is sound with respect to  $f$  if, for every abstract input  $a \in \mathbb{A}$ ,

$$f(\gamma(a)) \sqsubseteq_{\mathbb{C}} \gamma(g(a)).$$

That is, the abstract output may include extra concrete values, but it must contain every concrete result that can be produced from values represented by the input.

As a simple example, over mathematical integers, let  $f$  be the collecting semantics of adding one, so  $f(c) = \{z + 1 \mid z \in c\}$ . Over intervals, this operation is soundly abstracted by

$$g(a) = [\underline{a} + 1, \bar{a} + 1].$$

If  $a = [\underline{a}, \bar{a}]$ , then adding one to any value represented by  $a$  produces a value represented by  $g(a)$ , and therefore

$$f(\gamma(a)) \sqsubseteq_{\mathbb{C}} \gamma(g(a)).$$

The same idea applies to binary operations. If  $\bar{f} : \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C}$ , then its collecting semantics maps two concrete sets to all pairwise concrete results:

$$f(c_1, c_2) = \{\bar{f}(x_1, x_2) \mid x_1 \in c_1 \text{ and } x_2 \in c_2\}.$$

An abstract operator  $g : \mathbb{A} \times \mathbb{A} \rightarrow \mathbb{A}$  is sound if, for every pair of abstract inputs  $a_1, a_2 \in \mathbb{A}$ ,

$$f(\gamma(a_1), \gamma(a_2)) \sqsubseteq_{\mathbb{C}} \gamma(g(a_1, a_2)).$$

Equivalently, whenever individual concrete inputs are represented by the abstract inputs, their concrete result must be represented by the abstract output. This pointwise view is the form used later when encoding verifier soundness as a first order logic formula. In those implementation-level specifications, we use lowercase letters for individual concrete values and uppercase letters for the corresponding abstract values.

### 2.4.3 Best Abstract Operators

The abstraction and concretization functions can also be used to describe the most precise sound abstraction of a concrete operation. When  $\alpha$  and  $\gamma$  form a Galois connection, they satisfy

$$\alpha(c) \sqsubseteq_{\mathbb{A}} a \iff c \sqsubseteq_{\mathbb{C}} \gamma(a).$$

This condition says that  $\alpha(c)$  is the most precise abstract value that soundly represents the concrete set  $c$ . Given a concrete collecting semantics  $f$ , the best abstract operator is

$$g_{best} = \alpha \circ f \circ \gamma.$$

This operator first interprets the abstract input as a concrete set, applies the concrete semantics, and abstracts the result back into  $\mathbb{A}$ . It is the most precise sound abstraction of  $f$ . In practice, this construction is often too expensive to compute directly, so static analyzers usually implement abstract operators directly, trading precision for efficiency and implementation simplicity. This dissertation studies how to reason about such implemented operators in the eBPF verifier.

#### 2.4.4 Product Domains and Reductions

Static analyses often combine multiple abstract domains because different domains capture different properties of the same concrete value. The simplest combination is the Cartesian product [46]. Suppose  $\mathbb{A}_1$  and  $\mathbb{A}_2$  are abstract domains over the same individual concrete domain  $\mathcal{C}$ , with concretization functions

$$\gamma_{\mathbb{A}_1} : \mathbb{A}_1 \rightarrow \mathcal{C} \quad \text{and} \quad \gamma_{\mathbb{A}_2} : \mathbb{A}_2 \rightarrow \mathcal{C}.$$

Their product domain is

$$\mathbb{P} = \mathbb{A}_1 \times \mathbb{A}_2,$$

ordered component-wise:

$$(a_1, a_2) \sqsubseteq_{\mathbb{P}} (b_1, b_2) \iff a_1 \sqsubseteq_{\mathbb{A}_1} b_1 \wedge a_2 \sqsubseteq_{\mathbb{A}_2} b_2.$$

Because both components describe the same individual concrete value, the standard product concretization intersects the values represented by each component:

$$\gamma_{\mathbb{P}}(a_1, a_2) = \gamma_{\mathbb{A}_1}(a_1) \cap \gamma_{\mathbb{A}_2}(a_2).$$

For example, in the product of an interval domain and a parity domain, the abstract value  $([3, 5], \text{even})$  represents  $\{4\}$ ; the interval allows  $\{3, 4, 5\}$ , while the parity component restricts the value to be even. Thus, a product can represent more precise information than either component alone.

The Cartesian product does not, by itself, propagate this combined information back into the individual components. In the example above,  $([3, 5], \text{even})$  and  $([4, 4], \text{even})$  have the same product concretization, but the latter exposes the stronger interval fact. A reduced product [49] adds a reduction operator

$$\rho : \mathbb{P} \rightarrow \mathbb{P}$$

that refines a product element while preserving its concretization:

$$\gamma_{\mathbb{P}}(\rho(a)) = \gamma_{\mathbb{P}}(a).$$

Intuitively, reduction uses information from one component to strengthen another. For instance, it can refine  $([3, 5], \text{even})$  to  $([4, 4], \text{even})$ . Both product elements represent  $\{4\}$ , but the reduced one makes the interval component more precise.

Computing the exact reduced product can be impractical, so analyzers may use weaker reductions that improve precision without necessarily producing the most precise product element. Such a reduction remains sound as long as it preserves all feasible concrete values represented by the product. The eBPF verifier follows a similar strategy in its value-tracking implementation. It uses a reduction operator to refine relationships among its abstract domains. This resembles a reduced product approach [81], but it is not a direct implementation of the textbook construction. The verifier is not organized around a single product domain whose concretization is defined by intersecting component concretizations. Later chapters return to this structure in two ways. Chapter 3 studies how shared reduction can be used to scale verification, and Chapter 5 studies the precision properties of the verifier’s reduction operator.

## 2.5 Satisfiability Modulo Theories

Satisfiability modulo theories (SMT) is a framework for deciding whether a first order logic formula is satisfiable. At the propositional level, satisfiability asks whether there is an assignment of truth values to Boolean variables that makes a formula true. For example, the formula

$$(p \vee q) \wedge (\neg p \vee r)$$

is satisfiable because there are assignments to  $p$ ,  $q$ , and  $r$  that make both clauses true. Boolean satisfiability is NP-complete, and SMT inherits this worst-case difficulty. Nevertheless, modern SAT and SMT solvers are effective on many formulas that arise in program analysis, verification,

and synthesis.

SMT extends Boolean satisfiability by allowing formulas to include variables and operations from background theories. These theories provide meaning to non-Boolean expressions. For example, a formula may use linear integer arithmetic, such as  $x + y \leq 10 \wedge x \geq 0 \wedge y \geq 0$ , or fixed-width bit-vector operations, such as  $(x \& 7) = 0$ , where  $x$  is a machine integer and  $\&$  is bitwise conjunction. Theories such as integers, bit-vectors, arrays, and uninterpreted functions make SMT useful for reasoning about programs, whose semantics combine Boolean control flow with arithmetic, memory, and machine-level operations.

An SMT solver takes such a formula and determines whether it is satisfiable with respect to the chosen theories. If the formula is satisfiable, the solver may return a *model*, which assigns values to the variables so that all constraints hold. For example, the formula

$$x + y \leq 10 \wedge x \geq 0 \wedge y \geq 0 \wedge x > y$$

has models such as  $x = 2$  and  $y = 1$ . If no assignment satisfies the formula, the solver reports that the formula is unsatisfiable.

SMT solving can also be used to prove universal properties by searching for counterexamples. To prove that a property  $\varphi(x)$  holds for all  $x$ , one asks the solver whether the negation is satisfiable:

$$\exists x. \neg \varphi(x).$$

If this query is unsatisfiable, then no counterexample exists, and the original property  $\forall x. \varphi(x)$  holds. If the query is satisfiable, the solver’s model gives a concrete counterexample. Otherwise, the solver may time out or return unknown if it cannot determine satisfiability.

This dissertation uses an SMT-based approach in two ways. First, it uses SMT solving for verification. Soundness specifications for eBPF abstract operators are encoded as formulas, and an SMT solver is asked to find a counterexample in which a concrete instruction produces a value that is not represented by the verifier’s abstract result. If the formula is unsatisfiable, the encoded

soundness property holds. If the formula is satisfiable, the solver produces a model that explains how the operator violates the specification.

Second, the dissertation uses SMT solving for synthesis. In differential synthesis, the solver does more than check whether a property fails. It also chooses parts of a program. The unknowns may include instruction operands, data dependencies, control-flow choices, abstract values, and concrete values. A satisfying model therefore corresponds to an eBPF program fragment that meets the desired constraints. This SMT-based synthesis procedure underlies the executable witnesses for soundness violations and the programs that demonstrate precision differences between abstract operators.

## 2.6 Value Tracking in the eBPF Verifier

We now describe the abstract domains used by the verifier’s value-tracking analysis. These domains represent different views of the same scalar register value, including signed and unsigned numeric bounds and bit-level uncertainty.

### 2.6.1 Abstract Domains for Value Tracking

The eBPF verifier represents each scalar register using several abstract domains. It maintains four interval domains, unsigned and signed 64-bit bounds, written  $A_{u64}$  and  $A_{s64}$ , and unsigned and signed 32-bit bounds, written  $A_{u32}$  and  $A_{s32}$ . These domains track lower and upper bounds under different interpretations of the same register bits. The verifier also maintains a bit-level domain, called the *tnum* domain  $A_{tnum}$ , which records which bits of a register are known and which bits may vary [108].

We write  $\mathbb{U}_{64} = \{0, \dots, 2^{64} - 1\}$  and  $\mathbb{S}_{64} = \{-2^{63}, \dots, 2^{63} - 1\}$  for the unsigned and signed 64-bit integer ranges. The product domain for scalar register values is

$$\mathcal{A} = A_{u64} \times A_{s64} \times A_{tnum} \times A_{u32} \times A_{s32}.$$

Thus, the abstract value of a scalar register  $r$  has the form  $\hat{r} = \langle a_{u64}, a_{s64}, a_{tnum}, a_{u32}, a_{s32} \rangle \in \mathcal{A}$ . A value-tracking *abstract state* maps scalar registers to elements of  $\mathcal{A}$ .

The *tnum* domain tracks uncertainty at the bit level using unsigned 64-bit machine words. A *tnum* can be viewed as a tristate bit-vector, where each bit is known to be 0, known to be 1, or unknown, written  $\mu$ . The Linux kernel represents a *tnum* using two unsigned 64-bit words: a value word and a mask word. The value word records known bits, while the mask word records unknown bits. Formally,

$$A_{tnum} \triangleq \{ \langle value, mask \rangle \in \mathbb{U}_{64} \times \mathbb{U}_{64} \mid value \& mask = 0 \}.$$

The well-formedness condition  $value \& mask = 0$  says that no bit can be both known and unknown. The concretization of a *tnum* is

$$\gamma_{tnum}(\langle value, mask \rangle) = \{ x \in \mathbb{U}_{64} \mid (x \& \sim mask) = value \}.$$

Here,  $\sim mask$  denotes 64-bit bitwise complement. Equivalently,  $x$  must match *value* on every known bit and may differ only on bits marked unknown by *mask*. For example, a *tnum* with  $value = 0b10$  and  $mask = 0b01$  represents the set  $\{0b10, 0b11\}$ , because the low bit is unknown and the high bit is known to be 1. This domain is useful for bitwise operations, alignment reasoning, and partially known values, and is structurally similar to the bitfield domain [80].

The interval domains instantiate the interval abstraction from Section 2.4. The signed 64-bit interval domain  $A_{s64}$ , for example, models signed numeric bounds:

$$A_{s64} \triangleq \{ a = [\underline{a}, \bar{a}] \mid \underline{a}, \bar{a} \in \mathbb{S}_{64} \wedge \underline{a} \leq_{s64} \bar{a} \} \cup \{ \perp \}.$$

The verifier implementation stores these bounds in fields such as `s64_min` and `s64_max`. The

concretization function for this domain is

$$\gamma_{s64}(a) \triangleq \begin{cases} \{z \in \mathbb{S}_{64} \mid \underline{a} \leq_{s64} z \leq_{s64} \bar{a}\} & \text{if } a \neq \perp, \\ \emptyset & \text{if } a = \perp. \end{cases}$$

Conversely, the abstraction function maps a concrete set of signed 64-bit values to the smallest signed interval containing it:

$$\alpha_{s64}(c) \triangleq \begin{cases} [\min_{s64}(c), \max_{s64}(c)] & \text{if } c \neq \emptyset, \\ \perp & \text{if } c = \emptyset. \end{cases}$$

The other interval domains follow the same structure, adjusted for their bit-width and signedness. For unsigned domains such as  $A_{u64}$ , the concretization and abstraction functions use  $\mathbb{U}_{64}$ , unsigned comparison, and bounds in  $[0, 2^{64} - 1]$ . The 32-bit domains use the corresponding 32-bit signed and unsigned ranges.

## 2.6.2 Abstract Operators for Value Tracking

For each eBPF instruction, the verifier applies an abstract operator that updates the abstract state rather than executing the instruction on concrete values. Arithmetic and bitwise instructions update the abstract value of a destination register. Conditional branch instructions split the current state into two successor states, following the branch-refinement pattern introduced in Section 2.3 and illustrated in Figure 2.2. One successor corresponds to the taken branch, and the other corresponds to the fall-through branch. The taken-branch state is refined using the branch condition, while the fall-through state is refined using the complementary condition. These operators are implemented in the Linux kernel and are the main objects of verification in this dissertation.

As an interval example, consider signed 64-bit subtraction. Here the concrete operation is fixed-width signed subtraction, while  $\ominus_{s64}$  denotes the corresponding interval abstract operator. Given two intervals  $a_1$  and  $a_2$ , the abstract subtraction operator computes an interval containing all

possible signed differences:

$$a_1 \ominus_{s64} a_2 \triangleq \begin{cases} [\underline{a_1} -_{s64} \overline{a_2}, \overline{a_1} -_{s64} \underline{a_2}] & \text{if no overflow occurs,} \\ [-2^{63}, 2^{63} - 1] & \text{otherwise.} \end{cases}$$

Overflow is checked according to signed 64-bit arithmetic. If overflow may occur, the operator returns the full signed 64-bit interval. This loses precision, but preserves soundness because all possible concrete results remain represented.

Bitwise instructions such as AND, OR, and XOR rely especially heavily on the interaction between intervals and tnums. Their concrete behavior is naturally bit-level, so the tnum component often provides the strongest information about the result. At the same time, later verifier checks frequently depend on numeric bounds. The implementation must therefore maintain both bit-level and interval information for the same value.

Branch instructions have a different value-tracking effect. A conditional branch does not compute a new scalar value. Instead, it refines the abstract state according to the branch condition. For example, after a comparison against a constant, the verifier may strengthen the bounds of a register on the branch-taken path and apply a complementary refinement on the fall-through path. These refinements are important because later instructions are analyzed under the abstract state associated with the corresponding path.

Although the verifier’s value tracking can be described using the abstract interpretation terminology from Section 2.4, the production implementation is not organized as a clean product of independent domain transformers. The verifier stores several views of the same register value and updates them through kernel C code that often mixes operator-specific reasoning with cross-domain information. An operator may use tnum information to derive numeric bounds, or use interval information to strengthen bit-level facts. This implementation structure motivates the implementation-level soundness specifications described next.

## 2.7 Soundness Specifications for eBPF Abstract Operators

The previous sections defined soundness for abstract operators in general. We now specialize that notion to the implementation-level specifications used for the eBPF verifier. The goal is to check whether the verifier's C implementation of an abstract operator soundly overapproximates the corresponding concrete eBPF instruction.

### 2.7.1 Concrete and Abstract Values

We use lowercase letters for individual concrete values and uppercase letters for abstract values. Thus,  $p, q, r \in \mathcal{C}$  denote concrete values, while  $P, Q, R \in \mathbb{A}$  denote abstract values. We use a membership predicate  $mem_{\mathbb{A}}(p, P)$  to state that concrete value  $p$  is represented by abstract value  $P$ , i.e.,

$$mem_{\mathbb{A}}(p, P) \triangleq p \in \gamma_{\mathbb{A}}(P).$$

For example, in the signed 64-bit interval domain,

$$mem_{s64}(p, P) \triangleq \underline{P} \leq_{s64} p \leq_{s64} \overline{P}.$$

### 2.7.2 Arithmetic and Logic Operators

For arithmetic and logic instructions, the concrete instruction computes a new scalar value from its operands. Let  $\bar{g} : \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C}$  be the concrete semantics of such an instruction, and let  $abs_g$  denote the input-output behavior of the corresponding abstract operator extracted from the verifier's C code. For a single abstract domain  $\mathbb{A}$ , the implementation-level soundness condition is

$$\begin{aligned} \forall p, q \in \mathcal{C}, P, Q \in \mathbb{A} : \quad & mem_{\mathbb{A}}(p, P) \wedge mem_{\mathbb{A}}(q, Q) \\ & \wedge r = \bar{g}(p, q) \wedge abs_g(P, Q) = R \\ \implies & mem_{\mathbb{A}}(r, R). \end{aligned} \tag{2.1}$$

This formula says that whenever the abstract inputs represent the concrete inputs, the abstract output computed by the verifier must represent the concrete output. Its validity can be checked with an SMT solver by asking whether the negation is satisfiable. If the negation is unsatisfiable, then no counterexample exists and the encoded soundness condition is valid.

### 2.7.3 Branch Operators

Branch instructions require a different soundness specification. Unlike arithmetic and logic instructions, a branch does not compute a new scalar register value. Instead, it compares existing concrete values and updates the verifier's abstract state differently on the true and false branch outcomes.

For example, suppose the verifier tracks two registers in an unsigned interval domain. Before the branch, the first register is represented by  $([1,5])$ , and the second is represented by  $([3,3])$ . If the program branches on equality between these registers, then on the true branch the verifier learns that the first register must also have value (3). Therefore, the first register's abstract value can be updated from  $([1,5])$  to  $([3,3])$ . The concrete value stored in the register has not changed; only the verifier's knowledge about that value has become more precise.

Let  $\bar{g} : \mathcal{C} \times \mathcal{C} \rightarrow \mathbb{B}$  be the concrete semantics of a conditional branch, where  $\mathbb{B}$  is the Boolean domain. The result

$$b = \bar{g}(p, q)$$

records whether the branch condition is true or false. The verifier's abstract branch operator produces four abstract outputs:

$$(P_{\top}, P_{\text{F}}, Q_{\top}, Q_{\text{F}}) = \text{abs}_g(P, Q).$$

Here,  $P_{\top}$  and  $Q_{\top}$  are the abstract values assigned to the two input registers on the true branch, while  $P_{\text{F}}$  and  $Q_{\text{F}}$  are the abstract values assigned to them on the false branch.

The soundness condition for a branch says that the abstract values on the taken branch must

still represent the unchanged concrete values:

$$\begin{aligned}
& \forall p, q \in \mathcal{C}, b \in \mathbb{B}, P, Q \in \mathbb{A}. \quad mem_{\mathbb{A}}(p, P) \wedge mem_{\mathbb{A}}(q, Q) \\
& \quad \wedge b = \bar{g}(p, q) \\
& \quad \wedge (P_{\top}, P_{\text{F}}, Q_{\top}, Q_{\text{F}}) = abs_g(P, Q) \\
& \quad \implies (b \implies (mem_{\mathbb{A}}(p, P_{\top}) \wedge mem_{\mathbb{A}}(q, Q_{\top}))) \\
& \quad \wedge (\neg b \implies (mem_{\mathbb{A}}(p, P_{\text{F}}) \wedge mem_{\mathbb{A}}(q, Q_{\text{F}}))) .
\end{aligned}$$

This formula captures the main difference between branches and arithmetic or logic operators. An arithmetic or logic operator produces one concrete result and one abstract result. A branch instead preserves the concrete operands and produces abstract values for both possible control-flow outcomes. Soundness only requires the abstract values on the concrete branch outcome to represent the unchanged concrete operands.

#### 2.7.4 Product-Domain Specifications

The eBPF verifier uses several abstract domains for the same scalar register, so the specification is extended component-wise. To illustrate the extension, consider two abstract domains  $\mathbb{A}_1$  and  $\mathbb{A}_2$  over the same individual concrete domain  $\mathcal{C}$ . The first concrete input  $p$  is represented by abstract values  $P_1 \in \mathbb{A}_1$  and  $P_2 \in \mathbb{A}_2$ , while the second concrete input  $q$  is represented by  $Q_1 \in \mathbb{A}_1$  and  $Q_2 \in \mathbb{A}_2$ . If the verifier operator produces outputs

$$(R_1, R_2) = abs_g(P_1, P_2, Q_1, Q_2),$$

then soundness requires the concrete output  $r = \bar{g}(p, q)$  to be represented in both domains:

$$\begin{aligned}
\forall p, q, r \in \mathcal{C}, P_1, Q_1, R_1 \in \mathbb{A}_1, P_2, Q_2, R_2 \in \mathbb{A}_2. \quad & mem_{\mathbb{A}_1}(p, P_1) \wedge mem_{\mathbb{A}_2}(p, P_2) \\
& \wedge mem_{\mathbb{A}_1}(q, Q_1) \wedge mem_{\mathbb{A}_2}(q, Q_2) \\
& \wedge r = \bar{g}(p, q) \\
& \wedge (R_1, R_2) = abs_g(P_1, P_2, Q_1, Q_2) \\
& \implies mem_{\mathbb{A}_1}(r, R_1) \wedge mem_{\mathbb{A}_2}(r, R_2).
\end{aligned}$$

This is the same specification as before, applied component-wise. Each abstract input component must represent the corresponding concrete input, and each abstract output component must represent the concrete output.

The actual verifier specification follows the same pattern for the five value-tracking components  $A_{u64}$ ,  $A_{s64}$ ,  $A_{tnum}$ ,  $A_{u32}$ , and  $A_{s32}$ . Each concrete input must be represented by the corresponding input component in each domain, and the concrete output must be represented by every output component. Thus, the verifier may overapproximate the concrete result, but none of its value-tracking components may exclude it. For branch instructions, the same component-wise idea applies to the abstract values on the taken branch outcome.

## 2.8 Implementation-Level Verification with Agni

This dissertation builds on Agni [110], an implementation-level verification framework for the eBPF verifier’s value-tracking logic. Agni checks whether the Linux kernel implementation of verifier abstract operators soundly overapproximates the corresponding concrete eBPF instructions. It performs this check directly on the verifier implementation, rather than requiring a separate hand-written model of each abstract operator.

Table 2.1 lists the eBPF instructions handled by Agni’s value-tracking verification. The table includes the ALU/ALU64 operations that compute new scalar register values and the JMP/JMP32 operations that refine abstract register values according to branch conditions.

| Instruction | Class     | Effect                          |
|-------------|-----------|---------------------------------|
| ADD         | ALU/ALU64 | $r_d \leftarrow r_d + r_s$      |
| SUB         | ALU/ALU64 | $r_d \leftarrow r_d - r_s$      |
| AND         | ALU/ALU64 | $r_d \leftarrow r_d \& r_s$     |
| OR          | ALU/ALU64 | $r_d \leftarrow r_d   r_s$      |
| XOR         | ALU/ALU64 | $r_d \leftarrow r_d \oplus r_s$ |
| LSH         | ALU/ALU64 | $r_d \leftarrow r_d \ll r_s$    |
| RSH         | ALU/ALU64 | $r_d \leftarrow r_d \gg r_s$    |
| ARSH        | ALU/ALU64 | $r_d \leftarrow r_d \ggg_s r_s$ |
| JEQ         | JMP/JMP32 | branch if $r_d = r_s$           |
| JNE         | JMP/JMP32 | branch if $r_d \neq r_s$        |
| JGT         | JMP/JMP32 | branch if $r_d >_u r_s$         |
| JGE         | JMP/JMP32 | branch if $r_d \geq_u r_s$      |
| JLT         | JMP/JMP32 | branch if $r_d <_u r_s$         |
| JLE         | JMP/JMP32 | branch if $r_d \leq_u r_s$      |
| JSGT        | JMP/JMP32 | branch if $r_d >_s r_s$         |
| JSGE        | JMP/JMP32 | branch if $r_d \geq_s r_s$      |
| JSLT        | JMP/JMP32 | branch if $r_d <_s r_s$         |
| JSLE        | JMP/JMP32 | branch if $r_d \leq_s r_s$      |
| JSET        | JMP/JMP32 | branch if $(r_d \& r_s) \neq 0$ |

Table 2.1: eBPF instructions handled by Agni. The table lists the eBPF ALU/ALU64 and JMP/JMP32 instructions covered by Agni. Here  $r_d$  denotes the destination register,  $r_s$  denotes the source operand,  $\ggg_s$  denotes arithmetic right shift, and the subscripts  $u$  and  $s$  denote unsigned and signed comparisons. The ALU and JMP32 classes operate on 32-bit values, while ALU64 and JMP operate on 64-bit values.

### 2.8.1 From Verifier Code to SMT Formulas

Agni provides the infrastructure for translating verifier C code into SMT formulas. It does not attempt to model arbitrary kernel C code. Instead, it targets the value-tracking routines of the eBPF verifier. For each eBPF instruction, Agni extracts the verifier code that updates the abstract register state and translates the input-output behavior of that code into a formula over bit-vectors.

Agni compiles the verifier implementation to LLVM intermediate representation, specializes the code to one eBPF instruction at a time, inlines the relevant verifier helper routines, removes code that does not affect value tracking, and simplifies the result. After this specialization, the relevant code is loop-free. The resulting implementation formula describes how the verifier transforms

an input abstract state into an output abstract state for the selected instruction.

The translation relies on the structure of the verifier’s value-tracking code. The relevant routines update fields of `struct bpf_reg_state`, which stores the verifier’s abstract information for a register. Instead of modeling all of C memory, Agni models these fields directly as bit-vector variables. Reads from a field become reads of the corresponding variable, and writes to a field produce an updated version of that variable. Conditional statements in the verifier implementation are translated by recording the condition for each feasible control-flow path. When a field is assigned different values on different paths, the SMT formula represents the final field value as a conditional expression over those path conditions.

This dissertation does not study the C-to-SMT translation itself. We treat Agni as the infrastructure that provides implementation formulas for verifier abstract operators. The later chapters focus on how those formulas are checked, decomposed, and connected to executable witnesses and precision comparisons.

## 2.8.2 Implementation-Level Soundness Checking

Agni combines the extracted implementation formula with the soundness specifications introduced earlier to produce verification conditions. For arithmetic and bitwise instructions, Agni checks the soundness specification introduced in Section 2.7 and stated in Equation 2.1. The abstract operator in this check is extracted from the verifier’s C implementation. Thus, whenever the abstract inputs represent the concrete inputs and the concrete instruction computes a result  $r$ , the abstract output computed by the verifier implementation must represent  $r$ .

Agni checks this condition by asking an SMT solver whether the negation of the verification condition is satisfiable. If the negation is unsatisfiable, the encoded soundness property holds for the implementation. If it is satisfiable, the solver produces a model illustrating a soundness violation.

For branch instructions, the verification condition has the form described in Section 2.7.3. A branch does not modify the concrete register values. Instead, the verifier produces abstract val-

ues for the taken and fall-through outcomes. The implementation formula therefore relates the input abstract values to the abstract values computed for both outcomes, and the verification condition checks that the abstract values on the concrete branch outcome still represent the unchanged concrete operands.

Because the verifier stores several abstract views of the same scalar value, Agni checks soundness component-wise across the value-tracking domains. A concrete input must be represented by each relevant input component, and the concrete result must be represented by each output component. This matches the product-domain specifications introduced above, but it is applied to the actual kernel implementation, whose code often combines operator-specific updates with cross-domain reductions.

### 2.8.3 Refined Soundness Specification

Agni also introduced a refined soundness specification based on the verifier’s shared reduction logic. The generic specifications from Section 2.7 treat each abstract operator as if it could receive any combination of abstract input components, provided that there is at least one concrete value represented by all of those components. Agni observes that this is not how such inputs arise during verifier execution. After each operator-specific update, the verifier applies shared reduction logic that propagates information among the value-tracking components. Thus, the inputs to a later operator have already passed through this reduction step. Apparent soundness violations can therefore arise from abstract inputs that satisfy the generic specification but do not satisfy the stronger consistency invariant maintained by the verifier’s reduction logic.

The refined specification models this structure by preconditioning operator inputs with the shared reduction operator. Let  $sro$  denote the shared reduction operator. Instead of checking the operator directly on abstract inputs  $P$  and  $Q$ , the refined specification first computes

$$P' = sro(P) \quad \text{and} \quad Q' = sro(Q),$$

and then checks soundness on the reduced inputs  $P'$  and  $Q'$ . For arithmetic and bitwise instructions, the refined verification condition has the following shape:

$$\begin{aligned}
 \forall p, q \in \mathcal{C}, P, Q \in \mathbb{A} : \quad & P' = sro(P) \wedge Q' = sro(Q) \\
 & \wedge mem_{\mathbb{A}}(p, P') \wedge mem_{\mathbb{A}}(q, Q') \\
 & \wedge r = \bar{g}(p, q) \wedge abs_g(P', Q') = R \\
 & \implies mem_{\mathbb{A}}(r, R).
 \end{aligned} \tag{2.2}$$

Compared with Equation 2.1, Equation 2.2 checks soundness only after the verifier’s shared reduction step has been applied. Thus, the refined specification does not ask whether the operator is sound on every abstract input allowed by the generic specification. It asks whether the operator is sound on inputs after the reduction step that the verifier itself applies. This captures more of the verifier’s implementation structure and removes many violations that rely on abstract inputs that cannot occur during the verifier’s abstract execution.

In this dissertation, Agni provides the baseline infrastructure for extracting verifier C code into SMT formulas and checking implementation-level soundness conditions. The following chapters build on this starting point. Chapter 3 decomposes the monolithic verification task into operator-specific updates and shared reduction, making verification scalable for newer kernels. Chapter 4 starts from failed implementation-level soundness checks and turns them into executable witnesses. Chapter 5 adapts the same implementation-level reasoning to compare the precision of competing abstract operators.

## CHAPTER 3

### SCALABLE VERIFICATION THROUGH DIVIDE AND CONQUER

This chapter describes our approach for scaling the verification of abstract operators in the eBPF verifier. The verifier implements a collection of value-tracking operators in kernel C code, and our goal is to prove that these operators soundly overapproximate the corresponding concrete eBPF instructions. A direct approach is to verify each operator monolithically by extracting its C semantics and checking the resulting verification condition with an SMT solver. Although this approach is conceptually simple, it does not scale well to the structure of the verifier. Each operator contains instruction-specific logic, but it also invokes shared reduction logic that propagates information among the verifier’s abstract domains. As the shared reduction logic becomes more complex, inlining it into every operator proof produces large verification conditions and substantially increases solver cost.

Our key idea is to exploit the structure of the verifier implementation. We factor each verifier operator into two parts, an instruction-specific operator and a shared reduction operator. The instruction-specific operator computes the immediate abstract effect of an eBPF instruction, while the shared reduction operator propagates information across domains after the update. Instead of verifying the full composition monolithically for every instruction, we verify the instruction-specific operator and the shared reduction operator separately and then compose their correctness. This divide-and-conquer strategy avoids re-proving the same shared reduction logic for every abstract operator.

This decomposition introduces a new requirement. The operator-specific operator must be sound before shared reduction is applied, and the shared reduction operator must preserve all concrete values represented by its input. If an operator-specific operator produces an unsound intermediate result that is later repaired by reduction, then the full implementation may appear sound when checked monolithically, but the proof cannot be decomposed. Such cases expose latent soundness

issues in the operator-specific logic and prevent modular verification.

Based on these ideas, this chapter develops a modular verification framework for the eBPF verifier’s value-tracking operators. We first show how each operator can be decomposed into an instruction-specific component and a shared reduction component. We then present the soundness obligations for each component and show why their composition implies soundness of the full abstract operator. We also discuss how the modular proof exposes latent unsoundness in operator-specific logic and how fixing these intermediate unsoundness issues enables the full operator to be verified compositionally. Finally, we evaluate the approach by comparing monolithic verification with the divide-and-conquer strategy across verifier operators and kernel versions.

### 3.1 Implementation of Interval Domains in the eBPF Verifier

Chapter 2 introduced the interval domains used by the eBPF verifier for value tracking. This chapter works directly with the verifier’s implementation of those domains. In the implementation, the verifier represents the four interval components as pairs of lower and upper bounds: `u64_min/u64_max`, `s64_min/s64_max`, `u32_min/u32_max`, and `s32_min/s32_max`. These fields are updated both by instruction-specific abstract operators and by the shared reduction operator.

Each pair of bounds must remain well formed. For the signed interval components, this means

$$s64\_min \leq s64\_max \quad \text{and} \quad s32\_min \leq s32\_max.$$

For the unsigned components, the corresponding invariants are

$$u64\_min \leq u64\_max \quad \text{and} \quad u32\_min \leq u32\_max.$$

These inequalities are representation invariants of the verifier’s interval implementation. When they hold, the fields denote valid interval abstract values. When they are violated, the corresponding component no longer represents a well-formed range.

The distinction between imprecision and ill-formedness is important. An abstract operator may

safely lose precision by assigning a component to the full range of its domain. Such an update is conservative because it still overapproximates the possible runtime values. In contrast, an update that produces  $s64\_min > s64\_max$  or  $s32\_min > s32\_max$  is not a valid conservative approximation. It violates the representation invariant that later verifier logic assumes when performing bounds checks, branch reasoning, pointer-arithmetic checks, and cross-domain reduction.

Agni checks the verifier implementation at this level. Its verification conditions are generated from the verifier’s C code, so the soundness obligation is not only that the mathematical operation overapproximates concrete execution, but also that the concrete output fields form valid abstract values. The latent unsoundness studied in this chapter arises when some instruction-specific operators temporarily violate these interval well-formedness invariants before the shared reduction operator repairs the state.

### 3.2 A Divide-and-Conquer Verification Strategy

Agni verifies the eBPF verifier at the implementation level. Rather than proving the correctness of an idealized abstract operator, it extracts the input-output behavior of the verifier’s C implementation and checks that this behavior soundly overapproximates the corresponding concrete eBPF operation. This approach is useful because it reasons about the code that actually runs in the kernel. However, it also means that the size and structure of the generated verification conditions depend directly on the structure of the verifier implementation.

A key implementation pattern in the verifier is that each abstract operator can be viewed as the composition of two sub-operators:

$$abs_g = sro \circ op_g.$$

The composition is read right-to-left, so  $op_g$  runs first and  $sro$  runs afterward. Here,  $op_g$  is the instruction-specific operator and  $sro$  is the shared reduction operator. The instruction-specific operator  $op_g$  implements the abstract effect of a particular eBPF instruction, such as addition, multiplication, bitwise conjunction, or right shift. The shared reduction operator  $sro$  then propagates

information among the verifier’s abstract domains. For example, it may use information from the tnum component to refine interval bounds, or use interval bounds to refine bit-level information.

The original Agni strategy verified the full composition  $sro \circ op_g$  as a single verification condition. This monolithic strategy was faithful to the verifier implementation, but it duplicated the same shared reduction reasoning inside every operator proof. As the shared reduction operator grew more complex in newer kernel versions, the generated formulas became increasingly difficult for the SMT solver to discharge. Starting with Linux v6.4, this strategy no longer scaled in practice because verification could take weeks or time out.

The key idea of our divide-and-conquer strategy is to exploit the compositional structure of the implementation. Instead of generating one verification condition for  $sro \circ op_g$ , we generate separate verification conditions for  $op_g$  and  $sro$ . We then prove that the instruction-specific operator soundly overapproximates the concrete eBPF operation, and separately prove that shared reduction operator preserves all concrete values represented by its input abstract state. The correctness of the full operator follows by composing these two results.

Figure 3.1 illustrates the difference between the two strategies. In the monolithic strategy, each proof obligation includes both the instruction-specific operator and the shared reduction operator. In the divide-and-conquer strategy, the instruction-specific operators are verified independently, while the shared reduction operator is verified once. This decomposition separates instruction-specific reasoning from shared cross-domain refinement reasoning, which substantially reduces the size and redundancy of the generated SMT queries.

The decomposition also changes what is required of the verifier implementation. Under the monolithic strategy, an instruction-specific operator could produce an unsound intermediate state, as long as the subsequent shared reduction step repaired the state before the final output was checked. Under the divide-and-conquer strategy, this is no longer allowed. Each  $op_g$  must be sound on its own, before the shared reduction operator is applied. This stronger modularity requirement is what exposes the latent unsoundness discussed in Section 3.3.

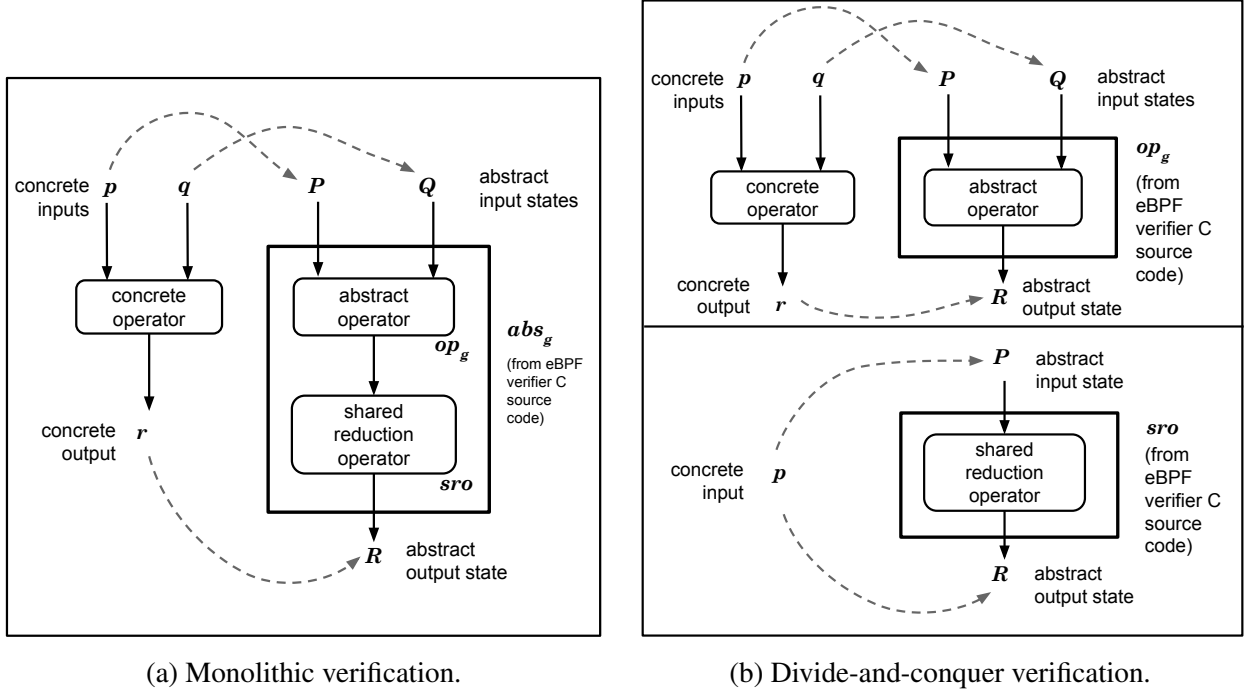


Figure 3.1: Comparison between Agni’s prior monolithic verification strategy and the divide-and-conquer strategy introduced in this chapter. In the monolithic strategy, Agni verifies the full abstract operator  $abs_g = sro \circ op_g$  as a single verification condition, which repeatedly includes the shared reduction logic in every operator proof. In the divide-and-conquer strategy, Agni verifies the instruction-specific operator  $op_g$  and the shared reduction operator  $sro$  separately. The separate  $sro$  check requires shared reduction to preserve membership and to return an abstract value at least as precise as its input. This separates instruction-specific reasoning from shared reduction reasoning and allows the soundness of the full abstract operator to follow by composition. Dotted arrows indicate membership between a concrete value and the corresponding abstract value; for example,  $p$  is represented by  $P$ .

### 3.2.1 Soundness Specification for $op_g$

The instruction-specific operator  $op_g$  has the same concrete semantics as the full abstract operator  $abs_g$ . The difference is that its output is the intermediate abstract state produced before the shared reduction operator is applied. Thus, the soundness specification for  $op_g$  is the same implementation-level specification from Section 2.7, except that  $abs_g$  is replaced by  $op_g$ .

We use  $\mathcal{C}$  for the domain of individual concrete eBPF machine values. The verifier tracks each scalar register using the product abstract domain

$$\mathbb{A} = \mathbb{A}_{u64} \times \mathbb{A}_{s64} \times \mathbb{A}_{tnum} \times \mathbb{A}_{u32} \times \mathbb{A}_{s32}.$$

Thus,  $p, q, r \in \mathcal{C}$  are concrete values, while an abstract value  $P \in \mathbb{A}$  contains one component for each value-tracking domain. The predicate  $mem_{\mathbb{A}}(p, P)$  means that concrete value  $p$  is represented by all components of  $P$ . Section 2.7 shows the corresponding component-wise expansion for multiple abstract domains.

Using this notation, the soundness specification for  $op_g$  is:

$$\begin{aligned} \forall p, q \in \mathcal{C}, P, Q \in \mathbb{A}. \quad & mem_{\mathbb{A}}(p, P) \wedge mem_{\mathbb{A}}(q, Q) \\ & \wedge r = \bar{g}(p, q) \wedge R = op_g(P, Q) \\ \implies & mem_{\mathbb{A}}(r, R). \end{aligned}$$

The premise states that each concrete input is represented by the corresponding input abstract value in the verifier's product domain. It also relates the concrete output  $r$  to the concrete eBPF operation  $\bar{g}$ , and relates the abstract output  $R$  to the implementation-level behavior of  $op_g$ . The conclusion requires the concrete output to be represented by the output abstract value.

This specification requires  $op_g$  to soundly overapproximate the concrete operation without help from the shared reduction operator. The shared reduction operator may improve the precision of the result later, but the modular proof does not allow  $op_g$  to produce an intermediate abstract state that excludes a possible concrete result.

### 3.2.2 Soundness Specification for $sro$

The shared reduction operator differs from an instruction-specific operator because it does not correspond to an eBPF instruction that changes the concrete value. Its purpose is to refine an existing abstract state by propagating information among domains. Therefore, the concrete semantics associated with  $sro$  is the identity function.

We write  $R = sro(P)$  for the output of the shared reduction operator on a product-domain

abstract value  $P \in \mathbb{A}$ . The soundness condition for  $sro$  is:

$$\begin{aligned} \forall p \in \mathcal{C}, P \in \mathbb{A}. \quad & mem_{\mathbb{A}}(p, P) \\ & \wedge r = p \wedge R = sro(P) \\ \implies & mem_{\mathbb{A}}(r, R) \wedge R \sqsubseteq_{\mathbb{A}} P. \end{aligned}$$

The shared reduction operator does not change the concrete value; its concrete  $P$  with a reduced abstract value  $R$ . The soundness condition requires two properties. First, every concrete value represented before reduction must still be represented after reduction. Second, the reduced value must be at least as precise as the input abstract value, written  $R \sqsubseteq_{\mathbb{A}} P$ . Thus, shared reduction may strengthen the verifier's abstract information, but it must not exclude any concrete value represented by the input.

This is exactly the role of a sound reduction step. It can make the abstract state more precise by propagating information between domains, but it cannot change the concrete semantics of the program. For example, reducing an interval using information from `tnum` is sound only if every concrete value that was possible before the reduction remains possible afterward.

### 3.2.3 Composing the Two Proof Obligations

The two modular specifications imply the soundness of the full abstract operator. Suppose the concrete inputs  $p$  and  $q$  are represented by their input abstract states. The  $op_g$  soundness condition ensures that the concrete result  $r = \bar{g}(p, q)$  is represented by the intermediate abstract state produced by  $op_g$ . Then, because  $sro$  is sound with respect to the identity function, applying  $sro$  to that intermediate state preserves the same concrete result. Therefore, the final output of  $sro \circ op_g$  represents the concrete output of the eBPF instruction.

This reasoning is what allows the divide-and-conquer strategy to replace one large proof obligation with two smaller ones. Instead of repeatedly inlining shared reduction into each operator proof, Agni verifies the shared reduction operator once and reuses that result compositionally. The benefit is especially important as the verifier evolves, because changes to shared reduction no

longer multiply the complexity of every operator-specific verification condition.

However, the compositional proof is only valid when both sub-operators satisfy their own specifications. In particular, every instruction-specific operator must be sound before shared reduction is applied. If  $op_g$  produces an unsound intermediate state that is later repaired by  $sro$ , the monolithic proof may hide the issue, but the modular proof will fail. This observation led us to investigate the operators that failed the  $op_g$  soundness obligation and to fix the latent unsoundness in their operator-specific logic.

### 3.3 Latent Unsoundness in Bitwise Operators

The divide-and-conquer proof strategy requires each instruction-specific operator to be sound before the shared reduction operator is applied. This modular requirement exposes intermediate states that can be hidden by a monolithic proof of the full composition  $sro \circ op_g$ . In particular, an instruction-specific operator may produce an abstract state that is not sound on its own, but that is later repaired by shared reduction. We refer to such implementation-level failures as *latent unsoundness*. They do not necessarily imply that the current verifier accepts an unsafe eBPF program, but they make the operator difficult to verify compositionally and make the implementation more fragile under future changes to shared reduction.

To find these cases, we use Agni [109] to check each instruction-specific operator without the shared reduction operator. Under this modular specification, Agni reports six unsound bitwise operators: `bpf_and`, `bpf_or`, `bpf_xor`, and their 32-bit counterparts `bpf_and_32`, `bpf_or_32`, and `bpf_xor_32`. The SMT models produced by Agni show that these operators can produce ill-formed signed interval components. In particular, their outputs may violate the interval well-formedness invariants introduced in Section 3.1, such as `s64_min <= s64_max` and `s32_min <= s32_max`.

#### 3.3.1 Root Cause of the Unsoundness

We explain the root cause using `bpf_and` as a representative example. The same pattern appears in `bpf_or`, `bpf_xor`, and the corresponding 32-bit operators.

```

1 case bpf_and:
2   out.tnum = tnum_and(in1, in2);
3   out.s32, out.u32 = interval_and_32(in1, in2);
4   out.s64, out.u64 = interval_and_64(in1, in2);
5

```

(a) Update order for bpf\_and.

```

1 def interval_and_64(in1, in2):
2   # ...
3   out.u64_min = in1.tnum.v
4   out.u64_max = min(in1.u64_max, in2.u64_max)
5
6   if (in1.s64_min < 0 or in2.s64_min < 0):
7     out.s64_min = INT64_MIN
8     out.s64_max = INT64_MAX
9   else:
10    out.s64_min = out.u64_min    # \
11    out.s64_max = out.u64_max    # / unsafe cast
12    # ...
13

```

(b) Signed bounds are inferred from unsigned bounds.

Figure 3.2: Source of latent unsoundness in the `bpf_and` abstract operator. Subfigure (a) shows the update order across abstract domains. The operator first updates the `tnum` component, then the 32-bit interval components, and finally the 64-bit interval components. Subfigure (b) shows the relevant 64-bit interval update. Here, `INT64_MIN` and `INT64_MAX` denote the endpoints of the signed 64-bit range,  $-2^{63}$  and  $2^{63} - 1$ . The signed bounds are inferred by copying the updated unsigned bounds in the `else` branch. The check on line 6 only tests whether the input signed lower bounds are negative. It does not ensure that the updated unsigned interval can be safely interpreted as a well-formed signed interval. If the unsigned interval crosses the signed 64-bit boundary, this assignment can produce `s64_min > s64_max`.

Figure 3.2a shows the structure of the `bpf_and` abstract operator. The verifier first updates the `tnum` component, then the 32-bit interval components, and finally the 64-bit interval components. This ordering matters because the interval updates use information from the already-updated `tnum` component to infer new bounds.

Figure 3.2b shows the relevant 64-bit interval update. The verifier derives the unsigned lower bound from the updated `tnum` value and derives the unsigned upper bound from the unsigned upper bounds of the operands. It then attempts to derive signed 64-bit bounds from these updated

unsigned bounds. If either input signed lower bound is negative, the verifier falls back to the full signed 64-bit range, preserving well-formedness at the cost of precision. Otherwise, it copies the updated unsigned bounds into the signed component.

The check before this assignment is too weak. It only tests whether either input signed interval may contain negative values. It does not test the property needed for the assignment itself, namely whether the updated unsigned interval remains well formed when its bounds are interpreted as signed 64-bit values.

There are three cases of interest when updated unsigned 64-bit bounds are used to derive signed 64-bit bounds.

First, the entire unsigned interval may lie below the signed boundary:

$$u64\_min \leq u64\_max \leq 2^{63} - 1.$$

In this case, both bounds are representable as nonnegative signed 64-bit values, so copying them into the signed component preserves the interval ordering. For example, the unsigned interval  $[5, 10]$ , when interpreted as signed 64-bit values, gives the signed interval  $[5, 10]$ .

Second, the entire unsigned interval may lie above the signed boundary:

$$2^{63} - 1 < u64\_min \leq u64\_max.$$

In this case, both bounds are interpreted as negative signed values. The signed values wrap around in the same order, so the resulting signed interval remains well formed. For example, the unsigned interval  $[2^{64} - 10, 2^{64} - 1]$ , when interpreted as signed 64-bit values, gives the signed interval  $[-10, -1]$ .

Third, the unsigned interval may cross the signed boundary:

$$u64\_min \leq 2^{63} - 1 < u64\_max.$$

This is the unsafe case. The lower unsigned bound is interpreted as a nonnegative signed value, while the upper unsigned bound is interpreted as a negative signed value. For example, the unsigned interval  $[2^{63} - 1, 2^{63}]$  is interpreted as the signed bounds  $[2^{63} - 1, -2^{63}]$ , which violates the well-formedness invariant  $s64\_min \leq s64\_max$ . The original implementation fails to distinguish this boundary-crossing case from the two safe cases above.

### 3.3.2 Patching Latent Unsoundness

Our patch fixes the latent unsoundness by checking whether the updated unsigned bounds form a well-formed interval after they are interpreted as signed 64-bit values. Instead of deciding whether to infer signed bounds from properties of the input intervals, the patched code checks the output bounds that will be assigned to the signed component.

Figure 3.3 shows the patched logic. The condition  $(s64) \text{ out.u64\_min} \leq (s64) \text{ out.u64\_max}$  tests whether casting the updated unsigned lower and upper bounds to signed 64-bit values preserves their ordering. If the condition holds, then the unsigned bounds can be copied into the signed component safely. This covers the two safe cases described above, where the unsigned interval lies entirely below or entirely above the signed boundary.

```

1  if ((s64) out.u64_min <= (s64) out.u64_max):
2      out.s64_min = out.u64_min;
3      out.s64_max = out.u64_max;
4  else:
5      out.s64_min = INT64_MIN;
6      out.s64_max = INT64_MAX;
7

```

Figure 3.3: Patch for the six affected bitwise abstract operators. The signed bounds are updated from the unsigned bounds only when the resulting signed interval is well formed. Otherwise, the operator falls back to the full signed 64-bit range.

If the condition does not hold, then the unsigned interval crosses the signed boundary. In that case, directly copying the unsigned bounds would produce an ill-formed signed interval. The patched code therefore falls back to the full signed 64-bit range. This loses precision, but it is a

sound conservative approximation and preserves the signed interval well-formedness invariant.

After applying the patch, Agni verifies the affected bitwise operators under the modular soundness specification for  $op_g$ . The fix therefore removes the latent unsoundness in the instruction-specific operators and enables the divide-and-conquer proof to apply compositionally. The corresponding patch fixes all six affected bitwise operators and has been upstreamed to the Linux kernel [5].

### 3.4 Evaluating Modular Verification of Value-Tracking Abstract Operators

We implemented the divide-and-conquer verification strategy in the Agni prototype [109]. Using this implementation, we check whether the patched instruction-specific operators satisfy the modular soundness specification for  $op_g$ , and we compare the verification time of the divide-and-conquer strategy against the prior monolithic strategy. In this section, we describe the experimental setup, the modular verification results for the patched operators, and the verification performance across Linux kernel versions.

#### 3.4.1 Experimental Setup

Implementing the divide-and-conquer strategy required extending Agni’s verification-condition generator to handle newer features used by the verifier and to generate separate verification conditions for  $op_g$  and  $sro$ . With these changes, Agni can verify the value-tracking abstract operators in recent Linux kernel versions using the modular divide-and-conquer strategy.

For the runtime experiments, we measure the time required to verify all value-tracking abstract operators in each kernel version. We compare the prior monolithic strategy, which verifies the full abstract operator including shared reduction, against the divide-and-conquer strategy, which verifies each instruction-specific operator separately and verifies the shared reduction operator once. The experiments were conducted on the CloudLab [52] framework using two 10-core machines with Intel Skylake CPUs running at 2.2 GHz and 192GB of RAM.

| eBPF instruction | Before patch | After patch |
|------------------|--------------|-------------|
| bpf_and          | ✗            | ✓           |
| bpf_and_32       | ✗            | ✓           |
| bpf_or           | ✗            | ✓           |
| bpf_or_32        | ✗            | ✓           |
| bpf_xor          | ✗            | ✓           |
| bpf_xor_32       | ✗            | ✓           |

Table 3.1: Verification results for the affected bitwise abstract operators in Linux v6.8. Before the patch, the operators fail the modular soundness check. After the patch, Agni verifies them under the modular specification.

### 3.4.2 Modular Verification of Patched Operators

The previous section showed that the bitwise operators `bpf_and`, `bpf_or`, and `bpf_xor`, together with their 32-bit counterparts, contained latent unsoundness in their operator-specific updates. These operators could produce ill-formed signed interval components before shared reduction was applied. We applied our fixes to Linux v6.8 and to selected older kernel versions, and then rechecked the affected operators using Agni’s modular soundness specification for  $op_g$ .

Table 3.1 summarizes the result. Before the patch, the affected bitwise operators failed the modular  $op_g$  soundness check. After the patch, Agni verifies that these instruction-specific operators are sound without assuming any repair by the shared reduction operator. Other value-tracking operators were already sound under the modular specification. Thus, the patch removes the latent unsoundness that prevented compositional verification of the bitwise operators.

### 3.4.3 Performance

We next compare the execution time of the prior monolithic strategy with that of the divide-and-conquer strategy. The monolithic strategy generates a single large verification condition for each full abstract operator, including the shared reduction logic. As this logic grew more complex, the resulting verification conditions became increasingly difficult for the SMT solver to discharge. In particular, after changes to shared reduction in Linux v6.4, the monolithic strategy no longer completed verification; even for earlier versions, it was already costly.

| Kernel version | Old strategy | New strategy |
|----------------|--------------|--------------|
| 4.14           | ~2.5 hrs     | < 5 min      |
| 5.5            | ~2.5 hrs     | < 5 min      |
| 5.9            | ~4 hrs       | < 5 min      |
| 5.13           | ~10 hrs      | < 5 min      |
| 5.19           | ~36 hrs      | < 15 min     |
| 6.3            | ~36 hrs      | < 15 min     |
| 6.4            | Timeout      | < 15 min     |
| 6.5            | Timeout      | < 15 min     |
| 6.6            | Timeout      | < 15 min     |
| 6.7            | Timeout      | < 15 min     |
| 6.8            | Timeout      | < 30 min     |

Table 3.2: Verification runtime for all value-tracking abstract operators in each kernel version. The old strategy verifies each full operator monolithically, while the new strategy separates instruction-specific updates from shared reduction.

The divide-and-conquer strategy verifies each instruction-specific operator separately and verifies shared reduction once. This avoids repeatedly inlining the shared reduction operator into every abstract-operator proof. Table 3.2 reports the time required to verify all value-tracking abstract operators in each kernel version.

The results show that the divide-and-conquer strategy substantially reduces verification time. With this strategy, Agni verifies all value-tracking abstract operators in a given kernel version in under 15 minutes for the tested versions before v6.8. For Linux v6.8, where changes to the verifier’s shared reduction operator made the reduction logic significantly larger, the monolithic strategy timed out, while the divide-and-conquer strategy verified all operators in under 30 minutes. These results demonstrate that separating  $op_g$  from  $sro$  makes verification more robust to changes in shared reduction logic. Future changes to shared reduction may again increase solver cost, but the modular structure provides a path for further decomposition; if needed, the shared reduction operator itself can be split into smaller components and verified compositionally.

### 3.5 Summary

This chapter presented a divide-and-conquer strategy for scaling implementation-level verification of the eBPF verifier’s value-tracking operators. The key observation is that verifier operators are implemented as an instruction-specific operator followed by a shared reduction operator. Rather than verifying the full composition monolithically for every operator, our approach verifies each  $op_g$  component separately and verifies the shared reduction operator  $sro$  once. This decomposition avoids repeatedly inlining the shared reduction logic into every proof obligation.

Applying this strategy exposed latent unsoundness in six bitwise abstract operators. These operators could produce ill-formed signed interval components before shared reduction was applied, which prevented modular verification. We identified the source of the problem, patched the affected operators, and confirmed with Agni that the patched instruction-specific operators satisfy the modular soundness specification for  $op_g$ . The patch has been upstreamed to the Linux kernel [5].

The evaluation shows that the divide-and-conquer strategy substantially reduces verification time across Linux kernel versions v4.14 through v6.8. The prior monolithic strategy required hours on older versions and timed out starting with v6.4. In contrast, the modular strategy verifies all value-tracking abstract operators within minutes, even for v6.8, where the shared reduction logic had grown substantially. These results show that compositional verification can make implementation-level checking of production verifier code practical as the kernel evolves.

More broadly, this work demonstrates that scaling formal verification for production systems requires both proof decomposition and engineering support for real-world development workflows. We enhanced Agni in response to feedback from Linux kernel developers, and some developers have begun using the prototype [1, 24, 25]. Our experience suggests that modular verification can make the cost of checking verifier operators low enough to be considered for continuous-integration workflows, but adoption also requires careful integration with existing kernel development practices.

## CHAPTER 4

### DIFFERENTIAL SYNTHESIS FOR GENERATING UNSOUND WITNESSES

The prior chapter built on Agni’s [110] implementation-level verification framework to make soundness checking for the eBPF verifier’s value-tracking logic scalable. It did so by decomposing each abstract operator into an instruction-specific operator and a shared reduction operator. This decomposition addresses the scalability of verification, but it does not address a separate question; when a soundness check fails, does the failure correspond to an actual eBPF program?

A failed verification condition produces an SMT model that describes a mismatch between concrete semantics and the verifier’s abstract semantics. The model is useful because it identifies concrete values and abstract states that violate the soundness condition. However, it is not yet an executable bug report. The verifier does not analyze arbitrary abstract states. It starts from restricted initial states and reaches later states only by applying the verifier’s abstract operators, branch-transfer logic, and reduction logic. Therefore, an SMT counterexample may correspond to a reachable verifier bug, or it may describe an abstract state that is latent or unreachable in actual verifier executions.

This reachability question is not answered by either verification strategy. The original monolithic Agni strategy checks the full verifier operator, including shared reduction logic, but a failed proof still gives only an abstract counterexample. The divide-and-conquer strategy decomposes every abstract operator into two components and verifies them, exposing intermediate failures more directly, but a failed modular proof still does not show that the failing intermediate state can arise from any eBPF program. Both strategies identify violated soundness obligations. Neither one determines whether the violation is reachable in an actual eBPF execution.

This chapter presents differential synthesis, a technique for closing this gap. Given a soundness failure, differential synthesis attempts to synthesize a concrete eBPF program that drives the verifier to abstract operands exposing the failure and then executes an instruction whose concrete

result is not represented by the verifier’s abstract output. Such a program is a witness of unsound behavior. It demonstrates a divergence between the concrete execution of an eBPF program and the verifier’s abstract interpretation of that program.

The key idea is to formulate witness generation as a bounded synthesis problem. We enumerate candidate loop-free eBPF instruction sequences and encode both their concrete execution and their abstract execution in the verifier. The SMT solver then searches for instruction operands, abstract states, and dataflow choices that make the final instruction expose the soundness violation. If the formula is satisfiable, the model yields a concrete eBPF program witnessing the mismatch. If no program is found within the bound, the result is inconclusive rather than a proof that no witness exists.

Differential synthesis serves two purposes. First, it helps debug verifier soundness failures by connecting an SMT counterexample to a concrete sequence of verifier states. Second, it makes verification results actionable for kernel developers by producing small eBPF programs that demonstrate the mismatch, rather than only reporting a low-level SMT model. The rest of this chapter describes the synthesis problem, the encoding for single- and multi-instruction witnesses, the treatment of branches, and the guarantees and limitations of the approach.

## 4.1 Differential Synthesis

A failed soundness check produces a local counterexample. It gives concrete operands, abstract operands, and outputs for which the verifier’s abstract result does not represent the corresponding concrete result. Such a counterexample shows that an abstract operator is locally unsound, but it does not show that the abstract operands used in the counterexample can arise during the analysis of an actual eBPF program. Differential synthesis adds this missing reachability requirement.

For a bound  $k$ , differential synthesis searches over loop-free instruction sequences of length  $k$ . The last instruction is fixed to the operator  $g$  whose soundness violation is being tested. The preceding  $k - 1$  instructions form a prefix whose purpose is to construct operands for  $g$  from abstract values that can appear in an initial verifier state. The final instruction must then produce a

concrete result that is not represented by the verifier’s abstract result.

The search combines enumeration with SMT solving. The enumerator chooses an opcode skeleton

$$i_1; i_2; \dots; i_{k-1}; g,$$

where each prefix instruction  $i_j$  is drawn from the allowed eBPF arithmetic, logic, and branch instructions, and the final instruction is the operator under test. The enumerator fixes only the instruction opcodes. It does not choose the concrete operands, abstract operands, or dataflow between instructions.

For each enumerated skeleton, we generate an existential SMT query. Unlike synthesis tasks that require a program to satisfy a specification for all inputs, this query is purely existential. The solver only needs to find one program, one set of operands, and one concrete execution that exposes the mismatch. It chooses the concrete values, abstract values, and operand sources that instantiate the skeleton as a witness program. A satisfying model therefore gives both the program dataflow and the concrete/abstract states that expose the violation. If all skeletons of length  $k$  are unsatisfiable, then no witness is found at that bound. The search is sound but incomplete; any generated witness demonstrates a real mismatch between concrete execution and abstract interpretation, but failure to find a witness within a bound does not prove that the violation is unreachable.

Figure 4.1 summarizes the structure of the witness produced by this search. Prefix instructions construct reachable abstract states while preserving membership between concrete and abstract execution. The final instruction exposes the violation by producing a concrete result not represented by the abstract result.

#### 4.1.1 Notation

We present the encoding using one domain  $\mathcal{C}$  of individual concrete machine values and one abstract domain  $\mathbb{A}$ . In this section,  $\mathbb{A}$  denotes the verifier’s full product abstract domain:

$$\mathbb{A} = \mathbb{A}_{u64} \times \mathbb{A}_{s64} \times \mathbb{A}_{tnum} \times \mathbb{A}_{u32} \times \mathbb{A}_{s32}.$$

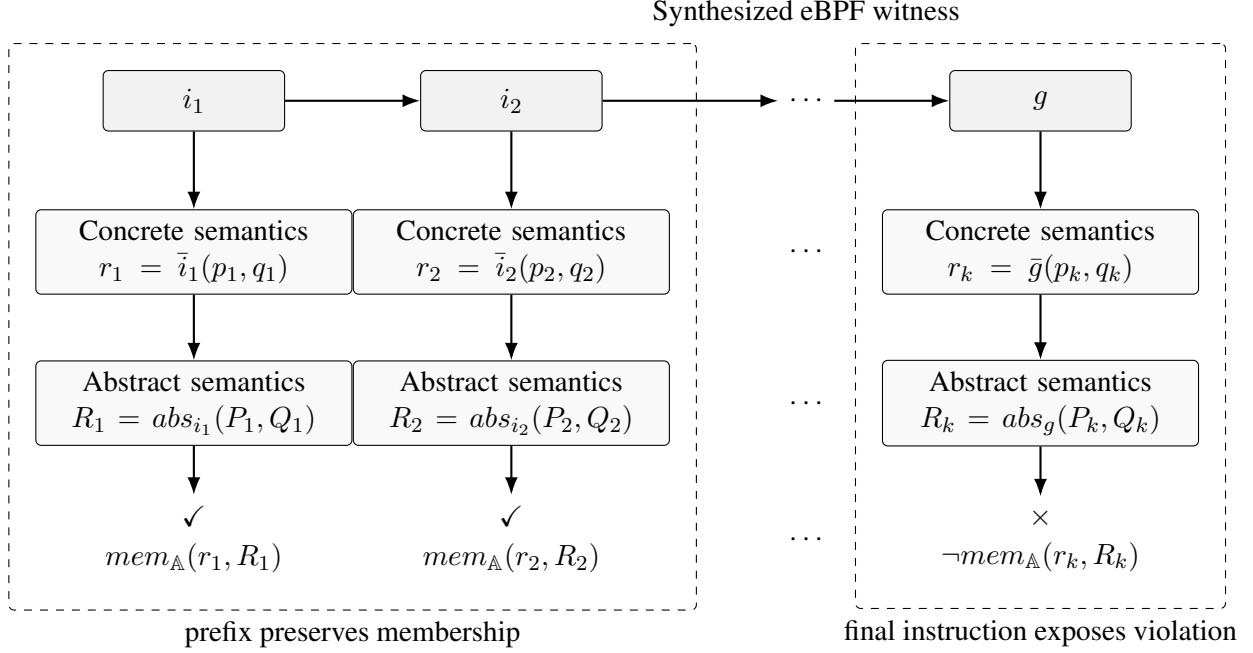


Figure 4.1: Concrete and abstract semantics in a synthesized witness. The prefix instructions construct abstract states that are reachable from the verifier’s initial abstract states and preserve membership. The final instruction is the operator under test; it exposes the violation by producing a concrete result that is not represented by the verifier’s abstract result.

Thus, an abstract value  $P \in \mathbb{A}$  contains one component for each of the verifier’s value-tracking domains. This lets the synthesis encoding use a single abstract value while still referring to the verifier’s full abstract state.

We use lowercase letters for concrete values and uppercase letters for the corresponding abstract values. Thus,  $x \in \mathcal{C}$  is a concrete value, and  $X \in \mathbb{A}$  is a full verifier abstract value that may represent it. We write  $\bar{i}$  for the concrete semantics of instruction  $i$ , and  $\text{abs}_i$  for the verifier’s abstract semantics for the same instruction. For example, if  $i$  is `bpf_or`, then  $\bar{i}(x, y)$  is the concrete bitwise-or of machine values  $x$  and  $y$ .

The predicate  $\text{mem}_{\mathbb{A}}(x, X)$  means that concrete value  $x$  is represented by abstract value  $X$ :

$$\text{mem}_{\mathbb{A}}(x, X) \triangleq x \in \gamma_{\mathbb{A}}(X).$$

For a single interval component with  $X = [\ell, u]$ , this is ordinary interval membership:

$$mem_{int}(x, [\ell, u]) \triangleq \ell \leq x \leq u.$$

For the tnum component, where  $X = (v, m)$ , membership requires the known bits of  $x$  to match  $v$ :

$$mem_{tnum}(x, (v, m)) \triangleq x \& \sim m = v.$$

For the verifier's full value-tracking state,  $mem_{\mathbb{A}}(x, X)$  abbreviates the conjunction of the membership predicates for all five components.

Let  $\mathcal{I} \subseteq \mathbb{A}$  be the set of abstract scalar values that can appear in an initial verifier state. Since  $\mathbb{A}$  is the product of the verifier's value-tracking domains, an element  $X \in \mathcal{I}$  has one component in each domain:

$$X = (X_{u64}, X_{s64}, X_{tnum}, X_{u32}, X_{s32}).$$

Membership in  $\mathcal{I}$  is defined component-wise by the verifier's allowed initial abstract values. For an interval component, the allowed initial values are singleton intervals and  $\top$ :

$$\mathcal{I}_{int} = \{[c, c] \mid c \in \mathcal{C}\} \cup \{\top\}.$$

For the tnum component, the allowed initial values are fully known tnums and  $\top$ :

$$\mathcal{I}_{tnum} = \{(c, 0) \mid c \in \mathcal{C}\} \cup \{\top\}.$$

Thus,  $X \in \mathcal{I}$  means that every component of  $X$  is allowed as part of an initial verifier state.

#### 4.1.2 Single-Instruction Witnesses

The simplest case is  $k = 1$ . There is no synthesized prefix, so the operator under test must be triggered directly from abstract values that can appear in an initial verifier state. The SMT query

still introduces concrete values, but these are not part of the verifier state. They are ghost values used to check whether the verifier's abstract state represents a possible concrete execution.

Suppose the operator under test is `bpf_or`. Let  $p_1, q_1 \in \mathcal{C}$  be the concrete operands, let  $P_1, Q_1 \in \mathbb{A}$  be their abstract operands, and let  $r_1, R_1$  be the concrete and abstract outputs. The single-instruction query is:

$$\begin{aligned}
& \exists p_1, q_1, r_1 \in \mathcal{C}, P_1, Q_1, R_1 \in \mathbb{A}. \\
& \underbrace{P_1 \in \mathcal{I} \wedge Q_1 \in \mathcal{I}}_{\text{choose initial abstract operands}} \\
& \wedge \underbrace{mem_{\mathbb{A}}(p_1, P_1) \wedge mem_{\mathbb{A}}(q_1, Q_1)}_{\text{choose concrete operands represented by them}} \\
& \wedge \underbrace{r_1 = \overline{bpf\_or}(p_1, q_1) \wedge R_1 = abs_{bpf\_or}(P_1, Q_1)}_{\text{relate concrete and abstract semantics}} \\
& \wedge \underbrace{\neg mem_{\mathbb{A}}(r_1, R_1)}_{\text{abstract result does not represent the concrete result}}.
\end{aligned}$$

If this formula is satisfiable, the model gives a one-instruction witness. If it is unsatisfiable, then the violating operands are not reachable directly from the verifier's initial abstract values, and the search must use a longer prefix.

### 4.1.3 Adding a Prefix

For  $k > 1$ , the synthesized prefix can construct operands for the final instruction. Each prefix instruction must preserve membership, while the final instruction must violate it.

Consider the two-instruction skeleton:

$$r_1 = bpf\_and(p_1, q_1); \quad r_2 = bpf\_or(p_2, q_2).$$

Here, `bpf_or` is the operator under test, and `bpf_and` is a prefix instruction. The concrete operands and results are  $p_1, q_1, r_1, p_2, q_2, r_2 \in \mathcal{C}$ , and the corresponding abstract operands and results are

$$P_1, Q_1, R_1, P_2, Q_2, R_2 \in \mathbb{A}.$$

The prefix instruction starts from abstract values that can appear at the beginning of verifier analysis:

$$P_1 \in \mathcal{I} \wedge Q_1 \in \mathcal{I} \wedge \text{mem}_{\mathbb{A}}(p_1, P_1) \wedge \text{mem}_{\mathbb{A}}(q_1, Q_1).$$

The query then relates the prefix inputs and outputs using the concrete and abstract semantics, and requires the output to remain represented:

$$r_1 = \overline{\text{bpf\_and}}(p_1, q_1) \wedge R_1 = \text{abs}_{\text{bpf\_and}}(P_1, Q_1) \wedge \text{mem}_{\mathbb{A}}(r_1, R_1).$$

The final instruction may use either initial abstract values or the prefix output  $(r_1, R_1)$ . To express this dataflow choice, we use a selection predicate. For instruction position  $j$ ,  $\text{select}_j(x, X)$  is defined relative to the result variables of the current instruction sequence:

$$\text{select}_j(x, X) \triangleq \underbrace{X \in \mathcal{I} \wedge \text{mem}_{\mathbb{A}}(x, X)}_{\text{initial abstract value}} \vee \underbrace{\bigvee_{\ell=1}^{j-1} (x = r_{\ell} \wedge X = R_{\ell})}_{\text{output of an earlier instruction}}.$$

The index  $\ell$  ranges over instruction positions earlier than  $j$ . Thus,  $\text{select}_j(x, X)$  either chooses an initial abstract value representing  $x$ , or reuses the concrete and abstract output produced by one earlier instruction in the same candidate sequence. When  $j = 1$ , the disjunction over earlier instruction outputs is empty and is interpreted as false, so  $\text{select}_1$  reduces to the initial-value case.

For the two-instruction skeleton above, the final operands satisfy:

$$\text{select}_2(p_2, P_2) \wedge \text{select}_2(q_2, Q_2).$$

The final instruction is constrained to expose the violation:

$$r_2 = \overline{\text{bpf\_or}}(p_2, q_2) \wedge R_2 = \text{abs}_{\text{bpf\_or}}(P_2, Q_2) \wedge \neg \text{mem}_{\mathbb{A}}(r_2, R_2).$$

Putting the pieces together, the two-instruction query is:

$$\begin{aligned}
& \exists p_1, q_1, r_1, p_2, q_2, r_2 \in \mathcal{C}, \quad P_1, Q_1, R_1, P_2, Q_2, R_2 \in \mathbb{A}. \\
& P_1 \in \mathcal{I} \wedge Q_1 \in \mathcal{I} \wedge \text{mem}_{\mathbb{A}}(p_1, P_1) \wedge \text{mem}_{\mathbb{A}}(q_1, Q_1) \\
& \wedge r_1 = \overline{\text{bpf\_and}}(p_1, q_1) \wedge R_1 = \text{abs}_{\text{bpf\_and}}(P_1, Q_1) \wedge \text{mem}_{\mathbb{A}}(r_1, R_1) \\
& \wedge \text{select}_2(p_2, P_2) \wedge \text{select}_2(q_2, Q_2) \\
& \wedge r_2 = \overline{\text{bpf\_or}}(p_2, q_2) \wedge R_2 = \text{abs}_{\text{bpf\_or}}(P_2, Q_2) \\
& \wedge \neg \text{mem}_{\mathbb{A}}(r_2, R_2).
\end{aligned}$$

Longer straight-line programs are encoded in the same way. Each instruction chooses operands from initial abstract values or earlier outputs. Prefix outputs must remain represented. The final output must violate membership.

#### 4.1.4 General Synthesis Query

The previous examples illustrate the SMT query generated for one enumerated instruction skeleton. For a fixed bound  $k$ , the enumerator chooses an opcode sequence

$$i_1; i_2; \dots; i_{k-1}; g,$$

where  $i_1, \dots, i_{k-1}$  are prefix instructions and  $g$  is the final operator under test. The enumerator fixes only the opcodes in the sequence. The SMT query then asks whether there exist concrete operands, abstract operands, and dataflow choices that instantiate this skeleton as a witness.

For instruction position  $j$ , let  $p_j, q_j \in \mathcal{C}$  be its concrete operands, let  $P_j, Q_j \in \mathbb{A}$  be the corresponding abstract operands, and let  $r_j, R_j$  be its concrete and abstract results. The operands and their sources are symbolic: the solver decides whether each operand comes from an initial abstract value or from the output of an earlier instruction.

The witness query has three parts:

$$\Phi_{witness} \triangleq \underbrace{\Phi_{flow}}_{\text{select operands}} \wedge \underbrace{\Phi_{prefix}}_{\text{construct reachable states}} \wedge \underbrace{\Phi_{violate}}_{\text{reach a violating state}} .$$

The flow constraint chooses the operands of every instruction:

$$\Phi_{flow} \triangleq \bigwedge_{j=1}^k (select_j(p_j, P_j) \wedge select_j(q_j, Q_j)) .$$

The predicate  $select_j$  is the dataflow mechanism in the query. It chooses a concrete/abstract pair either from an initial abstract value or from an earlier instruction output. As above, the empty disjunction is interpreted as false, so for  $j = 1$  both operands must come from initial abstract values.

The prefix constraint relates the inputs and outputs of the first  $k - 1$  instructions and requires every prefix result to remain represented:

$$\Phi_{prefix} \triangleq \bigwedge_{j=1}^{k-1} (r_j = \bar{i}_j(p_j, q_j) \wedge R_j = abs_{i_j}(P_j, Q_j) \wedge mem_{\mathbb{A}}(r_j, R_j)) .$$

Thus, the prefix may construct operands for the final instruction, but each prefix step must preserve membership. This is what enforces reachability: every intermediate abstract value used later in the program must arise from a sound prefix computation starting from valid initial abstract values.

The final instruction is fixed to  $g$ . Its operands are selected by the same flow constraint, but its result is required to violate membership:

$$\Phi_{violate} \triangleq (r_k = \bar{g}(p_k, q_k) \wedge R_k = abs_g(P_k, Q_k) \wedge \neg mem_{\mathbb{A}}(r_k, R_k)) .$$

For this enumerated skeleton, the solver checks satisfiability of:

$$\exists \vec{p}, \vec{q}, \vec{r} \in \mathcal{C}, \vec{P}, \vec{Q}, \vec{R} \in \mathbb{A}. \Phi_{witness} .$$

A satisfying model gives a witness program. It determines the initial abstract values, the concrete values represented by them, the operand choices, the abstract values tracked by the verifier, and the concrete values produced by execution. If the query is unsatisfiable, then this particular opcode skeleton does not yield a witness. If every skeleton at the current bound is unsatisfiable, then no witness was found in the explored search space.

The formulation above describes prefix instructions with one concrete result and one abstract result, such as arithmetic and logic instructions. Branch instructions fit the same search framework, but their outputs have a different shape because they preserve the concrete operands and produce outcome-dependent abstract values. We handle this extension in Section 4.1.5.

#### 4.1.5 Loop-Free Programs with Branches

Branches broaden the set of abstract values available to later instructions. As discussed in Section 2.7.3, a branch does not modify its concrete operands. Instead, it produces abstract values for the true and false branch outcomes. In the synthesis query, we use this behavior to make new concrete/abstract pairs available to subsequent instructions while keeping the generated program loop-free.

We encode branches so that both outcomes continue to the next instruction. Thus, a branch in the prefix is used for its effect on the verifier’s abstract state, not to introduce loops or divergent control flow. Conceptually, a branch takes two concrete/abstract operand pairs and produces two concrete/abstract output pairs:

$$(p_1, P_1), (q_1, Q_1) \rightsquigarrow (p_1, P_1^o), (q_1, Q_1^o).$$

The concrete values  $p_1$  and  $q_1$  are unchanged, while the post-branch abstract values  $P_1^o$  and  $Q_1^o$  are selected according to the concrete branch outcome.

For example, consider the two-instruction skeleton:

$$b_1 = \text{bpf\_jump\_gt64}(p_1, q_1); \quad r_2 = \text{bpf\_or}(p_2, q_2).$$

The branch starts from abstract values that can appear in an initial verifier state:

$$P_1 \in \mathcal{I} \wedge Q_1 \in \mathcal{I} \wedge \text{mem}_{\mathbb{A}}(p_1, P_1) \wedge \text{mem}_{\mathbb{A}}(q_1, Q_1).$$

The query relates the branch inputs to the concrete branch outcome and to the verifier's abstract branch outputs:

$$b_1 = \overline{\text{jump\_gt64}}(p_1, q_1),$$

$$(P_{\text{T}}, P_{\text{F}}, Q_{\text{T}}, Q_{\text{F}}) = \text{abs}_{\text{jump\_gt64}}(P_1, Q_1).$$

The concrete outcome determines which abstract values are available after the branch:

$$b_1 \Rightarrow (P_1^o = P_{\text{T}} \wedge Q_1^o = Q_{\text{T}} \wedge \text{mem}_{\mathbb{A}}(p_1, P_1^o) \wedge \text{mem}_{\mathbb{A}}(q_1, Q_1^o)),$$

$$\neg b_1 \Rightarrow (P_1^o = P_{\text{F}} \wedge Q_1^o = Q_{\text{F}} \wedge \text{mem}_{\mathbb{A}}(p_1, P_1^o) \wedge \text{mem}_{\mathbb{A}}(q_1, Q_1^o)).$$

The final instruction may use initial abstract values or either of the post-branch operand states.

Therefore, each final operand is constrained by a direct disjunction. For the left operand:

$$(P_2 \in \mathcal{I} \wedge \text{mem}_{\mathbb{A}}(p_2, P_2))$$

$$\vee (p_2 = p_1 \wedge P_2 = P_1^o)$$

$$\vee (p_2 = q_1 \wedge P_2 = Q_1^o).$$

For the right operand:

$$(Q_2 \in \mathcal{I} \wedge \text{mem}_{\mathbb{A}}(q_2, Q_2))$$

$$\vee (q_2 = p_1 \wedge Q_2 = P_1^o)$$

$$\vee (q_2 = q_1 \wedge Q_2 = Q_1^o).$$

The final instruction is constrained to expose the violation:

$$r_2 = \overline{\text{bpf\_or}}(p_2, q_2) \wedge R_2 = \text{abs}_{\text{bpf\_or}}(P_2, Q_2) \wedge \neg \text{mem}_{\mathbb{A}}(r_2, R_2).$$

Putting the pieces together, the full query for this two-instruction skeleton is:

$$\begin{aligned}
& \exists p_1, q_1, p_2, q_2, r_2 \in \mathcal{C}, \quad b_1 \in \mathbb{B}, \quad P_1, Q_1, P_1^o, Q_1^o, P_2, Q_2, R_2 \in \mathbb{A}. \\
& P_1 \in \mathcal{I} \wedge Q_1 \in \mathcal{I} \wedge \text{mem}_{\mathbb{A}}(p_1, P_1) \wedge \text{mem}_{\mathbb{A}}(q_1, Q_1) \\
& \wedge b_1 = \overline{\text{jump\_gt64}}(p_1, q_1) \\
& \wedge (P_{\text{T}}, P_{\text{F}}, Q_{\text{T}}, Q_{\text{F}}) = \text{abs}_{\text{jump\_gt64}}(P_1, Q_1) \\
& \wedge (b_1 \Rightarrow (P_1^o = P_{\text{T}} \wedge Q_1^o = Q_{\text{T}} \wedge \text{mem}_{\mathbb{A}}(p_1, P_1^o) \wedge \text{mem}_{\mathbb{A}}(q_1, Q_1^o))) \\
& \wedge (\neg b_1 \Rightarrow (P_1^o = P_{\text{F}} \wedge Q_1^o = Q_{\text{F}} \wedge \text{mem}_{\mathbb{A}}(p_1, P_1^o) \wedge \text{mem}_{\mathbb{A}}(q_1, Q_1^o))) \\
& \wedge ((P_2 \in \mathcal{I} \wedge \text{mem}_{\mathbb{A}}(p_2, P_2)) \vee (p_2 = p_1 \wedge P_2 = P_1^o) \vee (p_2 = q_1 \wedge P_2 = Q_1^o)) \\
& \wedge ((Q_2 \in \mathcal{I} \wedge \text{mem}_{\mathbb{A}}(q_2, Q_2)) \vee (q_2 = p_1 \wedge Q_2 = P_1^o) \vee (q_2 = q_1 \wedge Q_2 = Q_1^o)) \\
& \wedge r_2 = \overline{\text{bpf\_or}}(p_2, q_2) \wedge R_2 = \text{abs}_{\text{bpf\_or}}(P_2, Q_2) \\
& \wedge \neg \text{mem}_{\mathbb{A}}(r_2, R_2).
\end{aligned}$$

This query has the same overall structure as the straight-line synthesis query. The difference is that a branch contributes multiple concrete/abstract pairs for later use; its concrete operands are preserved, but their abstract values are refined according to the concrete branch outcome.

#### 4.1.6 Validation of Generated Witnesses

The SMT model gives a candidate eBPF program together with concrete operands and predicted abstract states. We validate generated witnesses by checking that the same mismatch appears in the actual kernel verifier.

For bugs with known proof-of-concept programs, the synthesized witnesses exhibit the same underlying verifier behavior. For previously unknown bugs, we instrument the corresponding kernel version to log the verifier’s abstract state while it analyzes the generated eBPF program. We also execute the program using the concrete operands reported by the SMT model. We then compare the logged verifier state and the concrete runtime result against the model.

For the manifested bugs we checked, the observed verifier states and concrete results agreed

with the SMT model. This validation confirms that the synthesized programs do not merely satisfy the logical encoding, but expose the same concrete-versus-abstract mismatch in the kernel verifier implementation.

## 4.2 Guarantees and Limitations

Differential synthesis is sound with respect to the modeled semantics but incomplete. Soundness means that any model of  $\Phi_{witness}$  gives a witness within the encoding: the synthesized program starts from abstract values allowed in an initial verifier state, the prefix constructs reachable abstract states while preserving membership, and the final instruction produces a concrete result not represented by the verifier’s abstract result.

Incompleteness comes from the bounded nature of the search. Failure to find a model only rules out witnesses in the explored search space; the violation may require a longer instruction sequence, a different instruction skeleton, or features not included in the encoding. Thus, an unsatisfiable synthesis query does not prove that the original soundness violation is unreachable.

## 4.3 Differential Synthesis in Practice

We evaluate differential synthesis as a witness-generation technique. The inputs to synthesis are operator-domain violations reported by Agni’s verification check under the refined soundness specification introduced in Section 2.8 and formalized in Equation 2.2. Each violation corresponds to an operator and abstract domain for which the verification condition exhibits unsound behavior. Given these reported violations, we ask whether differential synthesis can turn them into concrete eBPF witness programs.

The goal of this evaluation is to determine whether synthesis is useful as a practical debugging tool. When synthesis succeeds, it produces an executable program that manifests the reported violation within the explored bound. When synthesis fails within that bound, the result is inconclusive, but still useful: it identifies violations that may require a larger bound or further manual reachability analysis.

| Kernel<br>version | Total<br>violations | All witnesses<br>generated? | Witness length |    |   | P90<br>time |
|-------------------|---------------------|-----------------------------|----------------|----|---|-------------|
|                   |                     |                             | 1              | 2  | 3 |             |
| 4.14              | 21                  | ✗                           | 14             | 4  | 0 | ≈ 10 min    |
| 5.5               | 30                  | ✗                           | 0              | 20 | 2 | ≈ 60 min    |
| 5.7-rc1           | 99                  | ✓                           | 55             | 44 | 0 | ≈ 10 min    |
| 5.7               | 67                  | ✓                           | 39             | 28 | 0 | ≈ 40 min    |
| 5.8               | 67                  | ✓                           | 39             | 28 | 0 | ≈ 5 min     |
| 5.9               | 65                  | ✓                           | 39             | 26 | 0 | ≈ 5 min     |
| 5.10              | 65                  | ✓                           | 19             | 44 | 2 | ≈ 5 min     |
| 5.10-rc1          | 71                  | ✓                           | 39             | 32 | 0 | ≈ 40 min    |
| 5.11              | 62                  | ✓                           | 16             | 44 | 2 | ≈ 60 min    |
| 5.12              | 62                  | ✓                           | 16             | 44 | 2 | ≈ 60 min    |

Table 4.1: Differential synthesis results across Linux kernel versions. The table reports the number of soundness violations considered, whether the synthesizer generated witnesses for all violations in that version, the distribution of generated witnesses by instruction-sequence length, and  $P90$  time; the elapsed synthesis time by which 90% of the generated witnesses for that kernel version have been found.

#### 4.3.1 Generating Witness Programs for Unsound Kernels

We first evaluate whether differential synthesis can generate eBPF programs that manifest reported verifier bugs. Starting from the operator-domain pairs reported by Agni’s verification check, we run the synthesizer to search for bounded eBPF programs whose concrete execution diverges from the verifier’s abstract execution. A generated program is counted as a witness when it drives the verifier to the relevant abstract state and exposes the violation on the final instruction.

Table 4.1 summarizes the results. For each kernel version, the table reports the number of reported operator-domain violations, whether differential synthesis generated witnesses for all of them, the distribution of generated witnesses by instruction-sequence length, and the  $P90$  witness-generation time. Across the analyzed kernel versions, differential synthesis generated witnesses for  $\approx 97\%$  of the reported violations.

The results show that many violations cannot be manifested by a single instruction. In several cases, the smallest witness requires a multi-instruction program because the violating abstract input state must first be made reachable from the verifier’s initial abstract states. For example, in kernel

version 5.5, none of the reported violations could be manifested with a single eBPF instruction. Differential synthesis was nevertheless able to generate multi-instruction witnesses for most of them.

Differential synthesis generated witnesses for all reported violations in all but two kernel versions. For kernel versions 4.14 and 5.5, it generated witnesses for all but three and eight violations, respectively. The remaining violations may require longer instruction sequences than the bounds used in our experiments, or they may correspond to failures that are not reachable during actual verifier execution. Thus, a failed synthesis attempt does not prove that the violation is unreachable; it only means that no witness was found within the explored search space.

These results show that differential synthesis can turn most reported local violations into executable eBPF programs. This is useful for debugging because a witness program demonstrates the mismatch in the setting that matters to the verifier, analysis of an actual eBPF program. The few cases where no witness is found are also useful because they identify violations that may require a larger search bound or further manual reachability analysis.

### 4.3.2 Witness-Generation Performance

We conducted our experiments on the CloudLab [52] testbed using a machine with two 10-core Intel Skylake CPUs running at 2.20 GHz and 192GB of memory. We measure the time required to synthesize witness programs from the reported violations.

Table 4.1 reports *P90* witness-generation time for each kernel version. *P90* is the elapsed synthesis time by which 90% of the generated witnesses for that kernel version have been found. Across all analyzed versions, differential synthesis reached this point within approximately 5–60 minutes. This shows that most witnessable violations are discovered quickly.

The remaining runtime is dominated by the hardest cases. These typically correspond to violations that require longer instruction sequences or more complex operand and dataflow choices. Increasing the witness bound expands the number of opcode skeletons considered by the enumerator and increases the number of symbolic operands, abstract values, and dataflow choices in the

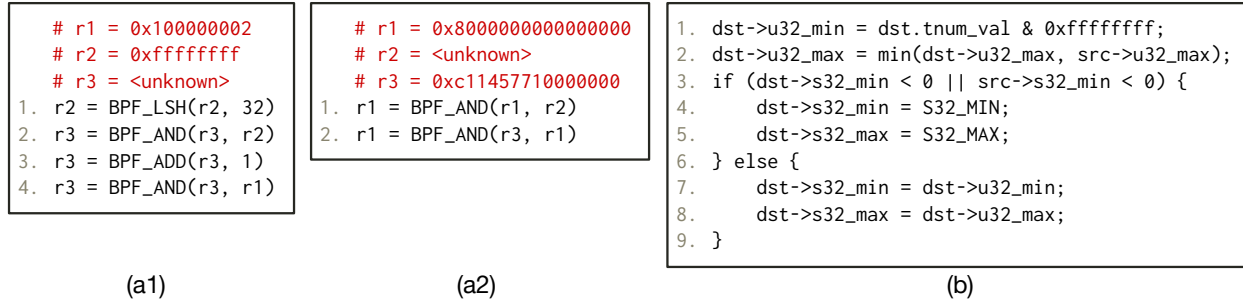


Figure 4.2: Comparison of manually crafted and synthesized witnesses. (a1) A manually crafted exploit. (a2) A POC produced by our synthesis approach. Instructions in red only set values in registers and are not counted as part of the exploit; instructions in black are counted. (b) An excerpt from the function `scalar32_min_max_and` in Linux kernel v5.13.

generated SMT queries.

### 4.3.3 Comparison with Existing Proofs of Concept

Some verifier bugs are known security vulnerabilities with manually written proof-of-concept programs [77, 87]. We compare the witnesses generated by differential synthesis against these known programs.

For every known CVE present in the kernel versions we analyzed, differential synthesis generated a witness program of equal or smaller size. Figure 4.2 shows one representative comparison. Figure 4.2(a1) shows a hand-crafted POC that triggers a known vulnerability in the verifier’s `scalar32_min_max_and` function [18]. Figure 4.2(a2) shows a POC produced by our synthesizer for the same bug. The synthesized program is generated automatically and is smaller than the hand-crafted proof of concept, requiring two instructions instead of four.

This result highlights a practical advantage of synthesis. The generated witnesses are not only automatic, but often smaller than hand-written programs, making them easier to inspect, debug, and use as regression tests.

## 4.4 Summary

This chapter presented differential synthesis, a bounded witness-generation technique for local verifier soundness violations. The key observation is that an SMT counterexample to a soundness

condition is not, by itself, an executable bug report: the abstract operands in the counterexample must also be reachable from the verifier’s initial abstract states. Differential synthesis addresses this by searching for loop-free eBPF instruction sequences that construct the required abstract operands and then expose the mismatch on a final instruction.

The encoding combines opcode enumeration with SMT solving. The enumerator fixes the instruction skeleton, while the solver chooses concrete values, abstract values, operand sources, and dataflow. Prefix instructions must preserve membership, and the final instruction must violate it. Branches fit this framework by adding outcome-dependent abstract values without changing the underlying concrete values.

The approach is sound with respect to the modeled semantics but incomplete because the search is bounded. A satisfying model yields an executable witness within the encoding; an unsatisfiable query only shows that no witness was found in the explored search space.

The evaluation shows that this bounded search is useful in practice. Starting from operator-domain violations reported by Agni’s verification check, differential synthesis generated witnesses for  $\approx 97\%$  of the reported violations, found 90% of generated witnesses within approximately 5–60 minutes for every analyzed kernel version, and produced witnesses for all known CVEs in the analyzed versions. The generated programs are often smaller than manually written proofs of concept, making them useful for validating, debugging, and regression testing verifier bugs.

## CHAPTER 5

### DIFFERENTIAL SYNTHESIS FOR COMPARING PRECISION OF ABSTRACT OPERATORS

The previous chapters focused on soundness. They asked whether the verifier’s abstract operators safely overapproximate concrete eBPF execution, and how to generate executable witnesses when this soundness condition fails. Soundness is the first requirement for the verifier; if the verifier accepts an unsafe program, then the kernel can be compromised. However, soundness alone is not enough for a useful verifier. The verifier must also be precise enough to accept safe programs that should be allowed to run.

This chapter shifts from soundness to precision. Instead of asking whether one abstract operator safely represents concrete execution, we compare two abstract operator implementations; an existing implementation and a proposed replacement. The central question is whether the new operator is at least as precise as the old operator, and whether it is strictly more precise on some inputs. This kind of comparison is important because verifier operators are handwritten and often involve subtle interactions between interval domains, bit-level information, and shared reduction logic.

A formal precision comparison is useful, but it is not by itself enough to show that a precision improvement matters in practice. The eBPF verifier does not analyze arbitrary abstract states. It starts from restricted initial states and reaches later states only by applying the verifier’s abstract operators, branch-transfer logic, and reduction logic. Therefore, a precision gap between two operators may exist only on abstract inputs that no real eBPF program can reach. Such a gap is formally valid, but it does not explain any observable verifier behavior.

To address this reachability gap, this chapter adapts differential synthesis from the previous chapter. For soundness, differential synthesis searched for a reachable abstract state in which the verifier’s abstract result failed to represent the concrete result. For precision, both operators are

assumed sound, and the goal is to find a reachable abstract state where one operator produces a strictly more precise result than the other. A synthesized program therefore serves as a precision witness. It demonstrates that the improvement is not only a lattice-theoretic fact about abstract inputs, but an observable difference in the verifier’s analysis of an actual eBPF program.

We develop this approach through a case study on the verifier’s multiplication operator, whose existing implementation loses precision on some signed inputs. We then define logical precision specifications for comparing abstract operators, extend witness synthesis to generate concrete eBPF programs that expose reachable precision gaps, and evaluate the approach on recent precision-related kernel patches. Finally, we use the same framework to measure the precision contribution of the verifier’s shared reduction operator, `BPF_SYNC`.

## 5.1 Why Precision Matters in the eBPF Verifier

The eBPF verifier must ensure that its static analysis is not only sound, but also precise enough for practical use. A sound but imprecise analysis can reject safe programs that should be accepted, simply because the verifier’s abstract state overapproximates program behavior too coarsely. This problem is especially important for values used in memory accesses. If the verifier loses tight bounds on an array index or pointer offset, it may be unable to prove that a later access is safe.

As discussed in the background chapter, the verifier tracks scalar values using several interval domains and the `tnum` bitwise domain, and it uses cross-domain reduction to propagate information between them. The precision of the verifier’s final abstract state therefore depends on both the individual abstract operators and their interaction with reduction.

Consider the simple program fragment in Figure 5.1, where a register is masked before being used as an array index. The mask guarantees that `r1` is one of  $\{0, 1, 2, 3\}$ , so the access to `arr[r1]` stays within the four-byte array. A precise abstract AND operator should preserve this fact. For example, after the mask the verifier should be able to infer that `r1` lies in the interval  $[0, 3]$ .

A sound but imprecise abstraction may instead produce a wider interval, such as  $[0, 8]$ . This interval still overapproximates the concrete values of `r1`, so it is sound. However, it is too coarse

---

```

1  /* arr has 4 bytes */
2  ...
3  r1 = r1 & 0b0011;    // keep only the low two bits
4  r0 = arr[r1];        // access index 0, 1, 2, or 3
5  exit;

```

---

Figure 5.1: Safe array access after masking an index register.

to prove the array access safe because it includes possible indices larger than 3. The verifier may therefore reject the program even though the concrete execution cannot access outside the array.

Similar safe rejections have prompted several recent kernel patches that improve the precision of verifier abstract operators [2, 4, 7–9, 21].

Designing efficient and precise abstract operators is difficult. Classical interval operators are well understood [48, 81], but fixed-width machine arithmetic, bit-level operations, and verifier performance constraints complicate their direct use. Bitwise operations are particularly challenging for interval domains because the result often depends on bit-level structure rather than only on numeric endpoints.

For example, consider bitwise and over unsigned intervals where

$$x \in [\underline{P}, \overline{P}] = [142, 145], \quad y \in [\underline{Q}, \overline{Q}] = [13, 15].$$

A simple sound interval abstraction can return

$$[0, \min(\overline{P}, \overline{Q})] = [0, 15].$$

This interval is sound because, for unsigned values,  $x \& y$  cannot be larger than either operand. However, it is imprecise. The actual possible results are only  $\{0, 1, 12, 13, 14, 15\}$ . Thus,  $[0, 15]$  includes values such as  $2, \dots, 11$  that cannot occur. The exact result set is sparse and cannot be represented precisely by a single interval. This is why bitwise operations require reasoning about bit-level structure, and why the verifier uses the `tnum` domain alongside its interval domains [108].

Even with this extra domain, combining bitwise information with signed and unsigned interval information precisely and efficiently remains difficult and has required multiple fixes in practice [4, 5, 7, 83, 97].

Overall, manual operator design in the verifier is labor-intensive and error-prone. Kernel developers often rely on test cases to validate changes to abstract operators [6], but test cases provide only example-based evidence. They do not establish whether a new implementation is at least as precise as the old one for all inputs, nor whether any precision improvement can arise in real verifier executions. This motivates both a formal precision comparison between competing operators and a witness-generation procedure for demonstrating reachable improvements.

The next section defines the verifier-specific precision comparison used in this chapter. It also explains how this comparison interacts with the verifier’s shared reduction operator, which propagates information between the scalar domains.

## 5.2 Reduction and Precision Comparison in the Verifier

The previous section motivated the need to compare verifier abstract operators by precision. This section defines the comparison used throughout the chapter. We first recall the standard abstract-interpretation view of precision, explain why the verifier’s scalar domain does not fit that view directly, and then define the component-wise comparison used for verifier abstract values. We then describe the verifier’s shared reduction operator and define the precision preorder for comparing abstract operator implementations.

### 5.2.1 Precision in the Verifier’s Product Domain

The abstract interpretation literature provides a standard way to compare precision by ordering abstract values in the abstract domain. In the usual interpretation, an abstract value is more precise when its concretization is smaller. Thus, if  $a_1 \sqsubseteq a_2$ , then  $a_1$  represents no more concrete values than  $a_2$ . For product domains, this comparison is often understood through a product concretization that combines information from the components into a single concrete set, typically by intersecting

component concretizations.

The eBPF verifier's scalar domain does not fit this intersection-based view directly. As in the previous chapters, the verifier tracks scalar values using

$$\mathbb{A} = \mathbb{A}_{u64} \times \mathbb{A}_{s64} \times \mathbb{A}_{tnum} \times \mathbb{A}_{u32} \times \mathbb{A}_{s32}.$$

The interval components track signed and unsigned bounds at 64-bit and 32-bit widths, while  $\mathbb{A}_{tnum}$  tracks bitwise information. These components describe the same machine value under different interpretations. Treating the product concretization as the intersection of all component concretizations would not match the verifier's intended semantics. For example, intersecting signed and unsigned interval concretizations can eliminate valid negative values, and intersecting 32-bit and 64-bit concretizations can incorrectly restrict a 64-bit value to a 32-bit range.

For this reason, we compare verifier abstract values component-wise. We write a verifier abstract value as

$$X = (X_{u64}, X_{s64}, X_{tnum}, X_{u32}, X_{s32}) \in \mathbb{A}.$$

Let  $\mathcal{D} = \{u64, s64, tnum, u32, s32\}$  be the set of component domains. For  $d \in \mathcal{D}$ ,  $X_d$  denotes the component of  $X$  in domain  $d$ , and  $\sqsubseteq_d$  denotes that component's abstract order. The verifier product order is

$$X \sqsubseteq Y \iff \forall d \in \mathcal{D}. X_d \sqsubseteq_d Y_d.$$

Thus,  $X$  is at least as precise as  $Y$  when each component of  $X$  is at least as precise as the corresponding component of  $Y$ .

We also keep concretizations component-wise. If  $\gamma_d$  is the concretization for component  $d$ , then

$$\gamma^*(X) = (\gamma_d(X_d))_{d \in \mathcal{D}}$$

denotes the tuple of component concretizations. We write  $\sqsubseteq^*$  for component-wise inclusion over such tuples. With this notation,  $X \sqsubseteq Y$  implies  $\gamma^*(X) \sqsubseteq^* \gamma^*(Y)$ . This component-wise compari-

son is the precision paradigm used throughout the rest of the chapter.

### 5.2.2 The Reduction Operator in the eBPF Verifier

The verifier propagates information among the components of  $\mathbb{A}$  using shared reduction logic, implemented as `BPF_SYNC`. This operator is shared across the verifier, it is not specific to one eBPF instruction, but is invoked after many instruction-specific abstract updates. Following the notation from the previous chapter, we write the instruction-specific abstract update for instruction  $g$  as  $op_g$ , and the shared reduction operator as  $sro$ . The full abstract operator used by the verifier is

$$abs_g = sro \circ op_g.$$

The operator  $op_g$  performs the instruction-specific transfer. For example, for an arithmetic instruction it updates the interval and `tnum` components according to the operation being analyzed. The reduction operator  $sro$  then refines the result by propagating information between domains. In this way, the final abstract state for an instruction depends not only on the operator-specific logic, but also on the shared reduction logic applied after it.

The reduction step matters because information in one verifier domain can constrain another. For example, if the `tnum` component shows that the high bits of a value are known to be zero, then the unsigned interval can be tightened to a smaller upper bound. If an unsigned interval is known to be small and non-negative, the signed interval can often be tightened as well. In the other direction, tight interval bounds can reveal bits that must be fixed, allowing the verifier to refine the `tnum` component.

At a high level,  $sro$  performs three kinds of propagation. First, it uses `tnum` information to tighten signed and unsigned intervals. Second, it propagates bounds between signed and unsigned domains and between 64-bit and 32-bit domains. Third, it uses the refined intervals to recover fixed bits in the `tnum` domain. These steps may enable one another; refining an interval can expose new bit-level information, and refining bit-level information can in turn tighten intervals.

The verifier does not compute an exact reduced product to a fixed point. Instead, it applies this reduction logic only a bounded number of times. This resembles a partially reduced product [81]. In a fully reduced product, reduction would be applied until no component can be further refined using information from the others. Such a fixed point can expose all precision available through the chosen reductions, but it may require repeated cross-domain propagation and can be too expensive for an analysis that runs inside the kernel. A partially reduced product applies only selected reductions, or applies them for a bounded number of rounds, gaining some of the precision benefits without paying the full computation cost.

The verifier follows this latter strategy. Its reduction logic performs a fixed sequence of verifier-specific refinements over the five scalar components, and the sequence is repeated only a small bounded number of times. Thus, BPF\_SYNC is not a generic reduced-product construction, but it plays the same practical role; it propagates useful information across domains while keeping verifier analysis predictable and efficient.

The previous chapter proved the soundness of this shared reduction operator. In this chapter, we focus on its precision role. In particular, the evaluation compares abstract operators with and without *sro* to measure when BPF\_SYNC strictly improves analysis precision.

### 5.2.3 Comparing the Precision of Abstract Operators in the Verifier

We now define the precision comparison used for verifier abstract operators. For a fixed instruction  $g$ , let  $abs_g^a$  and  $abs_g^b$  be two sound implementations of the abstract operator for  $g$ . We say that  $abs_g^a$  is *at least as precise as*  $abs_g^b$ , written  $abs_g^a \preceq abs_g^b$ , when

$$\forall X \in \mathbb{A}. abs_g^a(X) \subseteq abs_g^b(X).$$

For instructions with multiple operands, both implementations are compared on the same tuple of abstract operands.

Intuitively,  $abs_g^a \preceq abs_g^b$  means that the first operator never produces a less precise abstract

result than the second. Equivalently, for every input  $X$ , the component concretizations of  $abs_g^a(X)$  are included in the corresponding component concretizations of  $abs_g^b(X)$ . If both directions hold, the operators are equally precise. If neither direction holds, they are incomparable.

We say that  $abs_g^a$  is *strictly more precise than*  $abs_g^b$ , written  $abs_g^a \prec abs_g^b$ , when  $abs_g^a \preceq abs_g^b$  and the reverse preorder does not hold. Equivalently, there is some input  $X \in \mathbb{A}$  and some component  $d \in \mathcal{D}$  such that

$$(abs_g^a(X))_d \sqsubset_d (abs_g^b(X))_d.$$

When comparing two versions of an operator, we instantiate the generic notation as  $abs_g^{\text{new}}$  and  $abs_g^{\text{old}}$ . The desired relationship for a precision-improving replacement is

$$abs_g^{\text{new}} \preceq abs_g^{\text{old}} \quad \text{and} \quad abs_g^{\text{old}} \not\preceq abs_g^{\text{new}}.$$

The first condition says that the replacement never loses precision, and the second says that it is strictly more precise for some abstract input.

For reduction, we instantiate the same comparison by comparing the instruction-specific update alone against the full operator with reduction:

$$abs_g^{-R} = op_g \quad \text{and} \quad abs_g^R = sro \circ op_g.$$

If  $abs_g^R \preceq abs_g^{-R}$ , then applying reduction never makes the result less precise. If the reverse preorder fails, then reduction strictly improves precision for that instruction on some abstract input.

This preorder gives a formal comparison over abstract inputs. The next section uses multiplication as a concrete case study, showing how such a comparison arises when designing a more precise verifier operator.

### 5.3 Improving the Precision of Abstract Operators in the eBPF Verifier

While exploring abstract operators in the eBPF verifier, we observe that some operators perform overly conservative approximations of their concrete counterparts, which results in loss of precision. Specifically, the verifier’s multiplication operator exhibits this behavior when given negative operands. To tackle this imprecision, we first need to formalize how the verifier performs this operator and identify where the imprecision stems from. We then craft a new operator that is at least as precise as the old operator for all abstract inputs, but also more precise for some inputs. This section presents these efforts. We first describe how the abstract operator for multiplication (`bpf_mul`) is performed by the eBPF verifier and where imprecision stems from. Then we propose an improved multiplication abstract operator and discuss how its precision merits over the prior operator can be established using the comparison relation from the previous section.

Beyond serving as a stand-alone improvement, this case study motivates the need for a more systematic approach to design and validate abstract operators. Crafting a more precise operator is only part of the challenge. Ensuring that this improved precision is relevant to real eBPF programs is equally important. In particular, reasoning about all abstract inputs can overapproximate what is actually reachable in verifier executions, since the verifier begins analysis from constrained initial states and enforces strict invariants on how abstract states evolve. As such, many abstract inputs used in theoretical comparisons may never arise in practice. We present our techniques for precision comparison and witness generation in the following section (§5.4).

***How the eBPF verifier performs abstract multiplication (`bpf_mul`).*** Formally, we represent the existing multiplication operator in the eBPF verifier as  $abs_{\text{ebpf-mul}} : \mathbb{A}_{mul} \times \mathbb{A}_{mul} \rightarrow \mathbb{A}_{mul}$ , where  $\mathbb{A}_{mul} = \mathbb{A}_{u64} \times \mathbb{A}_{s64}$ . While the verifier uses five abstract domains, we focus on these two domains for clarity and brevity, as these domains are enough to illustrate how imprecision manifests in this operator.

**Operands.** We define the operator's operands  $P, Q \in \mathbb{A}_{mul}$  as

$$P = ([\underline{P}_{u64}, \overline{P}_{u64}], [\underline{P}_{s64}, \overline{P}_{s64}]), \quad Q = ([\underline{Q}_{u64}, \overline{Q}_{u64}], [\underline{Q}_{s64}, \overline{Q}_{s64}]),$$

where unsigned and signed interval invariants are maintained:

$$0 \leq \underline{(\cdot)}_{u64} \leq \overline{(\cdot)}_{u64} \leq 2^{64} - 1$$

and

$$-2^{63} \leq \underline{(\cdot)}_{s64} \leq \overline{(\cdot)}_{s64} \leq 2^{63} - 1.$$

**Bit-width constants.** The following constants represent the minimum and maximum numeric values that can be represented using a fixed number of bits, as used by the verifier, when interpreted as signed or unsigned integers:

$$\begin{aligned} U64_{\min} &= 0, & U64_{\max} &= 2^{64} - 1, \\ S64_{\min} &= -2^{63}, & S64_{\max} &= 2^{63} - 1, \\ U32_{\max} &= 2^{32} - 1. \end{aligned}$$

**Operator definition as represented by the eBPF verifier.** The existing verifier multiplication operator can be summarized as follows:

$$abs_{ebpf-mul}(P, Q) = \begin{cases} \top, & \text{if } \underline{P}_{s64} < 0 \vee \underline{Q}_{s64} < 0, \\ \top, & \text{if } \overline{P}_{u64} > U32_{\max} \vee \overline{Q}_{u64} > U32_{\max}, \\ ([\underline{P}_{u64} \cdot \underline{Q}_{u64}, \overline{P}_{u64} \cdot \overline{Q}_{u64}], \Delta_{s64}), & \text{otherwise.} \end{cases}$$

Here  $\top = ([U64_{\min}, U64_{\max}], [S64_{\min}, S64_{\max}])$  is the greatest element of  $\mathbb{A}_{mul}$ , and

$$\Delta_{s64} = \begin{cases} [S64_{\min}, S64_{\max}], & \text{if } \bar{P}_{u64} \cdot \bar{Q}_{u64} > S64_{\max}, \\ [\underline{P}_{u64} \cdot \underline{Q}_{u64}, \bar{P}_{u64} \cdot \bar{Q}_{u64}], & \text{otherwise.} \end{cases}$$

The operator  $abs_{ebpf-mul}$  computes precise interval products under restrictive conditions. Both operands must be non-negative and fit within 32-bit unsigned bounds, and the resulting product must not exceed the maximum representable 64-bit signed value,  $S64_{\max}$ . Every other case is conservatively widened to the full 64-bit range. While this ensures soundness, it significantly limits the utility of the operator in practice, especially for signed computations, leading to overly conservative analyses and potential rejection of safe programs. As shown in Table 5.1, this behavior results in avoidable precision losses. To address this issue, we present an improved multiplication operator that retains soundness while computing tighter interval bounds in many of these previously imprecise cases.

***Our more precise eBPF multiplication abstract operator.*** Our operator is grounded in interval arithmetic [82], which has become standard in abstract interpretation literature [48, 81]. Specifically, we adapt interval multiplication to 64-bit fixed-width arithmetic, accounting for overflow as defined by the eBPF instruction set [72]. We formally define our abstract multiplication operator as

$$abs_{our-mul} : \mathbb{A}_{mul} \times \mathbb{A}_{mul} \rightarrow \mathbb{A}_{mul}.$$

We use these two domains for brevity, but the operator is also directly extensible to the  $\mathbb{A}_{u32}$  and  $\mathbb{A}_{s32}$  domains. Our operator preserves the existing abstract multiplication for the  $tnum$  domain defined elsewhere [108]. Next, we define auxiliary products which we use to compute intervals:

$$\begin{aligned} U_{\ell\ell} &= \underline{P}_{u64} \cdot \underline{Q}_{u64}, & U_{uu} &= \bar{P}_{u64} \cdot \bar{Q}_{u64}, \\ S_{\ell\ell} &= \underline{P}_{s64} \cdot \underline{Q}_{s64}, & S_{\ell u} &= \underline{P}_{s64} \cdot \bar{Q}_{s64}, \\ S_{u\ell} &= \bar{P}_{s64} \cdot \underline{Q}_{s64}, & S_{uu} &= \bar{P}_{s64} \cdot \bar{Q}_{s64}. \end{aligned}$$

Lastly, let

$$prod_{\min} = \min\{S_{\ell\ell}, S_{\ell u}, S_{u\ell}, S_{uu}\}$$

and

$$prod_{\max} = \max\{S_{\ell\ell}, S_{\ell u}, S_{u\ell}, S_{uu}\}.$$

Our improved multiplication operator is defined as follows:

$$\boxed{abs_{\text{our-mul}}(P, Q) = (\Delta_{u64}, \Delta_{s64})}$$

where

$$\Delta_{u64} = \begin{cases} [U64_{\min}, U64_{\max}], & \text{if overflow occurs in } U_{\ell\ell} \text{ or } U_{uu}, \\ [U_{\ell\ell}, U_{uu}], & \text{otherwise,} \end{cases}$$

and

$$\Delta_{s64} = \begin{cases} [S64_{\min}, S64_{\max}], & \text{if overflow occurs in one of } S_{\ell\ell}, S_{\ell u}, S_{u\ell}, S_{uu}, \\ [prod_{\min}, prod_{\max}], & \text{otherwise.} \end{cases}$$

Our operator multiplies the operands' interval bounds, returning precise unsigned and signed 64-bit product intervals when all auxiliary products do not overflow. If any product overflows, the operator conservatively widens the affected domain to the full 64-bit range. Table 5.1 exemplifies precision gains of our operator over the existing one.

***Bridging formal precision and real-world execution.*** The operator above is designed to be never less precise than the kernel's current operator:

$$abs_{\text{our-mul}} \preceq abs_{\text{ebpf-mul}}.$$

However, even if this preorder holds for all abstract inputs, it does not by itself show whether the improvement is practically more precise in real eBPF executions. Abstract inputs for which our operator is more precise may not be reachable under the verifier's constraints. Further, most of the

| Operands $P, Q$                                          | $abs_{ebpf-mul}(P, Q)$ | $abs_{our-mul}(P, Q)$                  |
|----------------------------------------------------------|------------------------|----------------------------------------|
| $([2, 3], [2, 3]), ([4, 5], [4, 5])$                     | $([8, 15], [8, 15])$   | $([8, 15], [8, 15])$                   |
| $([1, 1], [1, 1]), ([0, 0], [-1, -1])$                   | $\top$                 | $([0, 0], [-1, -1])$                   |
| $([2^{34}, 2^{34}], [2^{34}, 2^{34}]), ([1, 1], [1, 1])$ | $\top$                 | $([2^{34}, 2^{34}], [2^{34}, 2^{34}])$ |
| $([0, 5], [0, 5]), ([0, 2], [-2, 2])$                    | $\top$                 | $([0, 10], [-10, 10])$                 |
| $([2^{63}, 2^{63}], [0, 2^{62}]), ([2, 8], [2, 8])$      | $\top$                 | $\top$                                 |

Table 5.1: Precision gains from the improved multiplication operator. The table compares the verifier’s multiplication operator  $abs_{ebpf-mul}$  with our improved multiplication operator  $abs_{our-mul}$ , for operands  $P, Q \in \mathbb{A}_{mul} = \mathbb{A}_{u64} \times \mathbb{A}_{s64}$ . Each operand is written as a pair of unsigned and signed intervals. In cases where the verifier’s operator widens to  $\top$ , our operator can still produce tighter bounds when the relevant products do not overflow.

verifier’s operators are handcrafted by kernel developers and may involve subtle bit-level invariants or interactions with multiple abstract domains in non-obvious ways, such as bitwise operations for signed intervals. This means that we cannot rely on intuition or isolated examples to assess their precision.

Hence, when a new abstract operator is proposed, in our case the multiplication operator, we argue it should achieve two goals. First, it should satisfy the precision preorder for all abstract inputs, or for a region of inputs of interest. Second, it should demonstrate concrete eBPF programs in which that improved precision is observable. For multiplication, this means programs for which the old operator widens to  $\top$ , while the new operator produces a tight bound.

The first task reduces to a lattice-theoretic proof obligation. The second task amounts to searching the program space for witnesses where precision differences manifest. Performing this search by hand is error-prone and does not scale to the many arithmetic, bitwise, and branch operators maintained in the verifier. Hence, we propose an automated pipeline that encodes precision specifications as logical formulas, systematically enumerates bounded eBPF instruction sequences, and invokes an SMT solver to find a satisfying assignment that separates two operators in terms of precision. The next section introduces this synthesis-driven framework for precision comparison and witness generation.

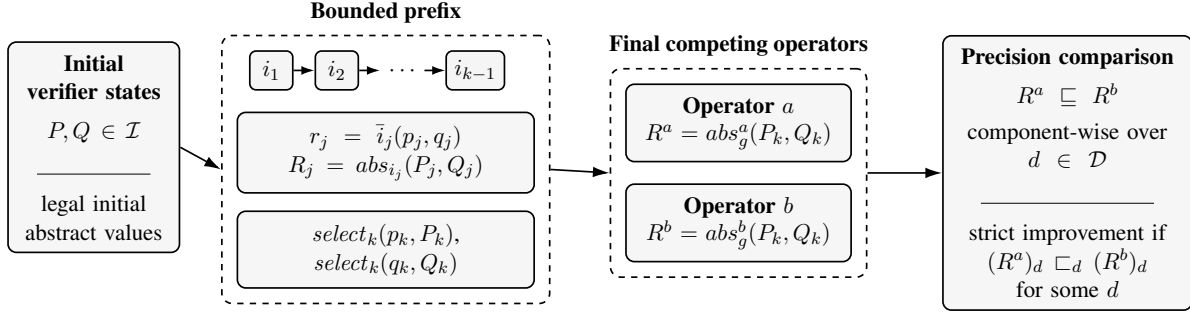


Figure 5.2: Overview of precision comparison with a bounded prefix. Starting from legal initial verifier states, a bounded instruction prefix constructs reachable operands for the final instruction  $g$ . Two competing abstract operators for  $g$  are then applied to the same operands, and their results are compared component-wise across the verifier domains.

#### 5.4 Precision Specifications and Witness Synthesis

The multiplication case study illustrates the two questions addressed by our framework: how to prove a precision relation between two abstract operators, and how to find concrete eBPF programs where a strict precision improvement is observable. This section gives the logical specifications for both tasks.

Figure 5.2 gives an overview of this workflow. Starting from legal initial verifier states, the synthesis procedure constructs a bounded instruction prefix that reaches operands for a final instruction  $g$ . The two competing abstract operators for  $g$  are then applied to the same reachable operands, and their results are compared component-wise across the verifier domains. This separates the abstract precision question from the reachability question: the comparison asks which operator is more precise on the selected operands, while the prefix establishes that those operands can arise in a real verifier execution.

We first encode the precision preorder as an SMT query over the two operator implementations. We then add reachability constraints and adapt differential synthesis from the previous chapter to generate concrete eBPF programs that expose reachable precision gaps.

### 5.4.1 Logical Specification for Precision Comparison

Let  $abs_g^{new}$  and  $abs_g^{old}$  be two sound implementations of the same abstract operator for instruction  $g$ . For a binary instruction, we write  $abs_g^{new}, abs_g^{old} : \mathbb{A} \times \mathbb{A} \rightarrow \mathbb{A}$ . Unary instructions and instructions with immediate operands are handled analogously.

To prove that the new operator is at least as precise as the old operator, we check the validity of the following query:

$$\begin{aligned}
 & \forall p, q \in \mathcal{C}, P, Q \in \mathbb{A} : \\
 & mem_{\mathbb{A}}(p, P) \wedge mem_{\mathbb{A}}(q, Q) \wedge R^{new} = abs_g^{new}(P, Q) \wedge R^{old} = abs_g^{old}(P, Q) \\
 & \implies R^{new} \sqsubseteq R^{old}.
 \end{aligned} \tag{5.1}$$

Here,  $P$  and  $Q$  are abstract inputs, while  $R^{new}$  and  $R^{old}$  are the abstract results produced by the two implementations. The predicate  $mem_{\mathbb{A}}(p, P)$ , defined in the previous chapter, states that concrete value  $p$  is represented by abstract value  $P$ . We reuse the same membership predicate here to keep the concrete and abstract executions connected.

The conclusion  $R^{new} \sqsubseteq R^{old}$  compares the two abstract results component-wise. For interval domains, this amounts to interval containment. Let  $X = (R^{new})_d$  and  $Y = (R^{old})_d$  be the interval components being compared, where  $X = [\underline{X}, \overline{X}]$  and  $Y = [\underline{Y}, \overline{Y}]$ . Then the new interval is at least as precise as the old interval when  $\underline{Y} \leq \underline{X}$  and  $\overline{X} \leq \overline{Y}$ . For the tnum domain, suppose  $(R^{new})_{tnum} = (av, am)$  and  $(R^{old})_{tnum} = (av', am')$ . We can ascertain  $(R^{new})_{tnum} \sqsubseteq_{tnum} (R^{old})_{tnum}$  by checking  $((av \& \sim am') = av') \wedge ((am \mid am') = am')$ .

We can also check whether the new operator is strictly more precise in a specific component. For example, to check whether the new operator improves the *s64* component for some input, we

ask whether the following formula is satisfiable:

$$\begin{aligned}
& \exists p, q \in \mathcal{C}, P, Q \in \mathbb{A} : \\
& mem_{\mathbb{A}}(p, P) \wedge mem_{\mathbb{A}}(q, Q) \wedge R^{\text{new}} = abs_g^{\text{new}}(P, Q) \wedge R^{\text{old}} = abs_g^{\text{old}}(P, Q) \\
& \wedge (R^{\text{new}})_{s64} \sqsubset_{s64} (R^{\text{old}})_{s64}.
\end{aligned} \tag{5.2}$$

The same query can be instantiated for any component in  $\mathcal{D} = \{u64, s64, tnum, u32, s32\}$ . This component-specific check is useful because an operator change may improve only one part of the verifier’s scalar abstract state. Together, Equations 5.1 and 5.2 establish the abstract precision relationship between two implementations. The first proves that the new operator never loses precision relative to the old one. The second identifies domains where the new operator is strictly more precise for some abstract inputs.

#### 5.4.2 Reachability in the Verifier Abstract State Space

The precision specifications above quantify over abstract inputs. This is the right level for proving an operator-level precision relation, but it may overapproximate the inputs that can arise in actual verifier executions. The verifier initializes registers in restricted abstract states, such as known constants or fully unknown values, and later states arise only through sequences of verifier abstract operations followed by reduction. As a result, many abstract states that are valid component-wise may never be reachable during the analysis of any eBPF program.

Table 5.2 illustrates this issue using only the 64-bit signed and unsigned interval domains. Some combinations of signed and unsigned bounds are valid when each component is viewed in isolation, but are ruled out by the verifier’s reduction logic. For example, a state may contain a tight unsigned interval while retaining a much wider signed interval. Such a state is not reachable if reduction would propagate the unsigned information into the signed domain and tighten the signed interval.

This distinction matters for precision comparison. If the abstract input that separates the new

| Case | Abstract value $X \in \mathbb{A}_{s64} \times \mathbb{A}_{u64}$ | Reachable? |
|------|-----------------------------------------------------------------|------------|
| R1   | $([-5, -1], [2^{64} - 5, 2^{64} - 1])$                          | ✓          |
| R2   | $([-100, 200], [0, 2^{64} - 1])$                                | ✓          |
| U1   | $([-10, -1], [0, 2^{64} - 1])$                                  | ×          |
| U2   | $([5, 2^{32}], [5, 10])$                                        | ×          |

Table 5.2: Reachable and unreachable verifier abstract states over the signed and unsigned 64-bit interval components. Abstract values are written as  $X = (X_{s64}, X_{u64}) \in \mathbb{A}_{s64} \times \mathbb{A}_{u64}$ . Cases R1 and R2 are reachable after verifier reduction. In R1, the signed and unsigned intervals are aligned and describe the same negative values under their respective interpretations. In R2, the signed interval crosses zero, so it cannot be soundly converted into a tighter unsigned interval. Cases U1 and U2 are ruled out by the verifier’s reduction logic: U1 would be reduced to  $([-10, -1], [2^{64} - 10, 2^{64} - 1])$ , and U2 would be reduced to  $([5, 10], [5, 10])$ .

and old operators is unreachable, then the precision gain is a formal fact about the abstract domain but not an observable improvement in the verifier. Since our goal is to demonstrate precision gains that can manifest in real verifier executions, we now synthesize eBPF programs whose analysis reaches inputs where the new and old operators differ.

### 5.4.3 Synthesizing Precision Witnesses

A precision witness is a concrete eBPF program whose verifier analysis reaches an abstract state where the new operator produces a strictly more precise result than the old operator. We use the same bounded synthesis setup as in Section 4.1. For a bound  $k$ , the enumerator searches over loop-free instruction sequences of the form

$$i_1; i_2; \dots; i_{k-1}; g,$$

where  $i_1, \dots, i_{k-1}$  form a prefix and  $g$  is the final operator under comparison. The prefix constructs reachable abstract operands for  $g$ , and the final instruction checks whether  $abs_g^{\text{new}}$  is strictly more precise than  $abs_g^{\text{old}}$ .

The notation follows Section 4.1.1. Concrete values are written with lowercase letters, their corresponding abstract values with uppercase letters, and  $mem_{\mathbb{A}}(x, X)$  states that concrete value

$x$  is represented by abstract value  $X$ . The set  $\mathcal{I} \subseteq \mathbb{A}$  contains the abstract scalar values that can appear in an initial verifier state.

The main difference from the previous chapter is the final constraint. In the soundness setting, the final instruction exposes a violation by requiring  $\neg mem_{\mathbb{A}}(r, R)$ . Here, the compared operators are assumed sound. The final instruction instead exposes a precision gap by requiring the new operator's result to be strictly more precise than the old operator's result in some component  $d \in \mathcal{D}$ .

**Single-instruction witnesses.** The simplest case is  $k = 1$ . There is no prefix, so the comparison is performed only on abstract operands drawn from the verifier's initial-state set  $\mathcal{I}$ . Suppose we want to expose a precision improvement in the  $s64$  component. Let  $p_1, q_1 \in \mathcal{C}$  be the concrete operands and let  $P_1, Q_1 \in \mathbb{A}$  be their abstract operands. The single-instruction query is:

$$\begin{aligned}
 & \exists p_1, q_1 \in \mathcal{C}, P_1, Q_1 \in \mathbb{A}. \\
 & \underbrace{P_1 \in \mathcal{I} \wedge Q_1 \in \mathcal{I}}_{\text{choose initial abstract operands}} \\
 & \wedge \underbrace{mem_{\mathbb{A}}(p_1, P_1) \wedge mem_{\mathbb{A}}(q_1, Q_1)}_{\text{choose concrete operands represented by them}} \\
 & \wedge \underbrace{R^{\text{new}} = abs_g^{\text{new}}(P_1, Q_1) \wedge R^{\text{old}} = abs_g^{\text{old}}(P_1, Q_1)}_{\text{relate inputs to both abstract operators}} \\
 & \wedge \underbrace{(R^{\text{new}})_{s64} \sqsubset_{s64} (R^{\text{old}})_{s64}}_{\text{new result is strictly more precise}}.
 \end{aligned}$$

If this formula is satisfiable, the model gives a one-instruction precision witness. If it is unsatisfiable, then no precision gap for this component is reachable directly from initial abstract values, and the search must use a longer prefix.

**Adding a prefix.** For  $k > 1$ , the prefix instructions construct operands for the final instruction. The operand-selection mechanism is the same as in Section 4.1.3. For instruction position  $j$ ,

$select_j(x, X)$  is defined relative to the result variables of the current instruction sequence:

$$select_j(x, X) \triangleq \underbrace{X \in \mathcal{I} \wedge mem_{\mathbb{A}}(x, X)}_{\text{initial abstract value}} \vee \underbrace{\bigvee_{\ell=1}^{j-1} (x = r_{\ell} \wedge X = R_{\ell})}_{\text{output of an earlier instruction}}.$$

The index  $\ell$  ranges over instruction positions earlier than  $j$ . Thus,  $select_j(x, X)$  either chooses an initial abstract value representing  $x$ , or reuses the concrete and abstract output produced by one earlier instruction in the same candidate sequence. When  $j = 1$ , the disjunction over earlier instruction outputs is empty and is interpreted as false, so  $select_1$  reduces to the initial-value case.

Consider the two-instruction skeleton

$$r_1 = \text{bpf\_and}(p_1, q_1); \quad r_2 = g(p_2, q_2).$$

Here,  $\text{bpf\_and}$  is a prefix instruction and  $g$  is the operator under comparison. The concrete operands and results are  $p_1, q_1, r_1, p_2, q_2, r_2 \in \mathcal{C}$ , and the corresponding abstract operands and prefix result are  $P_1, Q_1, R_1, P_2, Q_2 \in \mathbb{A}$ .

The prefix instruction starts from abstract values that can appear in an initial verifier state:

$$P_1 \in \mathcal{I} \wedge Q_1 \in \mathcal{I} \wedge mem_{\mathbb{A}}(p_1, P_1) \wedge mem_{\mathbb{A}}(q_1, Q_1).$$

The query then relates the prefix inputs and outputs using the concrete and abstract semantics:

$$r_1 = \overline{\text{bpf\_and}}(p_1, q_1) \wedge R_1 = \text{abs}_{\text{bpf\_and}}(P_1, Q_1).$$

Because the prefix result may be used as an operand for the final instruction, we also require the concrete prefix result to be represented by the abstract prefix result:

$$mem_{\mathbb{A}}(r_1, R_1).$$

The final operands are selected using the prefix output or fresh initial values:

$$\text{select}_2(p_2, P_2) \wedge \text{select}_2(q_2, Q_2).$$

The final instruction then compares the two abstract implementations:

$$R^{\text{new}} = \text{abs}_g^{\text{new}}(P_2, Q_2) \wedge R^{\text{old}} = \text{abs}_g^{\text{old}}(P_2, Q_2) \wedge (R^{\text{new}})_{s64} \sqsubset_{s64} (R^{\text{old}})_{s64}.$$

Putting the pieces together, the two-instruction precision-witness query is:

$$\begin{aligned} & \exists p_1, q_1, r_1, p_2, q_2 \in \mathcal{C}, \quad P_1, Q_1, R_1, P_2, Q_2 \in \mathbb{A}. \\ & P_1 \in \mathcal{I} \wedge Q_1 \in \mathcal{I} \wedge \text{mem}_{\mathbb{A}}(p_1, P_1) \wedge \text{mem}_{\mathbb{A}}(q_1, Q_1) \\ & \wedge r_1 = \overline{\text{bpf\_and}}(p_1, q_1) \wedge R_1 = \text{abs}_{\text{bpf\_and}}(P_1, Q_1) \wedge \text{mem}_{\mathbb{A}}(r_1, R_1) \\ & \wedge \text{select}_2(p_2, P_2) \wedge \text{select}_2(q_2, Q_2) \\ & \wedge R^{\text{new}} = \text{abs}_g^{\text{new}}(P_2, Q_2) \wedge R^{\text{old}} = \text{abs}_g^{\text{old}}(P_2, Q_2) \\ & \wedge (R^{\text{new}})_{s64} \sqsubset_{s64} (R^{\text{old}})_{s64}. \end{aligned}$$

If this formula is satisfiable, the model gives a two-instruction eBPF program whose verifier analysis reaches operands for  $g$  on which the new operator is strictly more precise than the old one.

**General synthesis query.** The general query follows Section 4.1.4. For a fixed bound  $k$ , the enumerator chooses an opcode sequence

$$i_1; i_2; \dots; i_{k-1}; g,$$

where  $i_1, \dots, i_{k-1}$  are prefix instructions and  $g$  is the final operator under comparison. The SMT query asks whether there exist concrete operands, abstract operands, and dataflow choices that instantiate this skeleton as a precision witness.

For instruction position  $j$ , let  $p_j, q_j \in \mathcal{C}$  be its concrete operands, let  $P_j, Q_j \in \mathbb{A}$  be the corre-

sponding abstract operands, and let  $r_j, R_j$  be its concrete and abstract results. The flow constraint is reused unchanged:

$$\Phi_{flow} \triangleq \bigwedge_{j=1}^k (select_j(p_j, P_j) \wedge select_j(q_j, Q_j)).$$

As above, the empty disjunction in  $select_1$  is interpreted as false, so the first instruction's operands must come from initial abstract values.

The prefix constraint relates the inputs and outputs of the first  $k - 1$  instructions:

$$\Phi_{prefix} \triangleq \bigwedge_{j=1}^{k-1} (r_j = \bar{i}_j(p_j, q_j) \wedge R_j = abs_{i_j}(P_j, Q_j) \wedge mem_{\mathbb{A}}(r_j, R_j)).$$

The membership constraint ensures that every prefix output used later by  $\Phi_{flow}$  remains paired with a concrete value it represents.

The final precision-gap constraint compares the two implementations of  $g$  on the operands selected for the final instruction:

$$\Phi_{gap} \triangleq (R^{new} = abs_g^{new}(P_k, Q_k) \wedge R^{old} = abs_g^{old}(P_k, Q_k) \wedge (R^{new})_d \sqsubset_d (R^{old})_d),$$

where  $d \in \mathcal{D}$  is the component in which we want to expose a strict precision improvement.

For this enumerated skeleton, the solver checks satisfiability of

$$\exists \vec{p}, \vec{q}, \vec{r} \in \mathcal{C}, \vec{P}, \vec{Q}, \vec{R} \in \mathbb{A}. \Phi_{flow} \wedge \Phi_{prefix} \wedge \Phi_{gap}.$$

A satisfying model gives a precision witness. It determines the initial abstract values, the concrete values represented by them, the operand choices, and the abstract values tracked by the verifier. If the query is unsatisfiable, then this particular opcode skeleton does not yield a witness. If every skeleton at the current bound is unsatisfiable, then no witness was found in the explored search space. As in the previous chapter, this is inconclusive: a witness may require a larger bound, or the

precision-improving input may be unreachable.

## 5.5 Evaluating Precision Comparison of Abstract Operators in the Verifier

We evaluate the precision-comparison framework along two dimensions. First, we ask whether the framework can formally compare two implementations of an abstract operator and determine whether a new implementation is at least as precise as the old one. Second, when a strict precision improvement exists, we ask whether differential synthesis can generate concrete eBPF programs that make the improvement observable in the verifier.

We evaluate these questions on recent Linux kernel patches that improve the precision of verifier abstract operators. We also use the framework to measure the precision impact of the verifier’s shared reduction operator, `BPF_SYNC`, which is applied after abstract operators execute. Overall, the evaluation addresses three questions:

1. **RQ1. Precision comparison.** Can the framework determine whether a proposed abstract operator is at least as precise as the implementation it replaces?
2. **RQ2. Witness generation.** When the framework identifies a precision gap, can differential synthesis generate a concrete eBPF program that exposes the gap during verifier analysis?
3. **RQ3. Impact of reduction.** Does the verifier’s shared reduction operator improve precision, and can the framework synthesize witnesses that demonstrate these improvements?

### 5.5.1 Experimental Setup

We conducted the experiments on a machine running Ubuntu 20.04 with an AMD Ryzen 5 3600 6-core CPU and 32GB of memory. The prototype is implemented in Python 3.12 and uses Agni’s C-to-logic translation infrastructure [110] to translate verifier abstract operators from C into logical formulas. We use the Z3 SMT solver [51] for both precision checking and witness synthesis. Each SMT query has a timeout of 10 minutes. For witness generation, we synthesize eBPF programs up to length 4.

| eBPF insn | Reference | $abs_g^{new} \preceq abs_g^{old}$ | $abs_g^{old} \preceq abs_g^{new}$ | Improved domains                                                         | Witness? |
|-----------|-----------|-----------------------------------|-----------------------------------|--------------------------------------------------------------------------|----------|
| BPF_AND   | [9]       | ✓                                 | ×                                 | $\mathbb{A}_{s64}, \mathbb{A}_{s32}$                                     | ✓        |
| BPF_SYNC  | [83]      | ✓                                 | ×                                 | <i>All</i>                                                               | ✓        |
| BPF_SYNC  | [7]       | ✓                                 | ×                                 | $\mathbb{A}_{tnum}$                                                      | ✓        |
| BPF_XOR   | [2]       | ✓                                 | ×                                 | <i>All</i>                                                               | ✓        |
| BPF_MUL   | [8]       | ✓                                 | ×                                 | $\mathbb{A}_{u64}, \mathbb{A}_{u32}, \mathbb{A}_{s64}, \mathbb{A}_{s32}$ | ✓        |

Table 5.3: Precision comparison results for verifier operator changes. For each precision-related change to an eBPF verifier abstract operator, the table reports the two-way precision preorder between the new and old implementations, the domains where the new operator is strictly more precise, and whether synthesis generated a concrete eBPF witness program. Here,  $abs_g^{new} \preceq abs_g^{old}$  means that the new implementation is at least as precise as the old implementation. The entry *All* refers to all verifier abstract domains in  $\mathcal{D}$ .

### 5.5.2 Evaluating Precision-Related Kernel Patches

We first evaluate recent kernel patches that were designed to improve the precision of verifier abstract operators. These patches span Linux versions v5.7 to v6.8 and modify arithmetic, logical, or reduction-related value-tracking logic. For each patch, we compare the patched operator against the previous implementation.

Table 5.3 summarizes the results. For each patch, the table reports whether the new operator is at least as precise as the old one, whether the old operator is at least as precise as the new one, which verifier domains improve, and whether the synthesizer generated a witness program. Across all evaluated patches, the framework confirms that the patched operator is at least as precise as the old implementation. In each case, the reverse precision relation does not hold, meaning that the patch is strictly more precise for some inputs.

The improved domains vary by patch. Some patches improve a single domain, such as  $\mathbb{A}_{s64}$ , while others improve several domains at once. To determine whether these improvements are observable in verifier executions, we invoke the synthesizer for each precision gap. The synthesizer generated concrete eBPF witness programs for all evaluated patches. These witnesses complement the hand-written examples that often accompany kernel patches, but provide a more systematic demonstration that the improvement can be reached from verifier initial states.

### 5.5.3 Generated Precision Witnesses

Here, we present two witness programs generated by our synthesizer for our improved multiplication abstract operator (§5.3) and for the bitwise *and* abstract operator `bpf_and` [9]. These witnesses are real eBPF programs that exhibit the precision merits of these operators. We illustrate our multiplication operator’s precision on signed values in Figure 5.3 using two domains,  $\mathbb{A}_{s64}$  and  $\mathbb{A}_{s32}$ . The program exemplifies how the operator handles multiplying two signed intervals  $[-10, 5] \times [-5, -5]$ . We observe that the old multiplication operator yields the widest range possible for these domains, while our operator yields the exact result expected for this multiplication,  $[-25, 50]$ . This improvement in precision could practically mean the verifier would accept a program in which the `r1` register might later be used to access memory. Importantly, in our patch to the Linux kernel, we included such an automatically generated witness embedded in its commit message. This patch has been upstreamed to kernel version v6.14 and beyond [8].

---

```

1  set r1          ; r1 is initially unknown
2  r1 = r1 & 15    ; r1([0, 15], [0, 15])
3  r1 = r1 - 10    ; r1([-10, 5], [-10, 5])
4  r1 = r1 * -5    ; new bpf_mul: r1([-25, 50], [-25, 50])
5                  ; old bpf_mul: r1([S64_min, S64_max],
6                  ;                  [S32_min, S32_max])
7  exit

```

---

Figure 5.3: Witness program demonstrating improved precision in `bpf_mul` for the signed interval components  $\mathbb{A}_{s64}$  and  $\mathbb{A}_{s32}$ .

Our second witness example, shown in Figure 5.4, illustrates precision improvement in the  $\mathbb{A}_{s64}$  domain for the `bpf_and` operator. The patch itself aimed at improving the handling of negative values when performing the bitwise *and* operation, since the old operator could result in verifier rejections, as detailed in [9]. Our synthesizer generates a program which performs bitwise *and* on two signed intervals where the old operator results in  $\top$ , and the proposed version returns a more precise interval  $[-2^{40}, 2^{40} - 1]$ .

---

```

1  set r1                ; r1 is initially unknown
2  set r2                ; r2 is initially unknown
3  r1 = r1 s>> 23        ; r1([-1099511627776, 1099511627775])
4  r2 = r2 s>> 32        ; r2([-2147483648, 2147483647])
5  r1 = r1 & r2          ; new bpf_and: r1([-1099511627776, 1099511627775])
6                        ; old bpf_and: r1([S64_min, S64_max])
7  exit

```

---

Figure 5.4: Witness program demonstrating a precision gap in `bpf_and` for the signed interval component  $\mathbb{A}_{s64}$ .

### 5.5.4 Precision Impact of the Reduction Operator

We also evaluate the precision contribution of the verifier’s shared reduction operator, `BPF_SYNC`. The reduction operator propagates information between the verifier’s domains after abstract operators execute. Although its soundness has been established in prior work [97], this experiment measures its effect on precision.

We compare each abstract operator with and without the reduction step using Linux kernel version v6.10. Table 5.4 summarizes the results. The results show that operators with reduction are always at least as precise as the corresponding operators without reduction. For many operators, reduction is also strictly more precise for some inputs. The improvements appear across a wide range of domains, often including all five verifier domains. This confirms that reduction is not only a soundness-preserving cleanup step, but also an important source of precision in the verifier.

The synthesizer generated witnesses for most domains where reduction produced a precision gain. In several cases, the witnesses required multiple instructions, showing again that precision improvements may depend on reaching nontrivial abstract states. For bitwise and shift operations, the synthesizer did not find witnesses for improvements in the `tnum` domain within the length and time bounds used in the experiment. This result is inconclusive. The missing witnesses may require longer programs, or the corresponding abstract inputs may be unreachable.

Overall, the evaluation shows that the framework can prove precision relations between competing abstract operators, synthesize concrete programs that demonstrate reachable precision im-

| eBPF<br>insn | $abs_g^R \preceq abs_g^{-R}$ | $abs_g^{-R} \preceq abs_g^R$ | Improved<br>domains                                                           | Synthesized<br>witness for                                                    | Witness<br>length |
|--------------|------------------------------|------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------|
| BPF_RSH      | ✓                            | ×                            | <i>All</i>                                                                    | $\mathbb{A}_{u64}, \mathbb{A}_{u32},$<br>$\mathbb{A}_{s64}, \mathbb{A}_{s32}$ | $\leq 4$          |
| BPF_ARSH     | ✓                            | ×                            | <i>All</i>                                                                    | $\mathbb{A}_{u64}, \mathbb{A}_{u32},$<br>$\mathbb{A}_{s64}, \mathbb{A}_{s32}$ | $\leq 4$          |
| BPF_LSH      | ✓                            | ×                            | <i>All</i>                                                                    | $\mathbb{A}_{u64}, \mathbb{A}_{u32},$<br>$\mathbb{A}_{s64}, \mathbb{A}_{s32}$ | $\leq 4$          |
| BPF_ADD      | ✓                            | ×                            | <i>All</i>                                                                    | <i>All</i>                                                                    | $\leq 3$          |
| BPF_SUB      | ✓                            | ×                            | <i>All</i>                                                                    | <i>All</i>                                                                    | $\leq 3$          |
| BPF_AND      | ✓                            | ×                            | <i>All</i>                                                                    | $\mathbb{A}_{u64}, \mathbb{A}_{s64}$                                          | $\leq 4$          |
| BPF_OR       | ✓                            | ×                            | $\mathbb{A}_{u64}, \mathbb{A}_{u32},$<br>$\mathbb{A}_{s64}, \mathbb{A}_{s32}$ | $\mathbb{A}_{u64}, \mathbb{A}_{s64}$                                          | $\leq 4$          |
| BPF_XOR      | ✓                            | ✓                            | <i>None</i>                                                                   | —                                                                             | —                 |

Table 5.4: Comparison of eBPF abstract operators with and without the shared reduction operator BPF\_SYNC on Linux v6.10. Here  $abs_g^R = op_g$  denotes the instruction-specific update without reduction, and  $abs_g^{-R} = sro \circ op_g$  denotes the full verifier operator with reduction. The table reports whether reduction is at least as precise as the unreduced operator, whether the reverse comparison also holds, the domains where reduction improves precision, and the domains for which synthesis generated eBPF witness programs. The entry *All* refers to all verifier abstract domains in  $\mathcal{D}$ .

provements, and quantify the precision benefit of the verifier’s shared reduction operator.

## 5.6 Summary

In this chapter, we propose a verifier-specific framework for comparing the precision of abstract operators. The verifier’s scalar domains describe the same machine value under different interpretations, so standard product-domain comparisons do not directly match the verifier’s representation. Our key idea is to compare abstract results component-wise across the verifier’s scalar domains and to use this comparison as a preorder between competing operator implementations. We use this preorder to compare operators directly, and also to study the precision contribution of the verifier’s shared reduction operator, BPF\_SYNC.

We also present an improved abstract operator for multiplication. The existing verifier operator widens to  $\top$  in several cases involving negative values or large unsigned operands. Our operator

uses interval multiplication to compute tighter signed and unsigned bounds when overflow does not occur. This case study motivates the need to go beyond abstract precision comparisons and ask whether the inputs that expose a precision gap can arise in real verifier executions.

To address this reachability question, we adapt differential synthesis from the soundness setting to the precision setting. Instead of searching for a concrete result that is missing from an abstract result, our synthesis procedure searches for a concrete eBPF program whose verifier analysis reaches operands on which one sound operator is strictly more precise than another. Such a program serves as a precision witness, demonstrating that the precision improvement is observable in an actual verifier execution.

Our evaluation applies this framework to verifier operator changes and to `BPF_SYNC`. The results show that the framework can prove precision relationships, identify the domains where precision improves, and synthesize concrete eBPF witnesses for reachable precision gaps.

## CHAPTER 6

### RELATED WORK

This dissertation builds on three foundations. The first is verification of systems software and program analyzers, the second is abstract interpretation, and the third is program synthesis. Verification provides the broader goal of proving safety properties for the Linux eBPF verifier, a security-critical program analyzer in the kernel. Abstract interpretation provides the semantic basis for the verifier’s value-tracking analysis and its scalar abstract domains. Program synthesis provides a way to generate executable eBPF witnesses for logical counterexamples and reachable precision gaps.

#### 6.1 eBPF Safety and Verification

Software verification provides the broader context for reasoning about the Linux eBPF verifier. Seminal work introduced deductive methods for proving program correctness [56, 62], temporal logic methods for specifying and reasoning about program behavior [78, 90], and model checking as an automated approach to exploring program state spaces [41–43]. Later techniques, including counterexample-guided abstraction refinement [40] and IC3/PDR [35], improved the scalability of automated safety proving.

The eBPF verifier is a security-critical program analyzer in the Linux kernel. It checks untrusted eBPF programs before they execute in kernel context, and its analysis determines whether those programs are safe to load. Bugs in this analysis can therefore become kernel safety or security bugs. Prior work has addressed this risk in two complementary ways. Formal-methods approaches aim to prove correctness properties of eBPF programs, verifier analyses, or related compiler infrastructure, while testing and fuzzing approaches generate eBPF programs that exercise the verifier and expose incorrect behavior.

### 6.1.1 Formal Methods for eBPF Safety

Prior work has studied eBPF safety at several layers of the system. PREVAIL verifies eBPF programs outside the Linux kernel using abstract interpretation over the zone domain [57]. Its relational abstraction improves precision and supports programs with loops, but PREVAIL is an independent verifier rather than the production in-kernel verifier. Our work instead studies the abstract operators used by the eBPF verifier that runs in Linux itself.

Other verification work targets eBPF compilation and execution. Jitk provides a trustworthy in-kernel interpreter infrastructure for system call policies [112]. Serval supports automated verification of systems code using symbolic interpreters [84], and Jitterbug verifies eBPF JIT compilers using stepwise specifications for per-instruction translations [85]. These approaches reason about compiling or executing eBPF programs after they have been accepted. In contrast, we focus on the earlier static-analysis step performed by the verifier.

VeRA verifies the range analysis used in JavaScript JIT compilers [36]. Although VeRA targets a different runtime and language, its soundness condition is closely related. A range operator is sound when the concrete result remains contained in the abstract range computed by the analysis. Our work uses a similar containment-based view for abstract operators, but adapts it to the multi-domain abstract state used by the eBPF verifier and to the concrete semantics of eBPF instructions.

A closer line of work verifies parts of the verifier’s value-tracking analysis itself. Vishwanathan et al. [108] study the verifier’s tnum domain, which tracks bit-level uncertainty in register values. Their work formalizes the tnum abstraction, proves soundness and optimality properties for several tnum arithmetic operators, and develops a sound and more efficient multiplication operator that was merged into the Linux kernel. This work is directly relevant to the bit-level component of the verifier’s value tracking, but it reasons about manually specified tnum operators rather than the kernel C implementation, and it focuses on a single abstract domain.

Bhat and Shacham formally verify parts of the Linux verifier’s range analysis [29]. Their work targets verifier C code and checks generated logical formulas against soundness specifications for value tracking. It studies the same abstract domains targeted by our work, including signed and

unsigned bounds, 32-bit and 64-bit views, and `tnum`. However, it focuses on a limited set of ALU operators, does not address branch operators, and does not provide executable witnesses of unsoundness.

Agni [110] builds on these efforts by providing a unified implementation-level verification framework for the eBPF verifier’s value-tracking analysis. Rather than manually proving isolated abstract-domain operators, Agni extracts verification conditions from the Linux kernel’s C implementation and checks them against soundness specifications. This allows it to reason across `tnum`, signed and unsigned interval domains, branch operators, arithmetic operators, and shared reduction logic. It also formalizes the non-modular nature of abstract operators in the verifier. Agni shows that implementation-level verification of the production eBPF verifier is feasible, but it also exposes limitations that motivate our work. First, verification conditions that combine operator-specific logic with shared reduction logic can become difficult for automated solvers as the verifier evolves. Second, a failed soundness check may produce only a logical counterexample, not an executable eBPF program that demonstrates the behavior in the verifier. Third, soundness does not capture precision. A sound abstract operator may still lose information and cause the verifier to reject safe programs. We address these limitations by making verification more modular, synthesizing executable witnesses for logical counterexamples, and comparing the precision of competing verifier operators across interacting scalar domains.

### 6.1.2 Testing and Fuzzing the eBPF Verifier

Recent work has tested the production Linux verifier by generating eBPF programs that stress its implementation and expose incorrect or inconsistent verifier behavior [66, 73, 88]. One approach generates structured programs that are likely to be accepted by the verifier, then uses sanitization or runtime instrumentation to check whether accepted programs can still trigger unsafe behavior [105]. Another approach compares verifier abstract states against concrete states observed during execution. It runs an eBPF program to collect concrete states, then constructs a modified program with embedded checks that require the verifier to prove those states are included in its ab-

stract approximations. A failure to prove the embedded checks exposes a verifier logic bug [104].

Fuzzing has also been applied to the broader eBPF subsystem. Syzkaller is a general-purpose kernel fuzzer [59], while BRF targets the eBPF runtime by generating programs that satisfy verifier constraints and exercise runtime components more effectively [66]. These systems focus on bug finding through testing. In contrast, our work focuses on formal verification of the eBPF verifier’s abstract domains and abstract operators.

### 6.1.3 eBPF Safety Beyond the Verifier

Some systems improve eBPF safety by changing the programming model or adding run-time enforcement. SafeBPF isolates eBPF programs from the rest of the kernel at run time [75]. Rex addresses the gap between high-level language safety and verifier constraints using safe Rust interfaces and lightweight runtime support [67]. BeePL proposes a correct-by-compilation approach for safe kernel extensions [91]. These systems provide safety by construction, isolation, or defense in depth. Our work instead keeps the eBPF verifier as the object of study and reasons about the soundness and precision of its existing abstract operators.

## 6.2 Abstract Interpretation and Domain Refinement

Abstract interpretation provides the main semantic foundation for our work [46, 48, 49]. It proves safety by computing abstract states that overapproximate concrete executions. The eBPF verifier follows this methodology by tracking abstract register states and rejecting programs when those states are insufficient to prove safety.

A central challenge in abstract interpretation is choosing abstract domains that are precise enough to prove useful properties while remaining efficient. Intervals are a classical numerical abstraction [48], and richer domains such as polyhedra and octagons improve precision at higher cost [79, 100]. The Linux eBPF verifier instead uses lightweight non-relational components, including signed and unsigned intervals at 64-bit and 32-bit widths, together with `tnum` for bit-level information [108]. This design reflects the verifier’s kernel setting, where analysis must be pre-

dictable and efficient.

The verifier’s register state is related to product domains, reduced products, and partially reduced products [44, 49, 81]. Product domains combine several abstractions, and reduced products improve precision by propagating information between them. Partially reduced products apply only selected reductions, trading off precision against the cost of maintaining all possible cross-domain relationships. The verifier’s shared reduction logic follows this pragmatic pattern by propagating information between signed and unsigned views, 32-bit and 64-bit views, and interval and tnum components.

The analogy to reduced products is useful, but it is not exact. The verifier does not compute a general fixed point of all cross-domain refinements. Its reduction logic is handwritten, bounded, and verifier-specific. We therefore use these product-domain constructions as conceptual foundations while reasoning about the actual reduction behavior implemented in the verifier. Improving cross-domain interaction remains an active area of interest in abstract interpretation, including work on disjunctive completion, domain cooperation, and reduced-product reasoning [28, 38, 45, 107].

Our work also uses the lattice-theoretic ordering of abstract domains to reason about precision. In many product domains, precision can be described through a single concretization of the product state. The verifier’s components do not fit this view cleanly because they describe the same fixed-width machine value under different interpretations. A signed interval, an unsigned interval, a 32-bit bound, and a tnum value are not independent facts about separate values. We therefore compare precision component-wise, using the partial order of each abstract domain. This comparison is close in spirit to standard abstract-domain precision orderings, but is specialized to the verifier’s non-standard combination of interval and bitwise abstractions.

### **6.3 Program Synthesis and Witness Generation**

Program synthesis constructs programs from specifications. Seminal work on counterexample-guided inductive synthesis and sketching reduces the search for programs to solver-aided search

over a restricted program space [27, 101]. Brahma similarly synthesizes loop-free programs by composing components from a library and searching for a valid dataflow between them [61]. In these settings, the goal is usually to synthesize an implementation that satisfies a specification for all inputs.

Synthesis has also been used to construct abstract operators. Manually crafting sound and precise abstract operators is non-trivial and error-prone, and prior work has developed automated techniques for deriving such operators [54, 92, 93, 106]. Amurth synthesizes abstract operators for non-relational domains using a user-provided DSL [68]. It explores the search space using a dual CEGIS [102] loop with positive counterexamples for soundness and negative counterexamples for precision. This aims to synthesize operators that are sound and as precise as possible within the chosen language. This line of work has been extended to reduced-product domains [69] and to abstract operators in the context of the eBPF verifier [111].

We use synthesis for a different but complementary purpose. We do not synthesize a verifier implementation or production eBPF code. Instead, we use synthesis to generate witnesses. A witness is a small eBPF program that demonstrates that a logical soundness issue or precision gap can arise in an actual verifier execution. Our precision-comparison framework can also complement operator-synthesis techniques by checking whether a synthesized operator is no less precise than an existing verifier operator, and by generating reachable eBPF witnesses when the synthesized operator is strictly more precise.

This use of synthesis is motivated by reachability. A logical model over abstract values may describe a state that the verifier can never reach from a real eBPF program. Such a model is useful for debugging a proof obligation, but it is less useful as evidence of real verifier behavior. Differential synthesis bridges this gap by searching for bounded instruction sequences whose concrete and abstract executions realize the logical condition.

This view is related to counterexample generation in model checking [30, 35, 39] and to solver-aided test generation [37, 58]. The difference is that the witness here must satisfy both the concrete eBPF semantics and the verifier’s abstract semantics. The generated program is therefore not

merely a test input, but evidence that a soundness failure or precision improvement is reachable under the verifier's own analysis.

## CHAPTER 7

### CONCLUSION

The goal of this dissertation is to make formal methods practical for the real-world development of the Linux eBPF verifier. As part of the kernel’s trusted computing base, the verifier depends on static analysis that is both sound to reject unsafe programs and precise enough to accept safe ones. We focus on its value-tracking abstract operators, which are handwritten, optimized for production use, and closely coupled with shared logic that refines information across domains.

We develop techniques that reason directly about the verifier implementation as it evolves. For soundness, we introduce scalable implementation-level verification and executable witness generation. For precision, we compare abstract operators and synthesize programs that make precision differences observable in real verifier executions. Together, these techniques show how formal methods can support practical development of production static analyzers. The patches and techniques developed in this dissertation have attracted interest from Linux kernel developers, and several of the ideas proposed here are already being used to improve and validate verifier changes.

#### 7.1 Dissertation Summary

We begin by addressing the scalability of implementation-level verification. Chapter 3 focuses on the eBPF verifier’s value-tracking abstract operators, which track scalar values across signed and unsigned interval domains, 32-bit and 64-bit bounds, and the `tnum` bitwise domain, while also interacting with shared cross-domain reduction logic. Prior work showed how to verify these operators directly from the kernel C implementation, but its monolithic strategy checked each full operator together with shared reduction and repeatedly verified the same reduction reasoning. As the verifier evolved, this approach became increasingly expensive and began to time out on newer kernel versions. We observe that each full operator can be decomposed into an instruction-specific operator  $op_g$  and a shared reduction operator  $sro$ , with  $abs_g = sro \circ op_g$ . We therefore verify each

$op_g$  independently, verify  $sro$  once, and compose the results to prove soundness of the full operator. This decomposition significantly reduces solver cost and also exposes latent unsoundness hidden by monolithic verification. In particular, it reveals that several bitwise operators could produce ill-formed signed interval components before shared reduction repaired the state. We identify the root cause, patch the affected operators, and show that the patched versions satisfy the modular soundness specification.

Once verification failures are reported, the next challenge is to make them actionable. Chapter 4 introduces differential synthesis for this purpose. A failed verification condition produces an SMT model describing a mismatch between concrete execution and abstract execution, but the abstract operands in that model may not be reachable during the analysis of any real eBPF program. We bridge this gap by searching for a bounded loop-free eBPF program that drives the verifier to abstract operands exposing the failure and then executes a final instruction whose concrete result is not represented by the verifier’s abstract output. The synthesis query combines opcode enumeration with SMT solving. The enumerator chooses an instruction skeleton, while the solver chooses concrete values, abstract values, operand sources, and dataflow. Prefix instructions must preserve membership between concrete and abstract execution, ensuring that the final operands are reachable from valid verifier initial states. When synthesis succeeds, the result is an executable eBPF witness that developers can run, inspect, and use for debugging or regression testing.

We then broaden the framework from soundness to precision. Chapter 5 starts from the observation that soundness is necessary for verifier safety, but not sufficient for verifier usability. Even when an abstract operator safely overapproximates concrete execution, it may lose enough information to cause the verifier to reject safe programs. This issue is especially important because verifier operators are handwritten and evolve over time as kernel developers improve the verifier’s reasoning about arithmetic, bitwise operations, and cross-domain refinement. When an operator is updated, developers need a way to check not only that the new implementation remains sound, but also that it does not lose precision relative to the implementation it replaces. To reason about precision in the verifier, we define a verifier-specific component-wise preorder for comparing ab-

tract results across the verifier’s scalar domains. Using this preorder, we compare competing implementations of verifier operators and adapt differential synthesis to generate precision witnesses. These witnesses are concrete eBPF programs whose verifier analysis reaches operands on which one sound operator is strictly more precise than another. This connects formal precision comparisons over abstract states to observable differences in real verifier executions. We use this framework to evaluate precision-improving kernel patches and to measure the precision improvements provided by the shared reduction operator. This framework also led us to develop a more precise multiplication operator, which has been upstreamed to the Linux kernel.

## 7.2 Future Directions

This dissertation shows that formal methods can be practical for reasoning about the soundness and precision of a production static analyzer such as the eBPF verifier. The approaches developed here can be refined further and adapted to other safety-critical systems that rely on static analysis. This section discusses several avenues for future work.

***Reachability-aware modular verification.*** Differential synthesis suggests a broader direction that treats soundness checking, reachability analysis, and modular verification as parts of a single framework. The current approach begins with abstract states known to be valid initially and searches for bounded instruction sequences that construct operands exposing a soundness violation. For a fixed bound  $k$ , the synthesis formula characterizes the states reachable within  $k$  instructions, but it also records the intermediate states, instructions, and dataflow choices along each path. Its size therefore grows with the bound and does not provide a concise description of the reachable state space.

Future work could seek a more general characterization through inductive predicates that summarize properties shared by all reachable verifier states. Candidate predicates could characterize the shape of these states through relationships between intervals and `tnum`, 32-bit and 64-bit components, and signed and unsigned bounds. Even a sound overapproximation could exclude many arbitrary abstract inputs considered by verification while remaining considerably smaller than the

full formula for  $k$  instructions. Such a characterization would go beyond the constraints imposed by Agni’s refined soundness specification by capturing relationships specific to reachable states.

These relationships could be inferred using IC3/PDR or related unbounded model-checking techniques. Prior work on applying property-directed reachability to abstract interpretation provides a foundation for this direction [96]. The main challenge would be to lift the joint concrete and abstract semantics used by differential synthesis into a transition-system model. SMT-based PDR may be a promising fit because Agni already represents verifier operators as fixed-width bit-vector formulas, although scalability to the complete verifier state remains an open question. Domain-specific predicates could guide invariant generalization, allowing the model to find deeper witnesses or prove violations unreachable.

This approach could also clarify why differential synthesis did not generate witnesses for every reported violation. In our evaluation, three violations in Linux v4.14 and eight in v5.5 remained without witnesses within the explored bounds. Their absence may have several explanations; the shortest witness may exceed the bound, the witness may require instructions or verifier features omitted from the encoding, the solver may be unable to complete the search, or the violating abstract state may be genuinely unreachable. Future experiments should distinguish unsatisfiable queries from timeouts, increase the bound systematically, and analyze whether the required operand states can be reached independently of the final failing instruction.

The verifier itself could also be structured to support more modular proofs. The present decomposition

$$abs_g = sro \circ op_g$$

separates an instruction-specific update from shared reduction, but each component could be decomposed further. Within  $op_g$ , separate modules could handle width normalization, operator-specific reasoning, individual abstract-domain updates, 32-bit-to-64-bit propagation, and conservative fallback behavior. Their interactions could explicitly record dependencies, such as an interval update that uses an already-updated  $t_{num}$ . Shared reduction could likewise be represented as a sequence of smaller reductions, each responsible for propagating information between particular

domains. Each module could then be verified against explicit input invariants and concrete-value preservation guarantees.

***From verification failures to automated kernel patches.*** A natural next step is to connect verification, witness generation, and program repair into a single automated workflow. In this dissertation, a failed soundness check can expose a bug in an abstract operator, and differential synthesis can often turn the failure into an executable eBPF witness. The final step still requires a developer to inspect the failure, understand the root cause, and patch the kernel implementation. Future work could automate this last step by using the failed verification condition and the synthesized witness as constraints for implementation repair.

Such a system would start from a soundness violation, generate an executable witness when the violation is reachable, and then search for a patch to the kernel C code that eliminates the violation while preserving the intended operator behavior. The repaired implementation could be checked again against the same soundness specification, and the witness could be reused as a regression test. This would turn the current verification and diagnosis pipeline into a repair loop, moving closer to a fully automated workflow for finding, explaining, and fixing verifier bugs in production kernel code.

***Differential synthesis as a test-case generator.*** Differential synthesis can also be used as a test-case generator for verifier development. One challenge in kernel development is constructing test cases that demonstrate the effect of a patch. This is especially difficult for patches to abstract operators, where an improvement may affect only a narrow region of the input space. For example, a patch may improve reasoning about the lower 32 bits of a scalar value, but manually constructing an eBPF program that exposes that improvement can be nontrivial.

Chapter 5 briefly discusses this use case in the context of patching the multiplication operator. Future work could extend this idea into a general workflow for generating verifier test cases from operator differences. Given two versions of an abstract operator, differential synthesis could search for an eBPF program whose verifier states expose a soundness or precision gap between them. The

resulting program would serve as a regression test for a bug fix or as evidence that a precision patch accepts programs that the previous implementation rejected.

Using differential synthesis in this way would reduce the manual effort needed to justify verifier patches. Rather than asking developers to hand-design programs that reach a specific corner of the abstract state space, the synthesis query would encode the desired difference and construct a program that realizes it. In practice, this would require adapting the specification to describe the values or state regions affected by a patch. The synthesis procedure would then target those values of interest, rather than searching for any difference between two operators. This would make differential synthesis useful not only for debugging verifier bugs, but also for producing focused test cases that document and validate incremental improvements to verifier precision.

## BIBLIOGRAPHY

- [1] Agni's verification of kernel 6.4 takes weeks. <https://github.com/bpfverif/agni/issues/13>
- [2] bpf: fix a verifier failure with xor. <https://lore.kernel.org/bpf/20200825064608.2017937-1-yhs@fb.com/>
- [3] bpf: fix incorrect sign extension in check\_alu\_op(). <https://github.com/torvalds/linux/commit/95a762e2c8c942780948091f8f2a4f32fcea1ac6f>
- [4] bpf: Fix reg\_bound\_offset 64->32 var\_off subreg propagation. <https://lore.kernel.org/linux-patches/20230508094431.898575322@linuxfoundation.org/>
- [5] bpf, Harden and/or xor value tracking in verifier. <https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf-next.git/commit/?id=1f586614f3ff>
- [6] bpf selftests. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/tools/testing/selftests/bpf/>
- [7] bpf: Verifier, adjust\_scalar\_min\_max\_vals to always call update\_reg\_bounds(). <https://lkml.iu.edu/hypermail/linux/kernel/2208.1/01778.html>
- [8] bpf, verifier: Improve precision of BPF\_MUL. <https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf-next.git/commit/?id=9aa0ebde0014>
- [9] bpf, verifier: improve signed ranges reasoning for BPF\_AND. <https://lore.kernel.org/bpf/20240719110059.797546-6-xukuohai@huaweicloud.com/>
- [10] bpf, x32: Fix bug with ALU64 LSH, RSH, ARSH BPF\_X shift by 0. <https://github.com/torvalds/linux/commit/68a8357ec15bdce55266e9fba8b8b3b8143fa7d2>
- [11] Cilium API-aware networking and security. <https://cilium.io/>
- [12] CVE-2017-16996 Mishandling of register truncation. <https://nvd.nist.gov/vuln/detail/CVE-2017-16996>
- [13] CVE-2017-17852 Mishandling of 32-bit ALU ops. <https://nvd.nist.gov/vuln/detail/CVE-2017-17852>
- [14] CVE-2017-17853 Mishandling of 32-bit ALU ops. <https://nvd.nist.gov/vuln/detail/CVE-2017-17853>
- [15] CVE-2017-17864 Mishandled comparison between pointer and unknown data types. <https://nvd.nist.gov/vuln/detail/CVE-2017-17864>
- [16] CVE-2018-18445 Mishandling of 32-bit RSH op. <https://nvd.nist.gov/vuln/detail/CVE-2018-18445>

- [17] CVE-2020-8835 Mishandling of bounds tracking for 32-bit JMPs. <https://nvd.nist.gov/vuln/detail/CVE-2020-8835>
- [18] CVE-2021-3490 The eBPF ALU32 bounds tracking for bitwise ops (AND, OR and XOR) in the Linux kernel did not properly update 32-bit bounds. CVE-2021-3490
- [19] Facebook’s Katran load balancer: Kernel XDP program. [https://github.com/facebookincubator/katran/blob/master/katran/lib/bpf/balancer\\_kern.c](https://github.com/facebookincubator/katran/blob/master/katran/lib/bpf/balancer_kern.c)
- [20] Linux BPF verifier. <https://github.com/torvalds/linux/blob/master/kernel/bpf/verifier.c>
- [21] Merge branch ‘bpf-register-bounds-logic-and-testing-improvements’. <https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf-next.git/commit/?id=cd9c127069c040d6b022f1ff32fed4b52b9a4017>
- [22] Netconf 2018 day 1. <https://lwn.net/Articles/757201/>
- [23] Suricata: ebpf and xdp. <https://suricata.readthedocs.io/en/latest/capture-hardware/ebpf-xdp.html>
- [24] workflows: CI for the verification step . <https://github.com/bpfverif/agni/commit/18b7513facda0d6f57e69c293f9a494ede683be7>
- [25] workflows: Initial CI for the LLVM-to-SMT step . <https://github.com/bpfverif/agni/commit/a30260d2b8c4c14f356f9501c4a9dac28f768f5d>
- [26] New GKE Dataplane V2 increases security and visibility for containers. <https://cloud.google.com/blog/products/containers-kubernetes/bringing-ebpf-and-cilium-to-google-kubernetes-engine> (2021)
- [27] Alur, R., Singh, R., Fisman, D., Solar-Lezama, A.: Search-based program synthesis. Commun. ACM **61**(12), 84–93 (nov 2018). doi:10.1145/3208071
- [28] Amadini, R., Jordan, A., Gange, G., Gauthier, F., Schachte, P., Søndergaard, H., Stuckey, P.J., Zhang, C.: Combining string abstract domains for javascript analysis: An evaluation. In: Legay, A., Margaria, T. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 41–57. Springer Berlin Heidelberg, Berlin, Heidelberg (2017)
- [29] Bhat, S., Shacham, H.: Formal verification of the linux kernel ebpf verifier range analysis. <https://sanjit-bhat.github.io/assets/pdf/ebpf-verifier-range-analysis22.pdf> (2022)
- [30] Biere, A., Cimatti, A., Clarke, E., Zhu, Y.: Symbolic model checking without bdds. In: Cleaveland, W.R. (ed.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 193–207. Springer Berlin Heidelberg, Berlin, Heidelberg (1999)
- [31] Borkmann, D.: bpf: Fix scalar32\_min\_max\_or bounds tracking. <https://github.com/torvalds/linux/commit/5b9fbef75b6a98955f628e205ac26689bcb1383e> (2020)

- [32] Borkmann, D.: bpf: Undo incorrect `__reg_bound_offset32` handling. <https://git.kernel.org/pub/scm/linux/kernel/git/netdev/net-next.git/commit/?id=f2d67fec0b43edce8c416101cdc52e71145b5fef> (2020)
- [33] Borkmann, D.: bpf: Fix `alu32` const subreg bound tracking on bitwise operations. <https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf.git/commit/?id=049c4e13714ecbca567b4d5f6d563f05d431c80e> (2021)
- [34] Borkmann, D.: bpf: Fix `signed_sub,add32_overflows` type handling. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=bc895e8b2a64e502fbba72748d59618272052a8b> (2021)
- [35] Bradley, A.R.: SAT-Based Model Checking without Unrolling. In: Proceedings of the 12th International Conference on Verification, Model Checking, and Abstract Interpretation. p. 70–87. VMCAI’11, Springer-Verlag, Berlin, Heidelberg (2011)
- [36] Brown, F., Renner, J., Nötzli, A., Lerner, S., Shacham, H., Stefan, D.: Towards a verified range analysis for javascript jits. In: Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation. pp. 135–150 (2020)
- [37] Cadar, C., Dunbar, D., Engler, D.: KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In: Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation. pp. 209–224. OSDI 2008, USENIX Association (2008)
- [38] Cheng, X., Wang, J., Sui, Y.: Precise sparse abstract execution via cross-domain interaction. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE ’24, Association for Computing Machinery, New York, NY, USA (2024). doi:10.1145/3597503.3639220, <https://doi.org/10.1145/3597503.3639220>
- [39] Clarke, E., Biere, A., Raimi, R., Zhu, Y.: Bounded model checking using satisfiability solving. *Formal methods in system design* **19**(1), 7–34 (2001)
- [40] Clarke, E., Grumberg, O., Jha, S., Lu, Y., Veith, H.: Counterexample-guided abstraction refinement. In: Emerson, E.A., Sistla, A.P. (eds.) *Computer Aided Verification*. pp. 154–169. Springer Berlin Heidelberg, Berlin, Heidelberg (2000)
- [41] Clarke, E.M.: Model checking. In: Ramesh, S., Sivakumar, G. (eds.) *Foundations of Software Technology and Theoretical Computer Science*. pp. 54–56. Springer Berlin Heidelberg, Berlin, Heidelberg (1997)
- [42] Clarke, E.M., Emerson, E.A.: Design and synthesis of synchronization skeletons using branching time temporal logic. In: Kozen, D. (ed.) *Logics of Programs*. pp. 52–71. Springer Berlin Heidelberg, Berlin, Heidelberg (1982)
- [43] Clarke, E.M., Henzinger, T.A., Veith, H., Bloem, R., et al.: *Handbook of model checking*, vol. 10. Springer (2018)

- [44] Cortesi, A., Costantini, G., Ferrara, P.: A Survey on Product Operators in Abstract Interpretation. *Electronic Proceedings in Theoretical Computer Science* **129**, 325–336 (sep 2013). doi:10.4204/eptcs.129.19
- [45] Cousot, P., Cousot, R.: Higher-order abstract interpretation (and application to comportment analysis generalizing strictness, termination, projection and per analysis of functional languages). In: *Proceedings of 1994 IEEE International Conference on Computer Languages (ICCL'94)*. pp. 95–112 (1994). doi:10.1109/ICCL.1994.288389
- [46] Cousot, P.: Lecture 13 notes: Mit 16.399, abstract interpretation. [http://web.mit.edu/afs/at\\_hena.mit.edu/course/16/16.399/www/lecture\\_13-abstraction1/Cousot\\_MIT\\_2005\\_Course\\_13\\_4-1.pdf](http://web.mit.edu/afs/at_hena.mit.edu/course/16/16.399/www/lecture_13-abstraction1/Cousot_MIT_2005_Course_13_4-1.pdf) (2005)
- [47] Cousot, P., Cousot, R.: Static determination of dynamic properties of programs. In: *Proceedings of the 2nd International Symposium on Programming, Paris, France*. pp. 106–130. Dunod (1976)
- [48] Cousot, P., Cousot, R.: Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*. p. 238–252. POPL '77, Association for Computing Machinery, New York, NY, USA (1977). doi:10.1145/512950.512973
- [49] Cousot, P., Cousot, R.: Systematic design of program analysis frameworks. In: *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*. p. 269–282. POPL '79, Association for Computing Machinery, New York, NY, USA (1979). doi:10.1145/567752.567778
- [50] Cree, E.: bpf/verifier: fix bounds calculation on BPF\_RSH. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=4374f256ce8182019353c0c639bb8d0695b4c941> (2017)
- [51] De Moura, L., Bjørner, N.: Z3: An efficient smt solver. In: *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. p. 337–340. TACAS'08/ETAPS'08, Springer-Verlag, Berlin, Heidelberg (2008). doi:10.1007/978-3-540-78800-3\_24
- [52] Duplyakin, D., Ricci, R., Maricq, A., Wong, G., Duerig, J., Eide, E., Stoller, L., Hibler, M., Johnson, D., Webb, K., Akella, A., Wang, K., Ricart, G., Landweber, L., Elliott, C., Zink, M., Cecchet, E., Kar, S., Mishra, P.: The design and operation of cloudlab. In: *Proceedings of the 2019 USENIX Conference on Usenix Annual Technical Conference*. p. 1–14. USENIX ATC '19, USENIX Association, USA (2019)
- [53] eBPF Community: ebpf documentation (2025), <https://ebpf.io/what-is-ebpf/>, accessed: 2025-02-24
- [54] Elder, M., Lim, J., Sharma, T., Andersen, T., Reps, T.: Abstract domains of affine relations. *ACM Trans. Program. Lang. Syst.* **36**(4) (Oct 2014). doi:10.1145/2651361, <https://doi.org/10.1145/2651361>

- [55] Fabre, A.: L4drop: Xdp ddos mitigations. <https://blog.cloudflare.com/l4drop-xdp-ebpf-based-ddos-mitigations/>
- [56] Floyd, R.W.: Assigning Meanings to Programs, pp. 65–81. Springer Netherlands, Dordrecht (1993). doi:10.1007/978-94-011-1793-7\_4, [https://doi.org/10.1007/978-94-011-1793-7\\_4](https://doi.org/10.1007/978-94-011-1793-7_4)
- [57] Gershuni, E., Amit, N., Gurfinkel, A., Narodytska, N., Navas, J.A., Rinetzky, N., Ryzhyk, L., Sagiv, M.: Simple and precise static analysis of untrusted linux kernel extensions. In: Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation. p. 1069–1084. PLDI 2019, Association for Computing Machinery, New York, NY, USA (2019). doi:10.1145/3314221.3314590
- [58] Godefroid, P., Klarlund, N., Sen, K.: DART: Directed automated random testing. In: Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation. pp. 213–223. PLDI 2005, ACM (2005). doi:10.1145/1065010.1065036
- [59] Google: Syzkaller: Kernel fuzzer. Software (2024), <https://github.com/google/syzkaller>
- [60] Gregg, B.: Bpf performance analysis at netflix. <https://www.slideshare.net/brendangregg/reinvent-2019-bpf-performance-analysis-at-netflix>
- [61] Gulwani, S., Jha, S., Tiwari, A., Venkatesan, R.: Synthesis of loop-free programs. SIGPLAN Not. **46**(6), 62–73 (jun 2011). doi:10.1145/1993316.1993506
- [62] Hoare, C.A.R.: An axiomatic basis for computer programming. Commun. ACM **12**(10), 576–580 (Oct 1969). doi:10.1145/363235.363259, <https://doi.org/10.1145/363235.363259>
- [63] Horn, J.: Arbitrary read+write via incorrect range tracking in ebpf. <https://bugs.chromium.org/p/project-zero/issues/detail?id=1454>
- [64] Horn, J.: bpf: fix 32-bit ALU op verification. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=468f6eafa6c44cb2c5d8aad35e12f06c240a812a> (2017)
- [65] Horn, J.: bpf: 32-bit RSH verification must truncate input before the ALU op. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=b799207e1e1816b09e7a5920fbb2d5fcf6edd681> (2018)
- [66] Hung, H.W., Amiri Sani, A.: Brf: Fuzzing the ebpf runtime. Proc. ACM Softw. Eng. **1**(FSE) (Jul 2024). doi:10.1145/3643778, <https://doi.org/10.1145/3643778>
- [67] Jia, J., Qin, R., Craun, M., Lukyanov, E., Bansal, A., Le, M.V., Franke, H., Jamjoom, H., Xu, T., Williams, D.: Safe and usable kernel extensions with Rex (2025). doi:10.48550/arXiv.2502.18832
- [68] Kalita, P.K., Muduli, S.K., D’Antoni, L., Reps, T., Roy, S.: Synthesizing abstract transformers. Proc. ACM Program. Lang. **6**(OOPSLA2) (Oct 2022). doi:10.1145/3563334, <https://doi.org/10.1145/3563334>

- [69] Kalita, P.K., Reps, T., Roy, S.: Synthesizing abstract transformers for reduced-product domains. In: Giacobazzi, R., Gorla, A. (eds.) *Static Analysis*. pp. 147–172. Springer Nature Switzerland, Cham (2025)
- [70] Kildall, G.A.: A unified approach to global program optimization. In: *Proceedings of the 1st ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*. pp. 194–206 (1973)
- [71] King, J.C.: Symbolic execution and program testing. *Communications of the ACM* **19**(7), 385–394 (1976)
- [72] Kline, E.: Bpf instruction set architecture (isa). <https://www.kernel.org/doc/html/latest/bpf/standardization/instruction-set.html>
- [73] Li, Y., Niu, W., Zhu, Y., Gong, J., Li, B., Zhang, X.: Fuzzing logical bugs in ebpf verifier with bound-violation indicator. In: *ICC 2023 - IEEE International Conference on Communications*. pp. 753–758 (2023). doi:10.1109/ICC45041.2023.10278676
- [74] Lian, Z., Li, Y., Chen, Z., Shan, S., Han, B., Su, Y.: ebpf-based working set size estimation in memory management. In: *2022 International Conference on Service Science (ICSS)*. pp. 188–195. IEEE (2022)
- [75] Lim, S.Y., Prasad, T., Han, X., Pasquier, T.: SafeBPF: Hardware-assisted defense-in-depth for eBPF kernel extensions (2024). doi:10.48550/arXiv.2409.07508
- [76] Lucas Leong: ZDI-20-1440: An incorrect calculation bug in the linux kernel eBPF verifier. <https://www.zerodayinitiative.com/blog/2021/1/18/zdi-20-1440-an-incorrect-calculation-bug-in-the-linux-kernel-ebpf-verifier>
- [77] Manfred Paul: CVE-2020-8835: Linux kernel privilege escalation via improper eBPF program verification. <https://www.zerodayinitiative.com/blog/2020/4/8/cve-2020-8835-linux-kernel-privilege-escalation-via-improper-ebpf-program-verification>
- [78] Manna, Pnueli, A.: *The temporal logic of reactive and concurrent systems*. Springer (1992)
- [79] Miné, A.: The octagon abstract domain. *Higher-order and symbolic computation* **19**(1), 31–100 (2006)
- [80] Miné, A.: Abstract domains for bit-level machine integer and floating-point operations. In: *WING'12 - 4th International Workshop on invariant Generation*. p. 16. Manchester, United Kingdom (Jun 2012), <https://hal.science/hal-00748094>
- [81] Miné, A.: Tutorial on static inference of numeric invariants by abstract interpretation. *Foundations and Trends® in Programming Languages* **4**(3-4), 120–372 (2017). doi:10.1561/25000000034
- [82] Moore, R.E.: *Interval analysis*. Prentice-Hall (1966)
- [83] Nakryiko, A.: BPF register bounds logic and testing improvements. <https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf-next.git/commit/?id=cd9c127069c0> (2023)

- [84] Nelson, L., Bornholt, J., Gu, R., Baumann, A., Torlak, E., Wang, X.: Scaling symbolic evaluation for automated verification of systems code with serval. In: Proceedings of the 27th ACM Symposium on Operating Systems Principles. p. 225–242. SOSP '19, Association for Computing Machinery, New York, NY, USA (2019). doi:10.1145/3341301.3359641
- [85] Nelson, L., Van Geffen, J., Torlak, E., Wang, X.: Specification and verification in the field: Applying formal methods to bpf just-in-time compilers in the linux kernel. In: Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation. OSDI'20, USENIX Association, USA (2020)
- [86] Nielson, F., Nielson, H.R., Hankin, C.: Constraint Based Analysis, pp. 141–209. Springer Berlin Heidelberg, Berlin, Heidelberg (1999). doi:10.1007/978-3-662-03811-6\_3, [https://doi.org/10.1007/978-3-662-03811-6\\_3](https://doi.org/10.1007/978-3-662-03811-6_3)
- [87] Palmiotti, V.: Kernel pwning with eBPF: a love story. <https://www.graplsecurity.com/post/kernel-pwning-with-ebpf-a-love-story>
- [88] Peng, C., Jiang, M., Wu, L., Zhou, Y.: Toss a fault to bpfchecker: Revealing implementation flaws for ebpf runtimes with differential fuzzing. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. p. 3928–3942. CCS '24, Association for Computing Machinery, New York, NY, USA (2024). doi:10.1145/3658644.3690237, <https://doi.org/10.1145/3658644.3690237>
- [89] Pierce, B.C.: Types and Programming Languages. MIT Press (2002)
- [90] Pnueli, A.: The temporal logic of programs. In: 18th Annual Symposium on Foundations of Computer Science (sfcs 1977). pp. 46–57 (1977). doi:10.1109/SFCS.1977.32
- [91] Priya, S., Besson, F., Sughrue, C., Steenvoorden, T., Fulford, J., Verbeek, F., Ravindran, B.: BeePL: Correct-by-compilation kernel extensions (2025). doi:10.48550/arXiv.2507.09883
- [92] Reps, T., Sagiv, M., Yorsh, G.: Symbolic implementation of the best transformer. In: Steffen, B., Levi, G. (eds.) Verification, Model Checking, and Abstract Interpretation. pp. 252–266. Springer Berlin Heidelberg, Berlin, Heidelberg (2004)
- [93] Reps, T., Thakur, A.: Automating abstract interpretation. In: Jobstmann, B., Leino, K.R.M. (eds.) Verification, Model Checking, and Abstract Interpretation. pp. 3–40. Springer Berlin Heidelberg, Berlin, Heidelberg (2016)
- [94] Rice, H.G.: Classes of recursively enumerable sets and their decision problems. Transactions of the American Mathematical society **74**(2), 358–366 (1953)
- [95] Rick Larabee: eBPF and Analysis of the get-rekt-linux-hardened.c Exploit for CVE-2017-16995. <https://ricklarabee.blogspot.com/2018/07/ebpf-and-analysis-of-get-rekt-linux.html>
- [96] Rinetzky, N., Shoham, S.: Property directed abstract interpretation. In: Jobstmann, B., Leino, K.R.M. (eds.) Verification, Model Checking, and Abstract Interpretation. pp. 104–123. Springer Berlin Heidelberg, Berlin, Heidelberg (2016)

- [97] Shachnai, M., Vishwanathan, H., Narayana, S., Nagarakatte, S.: Fixing latent unsound abstract operators in the ebpf verifier of the linux kernel. In: International Static Analysis Symposium. pp. 386–406. Springer (2024)
- [98] Shachnai, M., Vishwanathan, H., Narayana, S., Nagarakatte, S.: Comparing the precision of abstract operators in the ebpf verifier using differential synthesis. In: International Static Analysis Symposium. pp. 142–166. Springer (2025)
- [99] Shirokov, N.V.: XDP: 1.5 years in production. Evolution and lessons learned. In: Linux Plumbers Conference (2018)
- [100] Singh, G., Püschel, M., Vechev, M.: Fast polyhedra abstract domain. In: Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages. p. 46–59. POPL '17, Association for Computing Machinery, New York, NY, USA (2017). doi:10.1145/3009837.3009885
- [101] Solar-Lezama, A.: The sketching approach to program synthesis. In: Programming Languages and Systems: 7th Asian Symposium, APLAS 2009, Seoul, Korea, December 14–16, 2009. Proceedings 7. pp. 4–13. Springer (2009)
- [102] Solar-Lezama, A.: Program sketching. International Journal on Software Tools for Technology Transfer **15**(5), 475–495 (2013)
- [103] Starovoitov, A.: Bpf at facebook. <https://kernel-recipes.org/en/2019/talks/bpf-at-facebook/>
- [104] Sun, H., Su, Z.: Validating the eBPF verifier via state embedding. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 615–628. USENIX Association, Santa Clara, CA (Jul 2024), <https://www.usenix.org/conference/osdi24/presentation/sun-hao>
- [105] Sun, H., Xu, Y., Liu, J., Shen, Y., Guan, N., Jiang, Y.: Finding correctness bugs in ebpf verifier with structured and sanitized program. In: Proceedings of the Nineteenth European Conference on Computer Systems. p. 689–703. EuroSys '24, Association for Computing Machinery, New York, NY, USA (2024). doi:10.1145/3627703.3629562, <https://doi.org/10.1145/3627703.3629562>
- [106] Thakur, A., Reps, T.: A method for symbolic computation of abstract operations. In: Madhusudan, P., Seshia, S.A. (eds.) Computer Aided Verification. pp. 174–192. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
- [107] Toubhans, A., Chang, B.Y.E., Rival, X.: Reduced product combination of abstract domains for shapes. In: International Workshop on Verification, Model Checking, and Abstract Interpretation. pp. 375–395. Springer (2013)
- [108] Vishwanathan, H., Shachnai, M., Narayana, S., Nagarakatte, S.: Sound, precise, and fast abstract interpretation with tristate numbers. In: Proceedings of the 20th IEEE/ACM International Symposium on Code Generation and Optimization. p. 254–265. CGO '22, IEEE Press (2022). doi:10.1109/CGO53902.2022.9741267

- [109] Vishwanathan, H., Shachnai, M., Narayana, S., Nagarakatte, S.: Agni: Verifying the Verifier (eBPF Range Analysis Verification). <https://github.com/bpfverif/ebpf-range-analysis-verification-cav23> (2023)
- [110] Vishwanathan, H., Shachnai, M., Narayana, S., Nagarakatte, S.: Verifying the verifier: ebpf range analysis verification. In: Computer Aided Verification: 35th International Conference, CAV 2023, Paris, France, July 17–22, 2023, Proceedings, Part III. p. 226–251. Springer-Verlag, Berlin, Heidelberg (2023). doi:10.1007/978-3-031-37709-9\_12
- [111] Vishwanathan, H., Shachnai, M., Narayana, S., Nagarakatte, S.: Automatic synthesis of abstract operators for ebpf. In: Proceedings of the 3rd Workshop on EBPF and Kernel Extensions. p. 15–21. eBPF '25, Association for Computing Machinery, New York, NY, USA (2025). doi:10.1145/3748355.3748361, <https://doi.org/10.1145/3748355.3748361>
- [112] Wang, X., Lazar, D., Zeldovich, N., Chlipala, A., Tatlock, Z.: Jitk: A trustworthy in-kernel interpreter infrastructure. In: Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation. p. 33–47. OSDI'14, USENIX Association, USA (2014)
- [113] Yang, J., Chen, L., Bai, J.: Redis automatic performance tuning based on ebpf. In: 2022 14th International Conference on Measuring Technology and Mechatronics Automation (ICMTMA). pp. 671–676. IEEE (2022)